

Certificate-Carrying CAM Compilers

A Four-Chapter Textbook on Meaning, Intermediate Representations, Verification, and CNC
Execution

Prepared for the Dropcut Studio / Makera Z1 project

Pedagogical second edition - August 2026

Contents

Preface	i
The running example	ii
How to read definitions	ii
Audience and prerequisites	ii
Safety scope	iii
Notational conventions	iii
The four questions	iii
1 Meaning Before Motion: What a CAM Compiler Is	1
1.1 From operator request to physical behavior	2
1.1.1 The Cyber-Physical Compiler	3
1.1.1.1 From strings to physical behavior	3
1.1.1.2 Source meaning is usually a set	5
1.1.1.3 Why the target is not text	5
1.1.1.4 Three kinds of correctness	6
1.1.1.5 The semantic waist	6
1.1.1.6 A first design rule	7
1.1.1.7 Worked example: a rectangular pocket	7
1.1.1.8 Exercises	7
1.1.2 Manufacturing Intent as a Specification	7
1.1.2.1 Specifications before strategies	7
1.1.2.2 Required, permitted, and forbidden regions	8
1.1.2.3 Features as semantic handles	8
1.1.2.4 Intent is relational and partial	9
1.1.2.5 Planning as witness synthesis	9
1.1.2.6 Contracts for common operations	10
1.1.2.7 Worked example: rough then finish	10
1.1.2.8 Design rules	10
1.1.2.9 Exercises	10
1.2 The mathematical and geometric vocabulary we actually need	11
1.2.1 Mathematical Foundations: Sets, Relations, Metrics, and Orders	12
1.2.1.1 Sets and characteristic predicates	12
1.2.1.2 Relations instead of functions	12
1.2.1.3 Metrics and neighborhoods	13
1.2.1.4 Equivalence, preorder, and refinement	13
1.2.1.5 Lattices of information	14
1.2.1.6 Monotonicity and fixed points	14
1.2.1.7 Tolerance is not equality	14

1.2.1.8	Worked example: an uncertainty box	15
1.2.1.9	Exercises	15
1.2.2	Geometry, Frames, and Units	15
1.2.2.1	Frames are part of the type	15
1.2.2.2	Rigid transformations	16
1.2.2.3	Three-axis simplification and future proofing	16
1.2.2.4	Dimensional types	16
1.2.2.5	Tools and assemblies	17
1.2.2.6	Uncertain transforms	17
1.2.2.7	Worked example: work-frame travel	17
1.2.2.8	Design rules	18
1.2.2.9	Exercises	18
1.3	Four semantic views, nondeterminism, and refinement	18
1.3.1	One cut command viewed four ways	19
1.3.2	Four Semantic Views	20
1.3.2.1	Denotational semantics	21
1.3.2.2	Operational semantics	21
1.3.2.3	Axiomatic semantics	21
1.3.2.4	Trace and temporal semantics	22
1.3.2.5	Consistency among views	22
1.3.2.6	Which view to use	22
1.3.2.7	Worked example: probe semantics	23
1.3.2.8	Exercises	23
1.3.3	Nondeterminism, Approximation, and Refinement	23
1.3.3.1	Sources of nondeterminism	23
1.3.3.2	Exact preservation	23
1.3.3.3	Trace refinement	24
1.3.3.4	Bounded geometric refinement	24
1.3.3.5	Quantitative error composition	24
1.3.3.6	Approximation direction	24
1.3.3.7	Unknown is a valid result	25
1.3.3.8	Worked example: arc lowering	25
1.3.3.9	Design rules	26
1.3.3.10	Exercises	26
1.4	Chapter synthesis: specifying the running pocket	26
1.4.1	A student-readable refinement statement	27
1.4.2	Common misconceptions	27
1.4.3	Chapter exercises	27
2	A Language That Can Be Reasoned About	28
2.1	Staged JavaScript and the IR ladder	28
2.1.1	JavaScript as a Staged Macro Language	29
2.1.1.1	The convenience and danger of embedded JavaScript	29
2.1.1.2	Two languages, not one	30
2.1.1.3	Capability design	30
2.1.1.4	Determinism and declared inputs	30
2.1.1.5	Scope combinators	31
2.1.1.6	Mutability at the boundary	31
2.1.1.7	Worked example: staged pocket program	31
2.1.1.8	Design rules	32

2.1.1.9	Exercises	32
2.1.2	A Multi-Level IR Architecture	32
2.1.2.1	Why one IR is not enough	32
2.1.2.2	Authoring AST	33
2.1.2.3	Elaborated Plan IR	33
2.1.2.4	Manufacturing Intent IR	33
2.1.2.5	Geometric Toolpath IR	34
2.1.2.6	Scheduled Program IR	34
2.1.2.7	Machine IR	34
2.1.2.8	Controller IR	34
2.1.2.9	Serialized Job Bundle	35
2.1.2.10	Legality	35
2.1.2.11	Validation is evidence, not a dialect	35
2.1.2.12	Exercises	36
2.2	Composable paths, effectful actions, and explicit state	36
2.2.1	Paths and Curves as Composable Objects	37
2.2.1.1	Paths as arrows	37
2.2.1.2	Geometry versus parameterization	38
2.2.1.3	Segment algebra	38
2.2.1.4	Exact joins and approximate joins	38
2.2.1.5	Reversal and orientation	39
2.2.1.6	Path continuity is not enough	39
2.2.1.7	Worked example: arc witness	39
2.2.1.8	Design rules	40
2.2.1.9	Exercises	40
2.2.2	Actions, Effects, and Indexed Commands	40
2.2.2.1	A path is not an action	40
2.2.2.2	Why ordinary composition fails	40
2.2.2.3	CNC effects	41
2.2.2.4	Indexed commands	41
2.2.2.5	Limits of type-level safety	42
2.2.2.6	Algebraic effects versus monolithic state	42
2.2.2.7	Worked example: probe then offset	42
2.2.2.8	Exercises	42
2.2.3	State Tokens, SSA, and Ordering	42
2.2.3.1	The machine state as a linear resource	43
2.2.3.2	Relation to SSA	43
2.2.3.3	Multiple resources	43
2.2.3.4	Probe values and control flow	43
2.2.3.5	Effect summaries	44
2.2.3.6	Linear and affine use	44
2.2.3.7	Worked example: preventing an unsafe reorder	44
2.2.3.8	Exercises	44
2.3	Provenance, identity, and an extensible API	45
2.3.1	Provenance, Identity, and Reproducibility	45
2.3.1.1	Provenance is semantic data	45
2.3.1.2	A provenance chain	46
2.3.1.3	Content addressing	46
2.3.1.4	Reproducible passes	46
2.3.1.5	Machine and firmware identity	47

2.3.1.6	Provenance through optimization	47
2.3.1.7	Worked example: cache key	47
2.3.1.8	Exercises	47
2.3.2	Designing an Extensible CAM API	48
2.3.2.1	Separate stable concepts from plugins	48
2.3.2.2	Kernel independence	48
2.3.2.3	Explicit contexts	48
2.3.2.4	Result types and diagnostics	49
2.3.2.5	Escape hatches	49
2.3.2.6	API layers	49
2.3.2.7	Law-driven API testing	50
2.3.2.8	Worked example: compiling explicitly	50
2.3.2.9	Exercises	50
2.4	Chapter synthesis: the pocket through the language boundary	50
2.4.1	The same pocket at three IR levels	51
2.4.2	Why several IRs are easier than one universal IR	52
2.4.3	Three representation invariants	52
2.4.4	Common misconceptions	52
2.4.5	Chapter exercises	53
3	Passes, Proof Obligations, and Certificates	54
3.1	Pass contracts, legality, and elaboration	54
3.1.1	Complete example: validating arc linearization	55
3.1.2	Pass Contracts and Translation Validation	55
3.1.2.1	A pass is more than a function	55
3.1.2.2	Classes of pass relation	56
3.1.2.3	Verified compiler versus validating compiler	57
3.1.2.4	Checker soundness	57
3.1.2.5	Pass descriptors	57
3.1.2.6	Local and global validation	58
3.1.2.7	Worked example: arc linearization validator	58
3.1.2.8	Exercises	58
3.1.3	Elaboration and Static Checking	59
3.1.3.1	What elaboration does	59
3.1.3.2	Static semantics	59
3.1.3.3	Domain checking	59
3.1.3.4	Frame graph consistency	60
3.1.3.5	Defaults and provenance	60
3.1.3.6	Normal forms	60
3.1.3.7	Static versus physical checks	60
3.1.3.8	Worked example: resolving a pocket	61
3.1.3.9	Exercises	61
3.2	From intent to toolpaths, machine actions, controller IR, and bytes	61
3.2.1	Planning Features into Toolpaths	62
3.2.1.1	Planner input and output	62
3.2.1.2	Coverage and offset planning	62
3.2.1.3	Drop-cutter planning	63
3.2.1.4	Constant-scallop intent	63
3.2.1.5	Directionality and phase	63
3.2.1.6	Witness design	64

3.2.1.7	Worked example: pocket coverage	64
3.2.1.8	Exercises	64
3.2.2	Scheduling, Entry, and Linking	64
3.2.2.1	Planning produces islands; scheduling produces a program	64
3.2.2.2	Evolving stock state	65
3.2.2.3	Entry strategies	65
3.2.2.4	Traverse semantics	65
3.2.2.5	Configuration space	66
3.2.2.6	Precedence constraints	66
3.2.2.7	Worked example: two islands	66
3.2.2.8	Exercises	66
3.2.3	Machine Lowering and Capability Resolution	67
3.2.3.1	Machine profiles as semantic contracts	67
3.2.3.2	Algebraic capabilities	67
3.2.3.3	Coordinate resolution	67
3.2.3.4	Dynamic and geometric limits	67
3.2.3.5	Safe expansion of abstract operations	68
3.2.3.6	Unsupported and unknown behavior	68
3.2.3.7	Worked example: rapid lowering	68
3.2.3.8	Exercises	68
3.2.4	Controller IR and Modal Semantics	68
3.2.4.1	Modal syntax is compressed state	69
3.2.4.2	Controller state	69
3.2.4.3	Blocks are evaluated as units	69
3.2.4.4	Initial state	69
3.2.4.5	Modal compression	69
3.2.4.6	Probes and result semantics	70
3.2.4.7	Worked example: stale incremental mode	70
3.2.4.8	Exercises	70
3.2.5	Serialization, Round Trips, and Exact Bytes	70
3.2.5.1	Serialization is a compiler pass	70
3.2.5.2	Deterministic formatting	71
3.2.5.3	Rounding bounds	71
3.2.5.4	Parse-back validation	71
3.2.5.5	Injection through comments and metadata	72
3.2.5.6	Exact-byte binding	72
3.2.5.7	Safety epilogue	72
3.2.5.8	Worked example: round-trip checker	72
3.2.5.9	Exercises	72
3.2.6	Optimization Without Breaking Meaning	72
3.2.6.1	Optimization is constrained refinement	73
3.2.6.2	Common optimizations	73
3.2.6.3	Peephole validation	73
3.2.6.4	Reordering and commutativity	73
3.2.6.5	Objective evidence	74
3.2.6.6	Phase ordering	74
3.2.6.7	Worked example: arc fitting	74
3.2.6.8	Exercises	74
3.3	Assertions, invariants, weakest preconditions, and abstract interpretation	75
3.3.1	Do not confuse these statement kinds	76

3.3.2	Contracts, Hoare Logic, and Weakest Preconditions	76
3.3.2.1	Vocabulary matters	76
3.3.2.2	Hoare triples	77
3.3.2.3	Sequential composition	77
3.3.2.4	Weakest preconditions	77
3.3.2.5	CNC weakest-precondition example	78
3.3.2.6	Invariant induction	78
3.3.2.7	Separation of resources	78
3.3.2.8	Design by contract in APIs	79
3.3.2.9	Exercises	79
3.3.3	Abstract Interpretation for Machine Programs	79
3.3.3.1	Simulating one trace is not analyzing all traces	79
3.3.3.2	Abstract machine state	79
3.3.3.3	Interval transfer	80
3.3.3.4	Modal-state analysis	80
3.3.3.5	Joins and loss of precision	80
3.3.3.6	Reduced products	81
3.3.3.7	Unknown raw effects	81
3.3.3.8	Worked example: final spindle state	81
3.3.3.9	Exercises	81
3.4	Robust geometry, swept volumes, and quantitative error	81
3.4.1	Two claims, two approximation directions	82
3.4.2	Robust Numerics and Computational Geometry	83
3.4.2.1	Why geometric bugs are discontinuous	83
3.4.2.2	Orientation predicates	83
3.4.2.3	Exact topology, approximate geometry	83
3.4.2.4	Interval arithmetic	84
3.4.2.5	The dependency problem	84
3.4.2.6	Conservative meshing	84
3.4.2.7	Tolerance and topology	85
3.4.2.8	Worked example: collision of two nearly touching paths	85
3.4.2.9	Exercises	85
3.4.3	Swept Volumes, Stock Models, and Collision Proofs	85
3.4.3.1	Swept volume	85
3.4.3.2	Stock evolution	86
3.4.3.3	Tool versus holder	86
3.4.3.4	Dexel and voxel models	86
3.4.3.5	A grid size is not a guarantee	87
3.4.3.6	Configuration-space collision	87
3.4.3.7	Continuous path checking	87
3.4.3.8	Worked example: rapid through stock	87
3.4.3.9	Exercises	88
3.4.4	Error Budgets and Quantitative Refinement	88
3.4.4.1	A budget is a proof structure	88
3.4.4.2	Typed metrics	88
3.4.4.3	Budget allocation	88
3.4.4.4	Correlation and worst-case composition	89
3.4.4.5	Pass amplification	89
3.4.4.6	Residual-driven refinement	89
3.4.4.7	Runtime budgets	89

3.4.4.8	Worked example: final surface error	89
3.4.4.9	Exercises	90
3.5	Certificate schemas, trusted checkers, and proof-carrying CAM	90
3.5.1	Certificate Schemas and Proof Graphs	90
3.5.1.1	A certificate is a graph of propositions	91
3.5.1.2	Structured claims	91
3.5.1.3	Results and methods are orthogonal	92
3.5.1.4	Assumption records	92
3.5.1.5	Evidence types	92
3.5.1.6	Claim granularity	93
3.5.1.7	Completeness policy	93
3.5.1.8	Worked example: false evidence reuse	93
3.5.1.9	Exercises	94
3.5.2	Trusted Checkers and Proof-Carrying CAM	94
3.5.2.1	The producer is complicated	94
3.5.2.2	Trusted computing base	94
3.5.2.3	Independence	95
3.5.2.4	Proof objects versus recomputation	95
3.5.2.5	Checker resource bounds	95
3.5.2.6	Compositional certificates	95
3.5.2.7	Mechanization strategy	96
3.5.2.8	Worked example: validating a solver schedule	96
3.5.2.9	Exercises	96
3.6	Chapter synthesis: a certificate graph for the pocket	96
3.6.1	Why evidence cannot be reused casually	97
3.6.2	A practical pass interface	97
3.6.3	Common misconceptions	98
3.6.4	Chapter exercises	98
4	Execution, Optimization, and Engineering the Whole System	99
4.1	Temporal controller semantics and runtime assurance	99
4.1.1	Temporal Semantics and Controller Protocols	100
4.1.1.1	The protocol is part of the compiler target	100
4.1.1.2	States and transitions	100
4.1.1.3	Safety properties	101
4.1.1.4	Liveness properties	101
4.1.1.5	Atomic admission and execution	101
4.1.1.6	Ambiguous timeout	102
4.1.1.7	Message streams and loss	102
4.1.1.8	Model checking	102
4.1.1.9	Worked example: compound command classification	102
4.1.1.10	Exercises	102
4.1.2	Runtime Assurance and Physical Assumptions	102
4.1.2.1	Conditional assurance	103
4.1.2.2	Hash-bound execution	103
4.1.2.3	Runtime checks	103
4.1.2.4	Runtime assurance monitor	104
4.1.2.5	Stopping distance and latency	104
4.1.2.6	Physical model validation	104
4.1.2.7	Degradation policies	104

4.1.2.8	Worked example: changed G54	105
4.1.2.9	Design rules	105
4.1.2.10	Exercises	105
4.2	Operations research inside a checked feasible set	105
4.2.1	State the optimization result honestly	106
4.2.2	Feasibility Before Optimality	106
4.2.2.1	The feasible set comes first	107
4.2.2.2	Why safety is not a penalty	107
4.2.2.3	Decision variables	107
4.2.2.4	Constraint generation	108
4.2.2.5	Feasible, optimal, and near-optimal	108
4.2.2.6	Lexicographic objectives	108
4.2.2.7	Worked example: roughing schedule	109
4.2.2.8	Exercises	109
4.2.3	Sequencing and Precedence-Constrained Routing	109
4.2.3.1	From nearest neighbor to routing	109
4.2.3.2	Node and state expansion	109
4.2.3.3	Precedence graph	109
4.2.3.4	Asymmetric transition costs	110
4.2.3.5	Mixed-integer model sketch	110
4.2.3.6	Stock-dependent edges	110
4.2.3.7	Schedule witness	110
4.2.3.8	Worked example: four contours	111
4.2.3.9	Exercises	111
4.2.4	Free-Space Linking and Configuration Space	111
4.2.4.1	The link planner's problem	111
4.2.4.2	Baseline: certified retract policy	111
4.2.4.3	Visibility graphs and A*	112
4.2.4.4	Distance fields and Eikonal equations	112
4.2.4.5	Clearance-aware objectives	112
4.2.4.6	Dynamic feasibility	112
4.2.4.7	Link certificate	112
4.2.4.8	Worked example: low retract	113
4.2.4.9	Exercises	113
4.2.5	Feed Scheduling and Optimal Control	113
4.2.5.1	Path parameterization	113
4.2.5.2	Time-optimal path parameterization	113
4.2.5.3	Curvature and contour speed	114
4.2.5.4	Process constraints	114
4.2.5.5	Controller realization	114
4.2.5.6	Feed schedule witness	114
4.2.5.7	Worked example: circular path	115
4.2.5.8	Exercises	115
4.2.6	Robust and Multiobjective Optimization	115
4.2.6.1	Nominal optimization is brittle	115
4.2.6.2	Chance constraints	115
4.2.6.3	Scenario methods	116
4.2.6.4	Multiple objectives	116
4.2.6.5	Margin as an objective	116
4.2.6.6	Sensitivity analysis	116

4.2.6.7	Worked example: two link candidates	116
4.2.6.8	Exercises	116
4.3	Implementation architecture, the complete example, and testing	117
4.3.1	Package Architecture for a TypeScript CAM Compiler	117
4.3.1.1	Architectural boundaries	117
4.3.1.2	A semantics package	118
4.3.1.3	Checker packages	118
4.3.1.4	Artifact store	120
4.3.1.5	Worker boundaries	120
4.3.1.6	CLI and Studio parity	120
4.3.1.7	Dependency rules	120
4.3.1.8	Versioning	121
4.3.1.9	Worked package layout	121
4.3.1.10	Exercises	121
4.3.2	An End-to-End Certified Pocket Example	122
4.3.2.1	Source program	122
4.3.2.2	Authoring and elaboration	122
4.3.2.3	Intent IR	123
4.3.2.4	Planning	123
4.3.2.5	Scheduling and linking	123
4.3.2.6	Machine lowering	124
4.3.2.7	Controller lowering and serialization	124
4.3.2.8	Geometric final validation	124
4.3.2.9	Executable bundle	124
4.3.2.10	Runtime discharge	125
4.3.2.11	Failure example	125
4.3.2.12	Exercises	125
4.3.3	Testing: Properties, Fuzzing, Differential, and Metamorphic Methods	125
4.3.3.1	Tests are not proofs, but semantics improve tests	125
4.3.3.2	Unit tests	125
4.3.3.3	Property-based tests	126
4.3.3.4	Metamorphic testing	126
4.3.3.5	Differential testing	126
4.3.3.6	Fuzzing parsers and protocols	127
4.3.3.7	Numerical adversaries	127
4.3.3.8	Checker mutation testing	127
4.3.3.9	Physical test ladder	128
4.3.3.10	Exercises	128
4.4	The Dropcut/Z1 lessons and a staged migration path	128
4.4.1	The Dropcut/Z1 Case Study: What the Defects Teach	129
4.4.1.1	Scope of the reviewed snapshot	129
4.4.1.2	Case A: evidence for one claim minted another claim	129
4.4.1.3	Case B: same-realm script execution was described honestly but remained unsafe	129
4.4.1.4	Case C: free-text command classification used lexical prefixes instead of seman-	
	tics	130
4.4.1.5	Case D: a packed endpoint key changed topology	130
4.4.1.6	Cross-boundary findings	131
4.4.1.7	What should be preserved	131
4.4.1.8	Counterexample-driven architecture	131
4.4.1.9	Exercises	132

4.4.2	A Staged Migration Roadmap	132
4.4.2.1	Principles for migration	132
4.4.2.2	Phase 0: stop overclaiming	132
4.4.2.3	Phase 1: harden language and identity	132
4.4.2.4	Phase 2: executable semantics and final-byte validation	133
4.4.2.5	Phase 3: pass witnesses	133
4.4.2.6	Phase 4: robust geometric kernel	133
4.4.2.7	Phase 5: controller state machine	133
4.4.2.8	Phase 6: runtime assurance	134
4.4.2.9	Phase 7: optimization maturity	134
4.4.2.10	Operating-mode gates	134
4.4.2.11	Definition of done	135
4.4.2.12	Final perspective	135
4.4.2.13	Exercises	135
4.5	A failure trace that filenames cannot prevent	136
4.6	Chapter synthesis: from certificate to controlled execution	136
4.6.1	Feasibility before optimization in the running pocket	136
4.6.2	A small trusted core	137
4.6.3	Common misconceptions	137
4.6.4	Capstone exercises	137
A	Mathematical and Notational Reference	138
A.0.0.1	Sets and functions	138
A.0.0.2	Logic	138
A.0.0.3	Orders and lattices	139
A.0.0.4	Metric spaces	139
A.0.0.5	Geometry and morphology	139
A.0.0.6	Paths and trajectories	140
A.0.0.7	Error and uncertainty	140
A.0.0.8	Optimization	141
B	Reference IR and Certificate Schemas	142
B.0.0.1	Primitive identities and units	142
B.0.0.2	Frames and transforms	142
B.0.0.3	Geometry and setup	143
B.0.0.4	Paths	144
B.0.0.5	Intent IR	144
B.0.0.6	Toolpath and schedule IR	145
B.0.0.7	Canonical commands	146
B.0.0.8	Machine and controller IR	147
B.0.0.9	Diagnostics and provenance	147
B.0.0.10	Quantitative claims	148
B.0.0.11	Certificate graph	149
C	Checker Algorithms	151
C.0.0.1	Certificate graph checker	151
C.0.0.2	Translation-validation skeleton	151
C.0.0.3	Abstract-state proof checker	152
C.0.0.4	Continuous collision checker	152
C.0.0.5	Required-removal checker	153

C.0.0.6	Modal parse-back checker	153
C.0.0.7	Schedule checker	153
C.0.0.8	Error-budget composition	154
C.0.0.9	Runtime authorization checker	154
D	A Temporal Controller Model	155
D.0.0.1	Variables	155
D.0.0.2	Initial state	155
D.0.0.3	Connection and identification	156
D.0.0.4	Upload	156
D.0.0.5	Authorization and start	156
D.0.0.6	Hold, resume, abort, and alarm	157
D.0.0.7	Next-state relation	157
D.0.0.8	Invariants	158
D.0.0.9	Temporal properties	158
E	Selected Solution Sketches	159
E.1	Chapter 1, Exercise 3	159
E.2	Chapter 1, Exercise 4	159
E.3	Chapter 2, Exercise 3	159
E.4	Chapter 2, Exercise 5	159
E.5	Chapter 3, Exercise 1	160
E.6	Chapter 3, Exercise 4	160
E.7	Chapter 4, Exercise 1	160
E.8	Chapter 4, Exercise 4	160
F	Glossary	161
G	Guided Reading and Project Tracks	170
G.1	Compiler and language track	170
G.2	Verification and geometry track	170
G.3	Controller and runtime track	170
G.4	Planning and optimization track	170
G.5	Twelve-week implementation course	170
	References	172
	Source Snapshot and Reproducibility Notes	176

Preface

A first encounter with compiler theory can feel backward to a computer-aided manufacturing (CAM) programmer. The machine is sitting on the bench. The immediate problem appears to be concrete: generate a geometric path, encode it as G-code - the RS274-style family of languages commonly used to command computer numerical control (CNC) machines - send the file, and watch the mill. Why begin with meanings, relations, invariants, or proof objects?

Because every one of those apparently abstract ideas answers a practical question that otherwise remains implicit:

- What exactly did the operator ask the software to accomplish?
- Which differences between two toolpaths matter, and which are merely implementation choices?
- What must remain true when a high-level operation is lowered through several intermediate forms?
- What evidence justifies a green safety indicator?
- How can the host know that the controller is executing the exact bytes that were checked?
- Where may an optimizer change the program, and where must it preserve a hard constraint?

This edition is organized around those questions rather than around forty small topics. It has **four large chapters**:

1. **Meaning before motion.** We learn to state machining intent and physical behavior precisely.
2. **A language that can be reasoned about.** We design the JavaScript API and the intermediate representations that carry that meaning.
3. **Passes, proof obligations, and certificates.** We build a compiler in which every transformation has a checkable contract.
4. **Execution, optimization, and the whole system.** We connect verified artifacts to a real controller, optimize only inside a checked feasible set, and turn the theory into an engineering roadmap.

Before proceeding, several first-contact terms prevent avoidable ambiguity:

- A **specification** states what outcomes or behaviors are required, permitted, or forbidden without necessarily prescribing how to produce them.
- **Semantics** is a systematic account of meaning: what a program denotes or how it behaves, not merely whether its syntax is valid.
- An **artifact** is an immutable compiler input or output whose identity can be recorded: source, abstract syntax tree (AST), intermediate representation (IR), geometry, profile, bytes, evidence, or certificate.
- A **claim** is a precise proposition about a precise subject artifact; **evidence** is machine-checkable data used by a checker to establish that claim.
- An **invariant** is a property that holds initially and is preserved by every relevant transition, so it holds in every reachable state covered by the model.
- A **certificate** is a machine-checkable package of precise claims, assumptions, evidence, dependencies, checker identities, and artifact identities. It is not a generic declaration that a job is “safe.”

In this book, **verification** is reserved for establishing a named proposition about a named artifact under named assumptions by a stated method. Visual inspection, simulation, and tests remain valuable, but they receive their own more limited descriptions.

The reorganization is deliberate. New terms are introduced only after the problem that requires them has appeared. Each important term is then defined, used in a small example, and used again in the running CNC example. Counterexamples are included because many safety failures arise from a concept that seemed obvious until one adversarial input exposed the missing condition.

The running example

The same job appears throughout the book. We begin with a rectangular aluminum blank measuring $60\text{ mm} \times 40\text{ mm} \times 8\text{ mm}$. The operator requests a $30\text{ mm} \times 20\text{ mm}$ pocket, nominally 4 mm deep, followed by a finishing pass. A 3.175 mm end mill is loaded. Two clamps occupy modeled regions near the long edges. The work coordinate system is established by probing and is uncertain by a small, recorded amount. The target permits a roughing allowance before the finishing operation but forbids cutting beyond a final geometric tolerance.

This example is intentionally ordinary. A textbook architecture is useful only if it can explain an ordinary pocket without hand-waving. Across the chapters we will express the request as a specification, construct an inert plan, lower it through several IRs, derive toolpaths, schedule safe links, serialize controller bytes, generate a certificate graph, bind the job to the controller, and optimize the schedule without weakening the safety argument.

A second, smaller example accompanies it: a probing operation that measures a surface and then defines a work offset. The pocket shows geometric and process reasoning; the probe shows state, uncertainty, and controller protocol reasoning.

How to read definitions

A definition in this book has three parts:

1. **Motivation:** the concrete ambiguity or failure that makes the concept necessary.
2. **Definition:** the precise meaning used in the rest of the text.
3. **Application:** at least one small example and one CAM example.

When a familiar word receives a narrower technical meaning, that narrower meaning is stated explicitly. For example, *verification* is not used as a synonym for “the preview looked plausible.” It means establishing a named proposition about a named artifact under named assumptions by a stated method.

Audience and prerequisites

The primary reader knows JavaScript or TypeScript and the basics of computer numerical control (CNC) operation. Familiarity with vectors, matrices, and elementary logic helps, but the necessary mathematical language is introduced in Chapter 1. No prior course in formal methods, compilers, computational geometry, or operations research is assumed.

A mathematically experienced reader may move quickly through the definitions and concentrate on how the theories compose. A practical CAM developer should work through the examples because the distinction between a path, a trajectory, a controller command, and a physical execution is central to the architecture.

Safety scope

This book is an engineering and research text. It does not certify a machine, controller, workholding setup, tool, or job. Every software result is conditional on physical assumptions: calibration, tool dimensions, fixture geometry, work-coordinate registration, firmware behavior, material behavior, stopping dynamics, and sensor validity. The purpose of the architecture is to make those assumptions explicit and to prevent a weaker check from being presented as a stronger guarantee.

Notational conventions

- I usually denotes manufacturing intent.
- P usually denotes a program or plan.
- σ denotes a state; Σ is a set of states.
- τ denotes an execution trace.
- $\llbracket x \rrbracket$ denotes the meaning, or **denotation**, of x : the mathematical object assigned to it by a semantic model.
- ε denotes a declared approximation or uncertainty bound under a named metric.
- Lengths are in millimeters and angles are in radians unless a dimensional type says otherwise.
- A **path** is a geometric curve indexed by abstract progress; a **trajectory** is a pose or configuration indexed by time. Chapter 2 develops the distinction.
- An **execution trace** is a sequence of states and observable events produced while the system runs.
- Code examples are pedagogical TypeScript. They emphasize semantics and contracts rather than library-specific syntax.

The four questions

Before continuing, keep four questions visible:

1. **Meaning:** What set of outcomes is acceptable?
2. **Representation:** Which facts must the IR make explicit?
3. **Evidence:** Why should a checker believe that a pass preserved the relevant meaning?
4. **Execution:** Why should the operator believe that the certified artifact is the one the machine will execute under the checked setup?

Every major design decision in the book answers one of these questions.

Chapter 1

Meaning Before Motion: What a CAM Compiler Is

Chapter goal. Build a precise mental model of the problem before designing APIs or algorithms. By the end of the chapter, you should be able to state what a machining intent means, distinguish paths from trajectories and physical executions, use frames and units without ambiguity, and explain semantic refinement in ordinary language and in a compact mathematical form.

A student often begins with G-code because it is visible. The file contains coordinates and feeds; the controller accepts it; therefore the file appears to be the program's meaning. This chapter reverses that view. G-code is one late encoding. The meaning begins with the requested workpiece and the permitted process behavior.

The chapter proceeds from concrete to abstract and back again:

1. Start with the pocket request and ask what counts as success.
2. Introduce only the mathematical vocabulary needed to answer that question.
3. Examine four complementary ways to describe program meaning.
4. Use refinement to connect a high-level request to a lower-level implementation.
5. Reconstruct the pocket specification using the new vocabulary.

First definition - semantics. Semantics is the meaning assigned to a language construct or program. In CAM, meaning can concern acceptable final stock, step-by-step machine transitions, preconditions and postconditions, or whole execution traces.

First definition - refinement. Refinement is the relation in which a more concrete implementation resolves choices or adds detail without introducing behavior forbidden by the more abstract specification.

The most important habit is to separate three questions that are easy to conflate:

- **What is required?** For example, material in a pocket interior must be removed.
- **What is permitted?** For example, the planner may choose raster or offset clearing.
- **What is forbidden?** For example, the cutting sweep may not penetrate protected material beyond tolerance.

1.1 From operator request to physical behavior

A CAM compiler needs a source meaning before it can have a correctness theorem. The problem is that an operator usually describes a result while a machine executes a sequence. A rectangular pocket is a result; thousands of axis motions are one possible sequence. The first section introduces the terms that let us talk about both without confusing them.

Definition - specification. A specification states a condition that implementations must satisfy. It may describe acceptable inputs and outputs, permitted traces, geometric tolerances, resource constraints, or temporal behavior. A specification is about *what must hold*; an algorithm is one way to make it hold.

Small example. “Return the input elements in nondecreasing order” specifies sorting without choosing quicksort or mergesort.

CAM example. “Remove the pocket interior while preserving its wall and floor within tolerance” specifies a result without choosing raster or offset clearing.

Definition - manufacturing intent. A manufacturing intent is a specification of acceptable physical outcomes and process constraints. It deliberately leaves implementation choices open when the operator does not care which choice is made.

Small example. “Produce a circular hole of diameter 6 mm through the stock” is intent. “Execute these 240 helical line segments” is an implementation.

CAM example. The pocket intent states the required depth, tolerances, protected regions, roughing allowance, tool constraints, and perhaps surface-finish requirements. It does not require a particular stepover pattern unless the pattern itself matters.

Definition - artifact. An artifact is an immutable input or output of a compiler stage: source text, an abstract syntax tree (AST), an intermediate representation (IR) module, a tool model, a machine profile, a mesh, serialized G-code bytes, evidence, or a certificate. Treating these as artifacts makes identity and dependency explicit later.

Small example. A JSON machine profile and its SHA-256 digest form a versioned artifact.

CAM example. A gouge claim must name the exact target mesh, tool assembly, trajectory, and checker configuration to which it applies.

Definition - execution trace. An execution trace is a finite or infinite sequence of states and observable events produced while the system runs. A trace can include axis poses, modal state, spindle changes, probe contacts, upload acknowledgements, alarms, and stock changes.

Small example. `connected -> uploaded -> authorized -> started -> stopped` is a protocol trace.

CAM example. Two executions of the same nominal path may differ because a probe triggers at different positions within its uncertainty interval.

Definition - assumption. An assumption is a fact required by a claim but not established by the evidence for that claim. Assumptions must be visible because a proof is only as applicable as its assumptions.

Small example. A numeric proof may assume IEEE-compliant directed rounding.

CAM example. A fixture-clearance claim may assume that the modeled clamp geometry and the actual clamp placement agree within a recorded tolerance.

Definition - witness. A witness is producer-supplied data that demonstrates how an implementation satisfies a specification. A checker must validate the witness; the producer's assertion alone is not enough.

Small example. A topological sort is a witness that a schedule respects a precedence graph.

CAM example. A pocket planner may return toolpaths plus a coverage decomposition showing which path region is intended to remove each required stock cell.

Definition - planner. A planner is a producer that chooses one implementation from the alternatives allowed by an intent specification. It may be heuristic; trust comes from checking its result and witness, not from calling it a planner.

Small example. A route planner selects one route among many that connect the same endpoints.

CAM example. A pocket planner selects offsets, raster passes, or adaptive motion and emits a coverage witness.

Definition - canonical command. A canonical command is a machine-oriented action with explicit machining meaning but without the accidental syntax and modal compression of a particular controller dialect - a versioned syntax and operational interpretation.

Small example. `Traverse(path)` is canonical; `G0 X10 Y5` is one possible encoding under a specific initial modal state.

Definition - semantic waist. A semantic waist is a compact middle language that many high-level producers and many low-level backends can share. It preserves the distinctions required for correctness while avoiding target-specific syntax.

CAM example. Pocket, surfacing, drilling, and engraving planners can all emit the same canonical actions, while Z1, LinuxCNC, and other postprocessors lower those actions differently.

1.1.1 The Cyber-Physical Compiler

Section goals. By the end of this section, the reader should be able to distinguish a CAM compiler from a G-code generator, state an end-to-end refinement theorem, and identify the major semantic boundaries between intent and physical execution.

1.1.1.1 From strings to physical behavior

A naive description of a CAM program is:

```
geometry in -> G-code out
```

This description omits every fact that makes the software difficult. It does not say what the geometry means, what result the operator requested, how the tool and holder occupy space, how the work coordinate system relates to the machine, whether the controller interprets rapids as coordinated lines, whether a feed rate is feasible, or what should happen after an interrupted upload. It also gives no place to state correctness.

A better semantic stack is shown in the figure below.

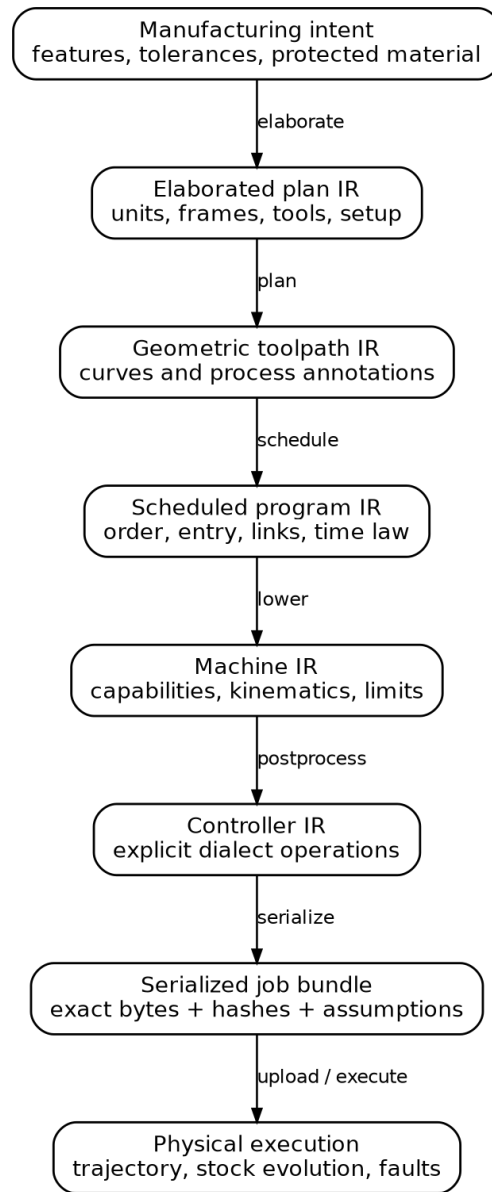


Figure 1.1: A semantic stack from manufacturing intent to physical execution.

At the top, manufacturing intent describes acceptable outcomes: remove a pocket to a depth, leave a finishing allowance, preserve a protected surface, or measure a datum. At the bottom, physical execution is a trace of machine poses, spindle states, sensor readings, communication events, faults, and stock changes. Every compiler stage removes freedom or adds implementation detail.

NIST’s canonical machining-command work used a similar separation in the opposite direction: an RS274 interpreter maps modal controller syntax to canonical machining operations [R1, R2]. A semantic CAM compiler begins with machining operations and lowers them to a controller dialect. STEP-NC likewise separates manufacturing features, working steps, and technology from immediate axis-level syntax [R3, R42, R43].

1.1.1.2 Source meaning is usually a set

A conventional expression such as $2 + 3$ has one mathematical result. A manufacturing operation usually permits many implementations. A pocket can be cut by raster passes, offsets, adaptive clearing, trochoidal motion, or a hybrid. The source operation should therefore denote a set of acceptable executions or outcomes.

Let an intent be I . Its meaning is:

$$\llbracket I \rrbracket \subseteq \mathcal{E},$$

where $\mathcal{E}$ is a space of physical executions or final workpiece states. A planner selects one implementation P such that the executions of P belong to the allowed set, up to declared approximation:

$$\text{compile}(I) = P,$$

$$\forall \tau \in \text{Exec}(P, A), \quad \alpha(\tau) \in N_\varepsilon(\llbracket I \rrbracket).$$

Here:

- A is a set of assumptions about the machine, setup, tools, and controller;
- τ is a possible execution trace;
- α abstracts low-level execution into observations relevant to the intent;
- N_ε is a tolerance neighborhood under a specified metric.

This is a refinement statement. The target program is allowed to choose details that the source left open, but it must not introduce behavior outside the source specification.

1.1.1.3 Why the target is not text

G-code text is an encoding of controller behavior. Its meaning depends on:

- modal state inherited from prior blocks;
- active units and coordinate systems;
- controller-specific interpretations of arcs, cycles, probes, and rapids;
- machine configuration and firmware version;
- asynchronous machine state such as homing, alarms, and active playback;
- numerical formatting and rounding;
- physical dynamics and following error.

Two files with different text can be semantically equivalent. Two identical files can behave differently under different initial modal states. Consequently, a compiler theorem should relate source intent to interpreted controller and physical traces, not to strings.

A useful end-to-end statement is:

$$\text{CheckBundle}(B) = \text{true} \wedge \text{AssumptionsHold}(A) \implies \forall \tau \in \text{Execute}(B, A), \text{Safe}(\tau) \wedge \text{Conforms}_\varepsilon(\tau, I).$$

The exact job bytes are part of B , but they are not the entire object.

1.1.1.4 Three kinds of correctness

A serious CAM compiler needs at least three notions of correctness.

Language correctness asks whether the program is well formed. Are all names resolved? Are units compatible? Are coordinates in known frames? Is a tool selected before it is used?

Geometric and process correctness asks whether the planned tool motion satisfies the manufacturing intent. Does it preserve protected material? Does it remove required stock? Does the holder avoid fixtures? Are entries and links feasible in the evolving stock state?

Control correctness asks whether the controller receives, stores, starts, pauses, resumes, aborts, and reports the intended job. Does authorization refer to the exact uploaded bytes? Can a timeout desynchronize the protocol? Does a feed hold actually reach the machine even when another command is pending?

The three cannot be collapsed. A geometrically correct path can be serialized incorrectly. A valid G-code file can be uploaded under the wrong name. A correct upload can be executed under a changed work offset.

1.1.1.5 The semantic waist

A robust system benefits from a small, machine-independent language of canonical machining actions. It serves as a semantic waist between high-level planners and low-level postprocessors.

Typical canonical actions include:

```
type CanonicalCommand =
| { kind: "selectTool"; tool: ToolRef }
| { kind: "setSpindle"; rpm: Rpm; direction: "cw" | "ccw" }
| { kind: "stopSpindle" }
| { kind: "traverse"; path: Path; clearance: ClearancePolicy }
| { kind: "cut"; path: Path; feed: FeedRate; intent: CutIntent }
| { kind: "probe"; path: Path; expectedContact: ContactModel }
| { kind: "dwell"; duration: Seconds }
| { kind: "pause"; reason: string };
```

There is no G0, G17, or I/J/K in this language. Those are controller encodings. Conversely, the canonical language must contain physical distinctions that G-code can obscure, such as the difference between free-space traverse and cutting feed.

1.1.1.6 A first design rule

Design rule

Define correctness at the highest semantic level that expresses the operator's intent, then carry that meaning through explicit refinement relations. Never use textual similarity to stand in for physical equivalence.

1.1.1.7 Worked example: a rectangular pocket

Suppose the source program requests a $30\text{ mm} \times 20\text{ mm}$ pocket, 4 mm deep, with 0.2 mm radial and axial roughing allowance. A correct source meaning does not name a raster or stepover. It states constraints on the residual stock and protected region.

Let R be the pocket region, z_f the nominal floor, and δ_r, δ_a the allowances. One possible specification is:

1. Material above $z_f + \delta_a$ inside the inward-offset pocket $R \ominus B_{\delta_r}$ must be removed.
2. Material outside R must not be removed beyond a geometric tolerance ε .
3. The fixture and holder must remain disjoint for the entire execution.
4. The machine must remain inside its admissible configuration set.

A planner can now select a strategy and produce a witness. A checker evaluates the witness against these four propositions. If the strategy changes from raster to offsets, the intent and checker need not change.

1.1.1.8 Exercises

1. Give two different G-code programs that plausibly implement the same straight cutting move. Identify the initial modal assumptions required for equivalence.
2. Define a manufacturing intent for drilling a through-hole without referring to a canned cycle or peck sequence.
3. Explain why a preview renderer is not automatically a verifier, even when its image appears correct.
4. Write an end-to-end correctness statement for a probing operation. Include at least three physical assumptions.
5. Classify each failure as language, geometric/process, control, or cross-boundary: wrong tool diameter, stale work offset, truncated upload, invalid arc radius, job never started.
6. Describe one circumstance in which rejecting a valid source program is preferable to emitting code.

1.1.2 Manufacturing Intent as a Specification

Section goals. The reader should be able to model a manufacturing operation as a predicate over outcomes, distinguish required removal from forbidden removal, and explain why planning is witness synthesis rather than mere path generation.

1.1.2.1 Specifications before strategies

CAM user interfaces often conflate an operation with an algorithm: “parallel finishing,” “adaptive clearing,” or “waterline.” These labels are useful, but they are strategies, not meanings. A semantic design starts with a predicate that any acceptable strategy must satisfy.

For a stock solid S_0 , target solid P , protected fixtures O , and tool assembly T , a manufacturing operation can be viewed as a relation:

$$\mathcal{M}(S_0, P, O, T, S_f, \tau),$$

where S_f is final stock and τ is the execution trace. The relation may constrain:

- material that must be absent;
- material that must remain;
- surface deviation;
- roughness or scallop height;
- tool engagement and process limits;
- machine and fixture clearance;
- operation ordering;
- observable measurements and their uncertainty.

A planner synthesizes a candidate x such that $\mathcal{M}$ is expected to hold. Verification checks the candidate independently.

1.1.2.2 Required, permitted, and forbidden regions

It is useful to partition material space into three concepts.

The **required-removal region** R_{req} must be removed, perhaps within a tolerance.

The **forbidden-removal region** R_{forbid} must be preserved. It includes the protected target, fixtures, table, and any explicit no-cut volumes.

The **permitted-removal region** R_{permit} may be removed without violating the operation. Roughing allowance and sacrificial stock often live here.

A conservative checker should not infer one property from another. Showing that the tool did not enter R_{forbid} does not show that it removed R_{req} . Showing that enough stock disappeared does not show that protected material survived.

This distinction directly determines approximation direction:

- no-gouge and collision claims use an **outer** approximation of the swept tool volume;
- guaranteed-removal claims use an **inner** approximation;
- residual-stock claims use an outer approximation of what remains.

1.1.2.3 Features as semantic handles

Manufacturing features are not merely UI objects. They preserve information needed by later passes and certificates. Examples include:

```
interface PocketFeature {
    id: FeatureId;
    boundary: Region2D<"workpiece">;
    floor: SurfaceRef;
    depth: Mm;
    radialAllowance: Mm;
    axialAllowance: Mm;
    tolerance: Mm;
    finishClass?: SurfaceFinishRequirement;
}
```

A low-level polyline cannot explain which portion is a floor finish, which wall must be preserved, or why one pass precedes another. Keeping feature identity in provenance allows diagnostics such as:

Pocket P17 floor tolerance unresolved near (18.2, 9.7, -4.0)
because the ball-end tool enclosure exceeds the remaining 0.012 mm budget.

1.1.2.4 Intent is relational and partial

A source operation can be intentionally incomplete. It might leave strategy, direction, entry, or exact feed unspecified. This is not a defect. The planner's job is to resolve such choices while satisfying constraints.

Let I define a relation between initial and final states:

$$I \subseteq \Sigma \times \Sigma.$$

The operation is legal from σ when at least one σ' satisfies $(\sigma, \sigma') \in I$. A planner may choose any such result, but must emit evidence that connects its path to the relation.

Partiality is also important. If the requested pocket is narrower than the selected tool, the intent may have no feasible implementation under the current tool set. A correct compiler rejects or requests a different tool. It does not silently distort the pocket.

1.1.2.5 Planning as witness synthesis

The conventional type:

Intent -> Toolpath

hides the proof obligation. A better conceptual type is:

```
interface Planned<I, P, W> {
    intent: I;
    program: P;
    witness: W;
}

type Planner<I, P, W> =
    (intent: I, context: PlanningContext) =>
        Result<Planned<I, P, W>, PlanningDiagnostic[]>;
```

The witness can contain:

- the offset regions used to construct passes;
- coverage cells and residual bounds;
- feature-to-path provenance;
- entry feasibility results;
- precedence decisions;
- claimed approximation bounds;
- solver objective and lower bound.

The checker need not repeat the full search. It verifies that the proposed witness establishes the intended relation.

1.1.2.6 Contracts for common operations

A **roughing** contract commonly requires removal of bulk stock while preserving an allowance envelope. It may tolerate a coarse residual model but must prevent fixture and target penetration.

A **finishing** contract typically constrains maximum normal deviation or scallop height on a target surface. It needs a target model and a tool/contact model.

A **drilling** contract constrains hole axis, diameter class, depth, breakthrough, and cycle behavior. It may include chip evacuation and peck constraints.

A **probing** contract produces a measurement value and an updated frame or offset, together with an uncertainty model. It is not simply a move that ends at a commanded coordinate.

A **traverse** contract requires the entire tool assembly to remain in free space. Its endpoint alone is insufficient.

1.1.2.7 Worked example: rough then finish

Consider a pocket with nominal target P and a 0.25 mm roughing allowance. Define the roughing protected region as:

$$P_{\text{rough}} = P \oplus B_{0.25},$$

where $\oplus$ is a Minkowski sum. A roughing toolpath is no-gouge if its outer swept volume W^+ is disjoint from the interior of P_{rough} :

$$W^+ \cap \text{int}(P_{\text{rough}}) = \emptyset.$$

The finishing operation uses the nominal target and a much smaller tolerance layer. The two operations have different specifications even if they share geometry. A certificate that says only “gouge checked” is incomplete unless it identifies which protected region, which tolerance, which tool model, and which artifact were checked.

1.1.2.8 Design rules

Design rule

Represent manufacturing operations as predicates or relations over physical outcomes. Treat strategy output as a witness to those predicates. Keep required removal, forbidden removal, and permitted removal as separate claims.

1.1.2.9 Exercises

1. Specify a contour-finishing operation using a target boundary, side selection, axial range, and tolerance.
2. Construct an example where no-gouge holds but required removal fails.
3. Construct an example where required removal holds but no-gouge fails.
4. Design a witness object for a raster pocket planner.
5. Explain why a probe operation must return a value and uncertainty rather than merely update position.
6. Define a rejection diagnostic for a tool that cannot enter a feature.
7. Identify which parts of a machining intent are stable under toolpath reordering and which are not.

Checkpoint - meaning. You should now be able to explain why a pocket intent denotes many possible implementations, name the exact artifacts a gouge claim depends on, and distinguish an assumption from evidence.

1.2 The mathematical and geometric vocabulary we actually need

The previous section creates an immediate need for mathematics. “Acceptable outcomes” is a set. “This implementation satisfies that request” is a relation. “Within tolerance” requires a metric. “This coordinate is in G54, not machine space” requires a frame. None of these ideas is introduced for ornament; each removes a concrete ambiguity.

Definition - set. A set is a collection of objects considered as one mathematical object.

Small example. $[0, 100]$ is the set of real numbers between 0 and 100, inclusive.

CAM example. The feasible tool-center positions form a set after obstacles and machine limits are excluded.

Definition - predicate. A predicate is a yes/no function describing a property. Every predicate $P(x)$ determines the set of values for which it is true.

Small example. $P(x) \equiv 0 \leq x \leq 100$ characterizes the interval $[0, 100]$.

CAM example. `insideTravel(q)` is a predicate over machine configurations; `linkClear(path, stock, fixtures)` is a predicate over a proposed link and setup.

Definition - relation. A relation between sets A and B is a set of pairs (a, b) . A function is a special relation with exactly one output for each allowed input. Relations are useful when a command can have several possible outcomes or when a specification permits many implementations.

Small example. “Student s is enrolled in course c ” is a relation, not naturally a single-valued function.

CAM example. A probe command relates one starting state to a set of possible contact states because contact position is uncertain.

Definition - metric. A metric $d(x, y)$ measures distance and obeys nonnegativity, identity, symmetry, and the triangle inequality.

Small example. Euclidean distance measures separation between two 3-D points.

CAM example. Hausdorff distance compares whole path images; normal-direction distance measures error relative to a target surface.

Definition - neighborhood. The ε -neighborhood of a point or set contains everything whose metric distance from it is at most ε .

Small example. A radius-1 disk is the 1-neighborhood of its center under Euclidean distance.

CAM example. A bounded-refinement claim may allow the emitted trajectory to remain inside an ε -neighborhood of the ideal path, provided the metric is appropriate to the machining property.

Definition - frame. A frame is a coordinate system together with its relation to other coordinate systems. Coordinate triples without a frame are incomplete physical statements.

Small example. The point $(0, 0, 0)$ in a camera frame and the point $(0, 0, 0)$ in a world frame usually denote different locations.

CAM example. $(10, 5, -2)$ in G54 must be transformed through the work-to-machine transform before machine-travel or fixture-clearance claims can be checked.

Definition - dimensional type. A dimensional type records a physical dimension such as length, speed, acceleration, angle, or spindle rate. It prevents operations that are syntactically valid numbers but physically meaningless.

Small example. Adding 3 mm to 2 s is rejected.

CAM example. The stock Z1 jog command's F field can be a dimensionless scale rather than a feed in millimeters per minute; a dimensional API makes that distinction visible.

1.2.1 Mathematical Foundations: Sets, Relations, Metrics, and Orders

Section goals. The reader should be able to use relations to model nondeterministic semantics, metrics to state approximation, partial orders to model information, and fixed points to describe iterative analysis.

1.2.1.1 Sets and characteristic predicates

A geometric solid can be idealized as a set $S \subseteq \mathbb{R}^3$. Equivalently, it can be represented by a characteristic predicate:

$$\chi_S(x) = \begin{cases} 1 & x \in S, \\ 0 & x \notin S. \end{cases}$$

Different computational representations approximate this set: triangle meshes, signed distance fields, boundary representations, voxels, dexels, constructive solids, or implicit functions. The semantics should name the set; the implementation should state how its representation encloses or approximates that set.

Common set operations have direct manufacturing meanings:

- $A \cup B$: combined occupied volume;
- $A \cap B$: collision or overlap;
- $A \setminus B$: stock after removal;
- $A \oplus B$: dilation, tool compensation, or configuration obstacle;
- $A \ominus B$: erosion, inward allowance, or robust protected core.

1.2.1.2 Relations instead of functions

A function maps each input to one output. Physical commands may fail, measurements may vary, and controller behavior may be nondeterministic. A relation is therefore often a more honest semantic object:

$$R \subseteq A \times B.$$

For a command c , define:

$$\llbracket c \rrbracket \subseteq \Sigma \times \Sigma \times \mathcal{T} \times \mathcal{O},$$

where $\mathcal{T}$ contains traces and $\mathcal{O}$ contains outcomes such as success, alarm, timeout, or uncertain completion.

Relations compose:

$$R; S = \{(a, c) \mid \exists b. (a, b) \in R \wedge (b, c) \in S\}.$$

This is the semantic basis of sequential execution. It also exposes an important point: an ambiguous timeout may produce several possible successor states. The host must not pretend that the state is unchanged.

1.2.1.3 Metrics and neighborhoods

Approximate correctness needs a metric d . For points, Euclidean distance is common:

$$d(x, y) = \|x - y\|_2.$$

For compact sets, the Hausdorff distance is:

$$d_H(A, B) = \max \left\{ \sup_{a \in A} \inf_{b \in B} d(a, b), \sup_{b \in B} \inf_{a \in A} d(a, b) \right\}.$$

For a target surface, normal deviation may be more meaningful than Hausdorff distance. For gouge, maximum penetration depth into a protected solid may be the relevant quantity. For a frame transform, translation and rotation errors need separate units and propagation rules.

The tolerance neighborhood of a set is:

$$N_\varepsilon(A) = \{x \mid d(x, A) \leq \varepsilon\}.$$

A certificate must name both ε and the metric. “Verified to 0.02 mm” is incomplete if it does not say what distance is bounded.

1.2.1.4 Equivalence, preorder, and refinement

Exact semantic equivalence is symmetric:

$$P \equiv Q \iff \text{Beh}(P) = \text{Beh}(Q).$$

Refinement is directional:

$$Q \sqsubseteq P \iff \text{Beh}(Q) \subseteq \text{Beh}(P).$$

The target Q makes at least as many decisions as source P and introduces no disallowed behavior. Refinement is generally a preorder: reflexive and transitive, but not necessarily antisymmetric at the syntactic level.

Compiler passes frequently establish different relations:

- normalization: equivalence;
- scheduling: trace refinement;
- curve sampling: bounded geometric refinement;
- optimization: feasible refinement plus objective improvement;
- error recovery: perhaps no refinement at all unless modeled explicitly.

1.2.1.5 Lattices of information

Static analysis often computes information ordered by precision. Let $a \sqsubseteq b$ mean that a is at least as precise as b , or choose the reverse convention consistently.

An interval domain illustrates the idea. The exact value 5 is represented by $[5, 5]$. The interval $[4, 6]$ is less precise, and $[-\infty, \infty]$ represents complete ignorance. Joining two control-flow paths requires an upper bound that contains both possibilities.

A complete lattice provides:

- a bottom element $\perp$;
- a top element $\top$;
- least upper bounds $\sqcup$;
- greatest lower bounds $\sqcap$;
- fixed points for recursive or looping behavior.

Abstract interpretation uses these structures to compute conservative invariants [R7].

1.2.1.6 Monotonicity and fixed points

A transfer function F on an abstract domain should be monotone:

$$a \sqsubseteq b \implies F(a) \sqsubseteq F(b).$$

For a loop or recursive subprogram, the analysis seeks a fixed point:

$$F(x) = x.$$

Iterating from $\perp$ may converge to the least fixed point. Widening accelerates convergence when domains contain infinite ascending chains; narrowing can recover precision afterward.

Even a nominally straight-line G-code file can acquire loops through subprogram calls, macros, canned cycles, or controller-specific repetition. A validator must either analyze these constructs soundly or reject them.

1.2.1.7 Tolerance is not equality

A common practical relation is:

$$x \approx_\varepsilon y \iff d(x, y) < \varepsilon.$$

It is reflexive and symmetric, but not transitive. For $x = 0$, $y = 0.75\varepsilon$, and $z = 1.5\varepsilon$, both adjacent pairs match while x and z do not.

Therefore, tolerance-based endpoint matching cannot be treated as mathematical equality without additional machinery. Repeated snapping or chaining can accumulate error or create inconsistent topology. This observation will become central in Chapter 2’s treatment of path identity and in Chapter 4’s contour-key case study.

1.2.1.8 Worked example: an uncertainty box

Suppose a probed work origin has independent coordinate bounds:

$$x \in [9.98, 10.02], \quad y \in [4.99, 5.01], \quad z \in [-0.015, 0.015].$$

A point $(20, 12, -3)$ in the work frame maps to a machine-frame box, not a single point. A travel checker should propagate the box through the frame transform and prove that the entire enclosure lies within machine limits. Checking only the nominal transform is a simulation, not a robust travel proof.

1.2.1.9 Exercises

1. Prove that relational composition is associative.
2. Give a counterexample to transitivity of $\approx_\varepsilon$.
3. State no-gouge as a set-disjointness proposition using a tolerance erosion.
4. Describe an abstract spindle domain that distinguishes definitely off, definitely on, and unknown.
5. Explain why a join at a control-flow merge loses information.
6. Give two metrics that could be relevant to finishing a freeform surface and explain their difference.
7. Model an uncertain command timeout as a relation with multiple outcomes.

1.2.2 Geometry, Frames, and Units

Section goals. The reader should be able to model rigid transforms, prevent frame and unit confusion in an API, propagate transform uncertainty, and distinguish tool-center geometry from full tool-assembly geometry.

1.2.2.1 Frames are part of the type

A CNC program contains many coordinate systems:

- machine coordinates;
- work coordinate systems such as G54;
- stock coordinates;
- part-design coordinates;
- fixture coordinates;
- tool-tip and tool-center coordinates;
- camera or probe coordinates.

A tuple of three numbers is not enough to identify a point. The frame is part of its meaning:

```
interface Point3<F extends FrameId> {
    readonly x: Mm;
    readonly y: Mm;
    readonly z: Mm;
```

```
  readonly frame: F;
}
```

The compiler should reject addition or comparison across unrelated frames unless an explicit transform is supplied.

1.2.2.2 Rigid transformations

A pose in three-dimensional Euclidean space can be represented by an element of $SE(3)$:

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix},$$

where $R \in SO(3)$ is a rotation and $p \in \mathbb{R}^3$ is a translation. A point in homogeneous coordinates transforms as:

$$\bar{x}_B = T_{B \leftarrow A} \bar{x}_A.$$

Transforms compose:

$$T_{C \leftarrow A} = T_{C \leftarrow B} T_{B \leftarrow A}.$$

A frame graph should make direction explicit. Naming a transform merely `workOffset` encourages inversion mistakes.

1.2.2.3 Three-axis simplification and future proofing

For a fixed-orientation three-axis mill, tool orientation is usually constant and the relevant transform may be a translation plus possibly a setup rotation. The semantic model should still avoid baking this simplification into every abstraction. Paths can be parameterized over position only now while poses and transforms remain general enough for indexed or rotary axes later.

A good principle is to make the common case cheap without making the general case impossible.

1.2.2.4 Dimensional types

Bare numbers erase physical dimensions. The following errors all type-check in ordinary JavaScript:

```
const feed = 600;      // mm/min or mm/s?
const speed = 12000;   // rpm or rad/s?
const angle = 90;      // degrees or radians?
const depth = -3;      // mm, inch, or machine units?
```

Use nominal or branded types at API boundaries:

```
type Mm = number & { readonly __unit: "mm" };
type MmPerMin = number & { readonly __unit: "mm/min" };
type Rpm = number & { readonly __unit: "rpm" };
type Radians = number & { readonly __unit: "rad" };
```

Types alone are insufficient. Constructors must enforce finite values and domain restrictions:

```

function mm(x: number): Mm {
  if (!Number.isFinite(x)) throw new RangeError("length must be finite");
  return x as Mm;
}

function positiveFeed(x: number): MmPerMin {
  if (!Number.isFinite(x) || x <= 0) {
    throw new RangeError("feed must be finite and positive");
  }
  return x as MmPerMin;
}

```

A cast can forge a brand, so runtime elaboration remains part of the trusted language boundary.

1.2.2.5 Tools and assemblies

The tool tip is not the only moving geometry. A tool assembly may include:

- cutting body;
- non-cutting shank;
- collet;
- nut;
- holder;
- spindle nose;
- probe body.

Different claims use different subsets:

- target gouge usually concerns the cutting geometry;
- fixture collision concerns the entire assembly;
- required removal concerns a guaranteed inner cutting volume;
- machine travel concerns the kinematic reference point and axis limits.

A Tool should therefore not be only a diameter and flute length. It should refer to geometric models and uncertainty bounds.

1.2.2.6 Uncertain transforms

A probed or manually set transform is not exact. One useful representation is a nominal transform plus a bounded perturbation:

$$T = \hat{T} \exp(\xi^\wedge), \quad \xi \in \Xi,$$

where ξ is a six-dimensional twist and Ξ is an uncertainty set. A simpler three-axis system can maintain independent translation intervals and a small angular bound.

A claim about machine travel or collision should quantify over all transforms in the allowed uncertainty set. If that is too expensive, the certificate must state that only the nominal transform was simulated.

1.2.2.7 Worked example: work-frame travel

Suppose a toolpath lies within $x \in [0, 180]$ in work coordinates. The machine travel is $x \in [0, 200]$. It is incorrect to conclude that the path fits unless the work-to-machine transform is known. With a 30 mm positive offset, the

path reaches machine $x = 210$ and violates travel.

The correct check is:

$$\forall q_w \in Q_w, \forall T \in \mathcal{T}_{m \leftarrow w}, \quad Tq_w \in Q_m.$$

For interval boxes and pure translations, this reduces to interval addition. For rotations, the enclosure must account for coupling among axes.

1.2.2.8 Design rules

Design rule

Make units, frames, and tool-assembly identity explicit in the IR. Treat frame transforms and tool dimensions as assumptions with uncertainty, not as unexamined scalar constants.

1.2.2.9 Exercises

1. Define types for machine, work, stock, and part frames. Show one invalid operation the type system should reject.
2. Derive the inverse of an $SE(3)$ transform.
3. Explain why holder collision and target gouge require different tool solids.
4. Propagate an interval translation through an axis-aligned path bounding box.
5. Design a serializable ToolAssembly schema with uncertainty fields.
6. Identify the units and frame of every number in G1 X10 Y20 F600.
7. Explain why a runtime-checked constructor is still necessary when TypeScript brands are used.

Checkpoint - mathematical language. Given a point, path, or stock state, identify its frame and units; explain when a relation is more appropriate than a function; and name the metric used by any tolerance statement.

1.3 Four semantic views, nondeterminism, and refinement

One semantic description cannot answer every question. A set of acceptable final parts is excellent for stating intent but says little about modal controller state. A step-by-step transition system explains controller execution but can obscure the high-level manufacturing result. Hoare-style contracts explain local obligations. Temporal logic explains what must always or eventually hold across a protocol. We therefore use four semantic views and require them to agree where they overlap.

Definition - denotation. A denotation is the mathematical object assigned as the meaning of a program construct: a value, function, relation, set of outcomes, or set of traces.

Small example. The denotation of $2 + 3$ can be the number 5.

CAM example. The denotation of a pocket intent is a set of acceptable residual-stock states and process traces, not one polyline.

Definition - semantics. Semantics is a systematic account of meaning. Syntax tells us which programs are well formed; semantics tells us what those programs denote or how they behave.

Example 1. The syntax G91 G1 X1 is a valid sequence of words. Its operational meaning depends on the current position and on incremental mode.

Example 2. A high-level Pocket node denotes a set of acceptable final stock states rather than a controller command.

Definition - refinement. An implementation refines a specification when it resolves choices or adds detail without introducing behavior the specification forbids. If approximation is allowed, the relation must name a metric and a bound.

Small example. Choosing quicksort refines “return a sorted permutation” if the result is sorted and contains exactly the input elements.

CAM example. Choosing raster clearing refines a pocket intent if all possible executions remain inside the permitted geometric and process envelope.

Definition - nondeterminism. Nondeterminism means that one program state may have several permitted next states or outcomes. It can model deliberate implementation freedom, sensor uncertainty, scheduling, communication faults, or physical variation.

Small example. A set-valued specification allows any shortest path when several have equal cost.

CAM example. A probe may contact anywhere in an uncertainty interval; the checker must reason about the whole set rather than one nominal contact point.

Definition - approximation. An approximation replaces an exact object by another object that is easier to compute or represent. A useful assurance claim must state the direction of approximation and a quantitative error bound.

Small example. Replacing a circle by an inscribed polygon is an inner geometric approximation; a circumscribed polygon is an outer one.

CAM example. A conservative collision checker uses an outer enclosure of the tool-and-holder sweep so that an empty intersection proves the true sweep is also clear.

Definition - safety property. A safety property states that a bad event never occurs. A finite trace prefix can demonstrate a violation.

CAM example. A cutting move never occurs while the spindle is known to be stopped.

Definition - liveness property. A liveness property states that a desired event eventually occurs under stated environment and fairness assumptions.

CAM example. An accepted abort request eventually leads to Stopped or an explicit fault state.

Definition - temporal logic. Temporal logic is a language for stating properties over traces, using operators such as always $\Box$ and eventually $\Diamond$. Chapter 4 applies it to the controller protocol.

1.3.1 One cut command viewed four ways

Consider `Cut(path = p, tool = T1, feed = 400 mm/min)`. The command is the same; the question changes.

View	Question	Statement for the running pocket
Denotational	What mathematical result is allowed?	Final stock equals prior stock minus material removed by the T1 sweep, while protected target remains within tolerance.
Operational	How does state change step by step?	Check pre-state, advance along the trajectory, update pose and stock, emit motion events, or enter a fault outcome.
Axiomatic	What must hold before and after?	Pre: homed, known work coordinate system (WCS), T1 loaded, spindle valid, path clear. Post: pose at path end and stock updated according to the cut model.
Trace/temporal	What must always or eventually hold?	Always, cutting implies authorization and valid process state; after abort, eventually stopped or faulted.

The four statements are not competitors. They are projections of one intended semantics onto different engineering questions.

1.3.2 Four Semantic Views

Section goals. The reader should be able to explain denotational, operational, axiomatic, and trace semantics; select the appropriate view for a question; and connect the views through consistency conditions.

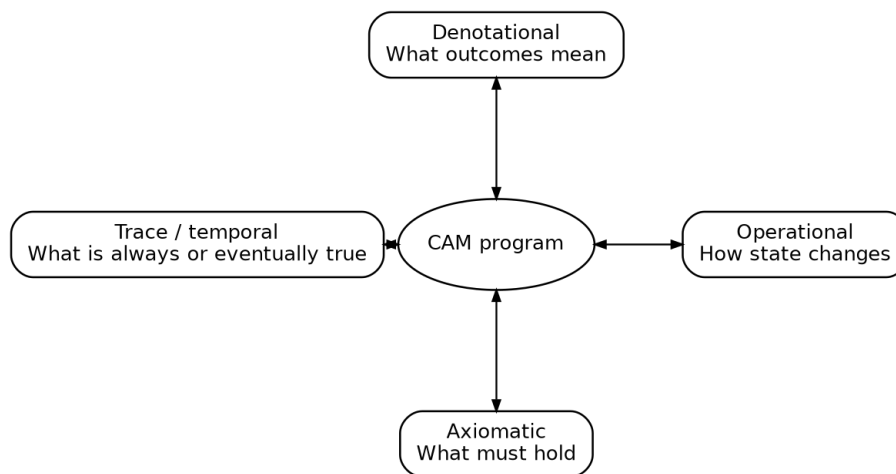


Figure 1.2: Four complementary semantic views of a CAM program.

No one semantic style answers every engineering question. A credible CAM compiler uses several views of the same language.

1.3.2.1 Denotational semantics

Denotational semantics assigns a mathematical meaning to a program. For manufacturing intent, the meaning is often a set of acceptable outcomes. For a path, it may be a continuous curve. For a command, it may be a relation on states.

Let a machine state be:

$$\sigma = (q, \dot{q}, F, W, T, S_p, C, S_{stock}, P, O, M, t),$$

where the components include axis state, frame graph, work system, active tool, spindle and coolant state, stock, target part, obstacles, modal/controller state, and time.

A command can denote:

$$\llbracket c \rrbracket : \Sigma \rightarrow \mathcal{P}(\Sigma \times \text{Trace} \times \text{Outcome}).$$

The powerset captures nondeterminism and failure.

1.3.2.2 Operational semantics

Operational semantics explains execution step by step [R6]. A small-step relation has the form:

$$\langle c, \sigma \rangle \rightarrow \langle c', \sigma' \rangle.$$

A big-step relation for an atomic command is:

$$\langle c, \sigma \rangle \Downarrow (\sigma', e),$$

where e is an event or trace fragment.

A simplified cutting rule is:

$$\frac{\text{Homed}(\sigma) \quad \text{WCSKnown}(\sigma) \quad \text{ToolLoaded}(\sigma, T) \quad \text{SpindleValid}(\sigma) \quad f > 0}{\langle \text{Cut}(\gamma, T, f), \sigma \rangle \Downarrow (\sigma[q := \text{end}(\gamma), S_{stock} := S_{stock} \setminus \text{Sweep}(T, \gamma)], \text{motionTrace}(\gamma, f))}.$$

Rules like this are executable specifications for interpreters and validators.

1.3.2.3 Axiomatic semantics

Axiomatic semantics reasons with assertions before and after commands [R4, R5]. A Hoare triple is:

$$\{P\} c \{Q\}.$$

For a cut:

$$\{\text{Homed} \wedge \text{WCSKnown} \wedge \text{Tool} = T \wedge \text{SpindleOn} \wedge \text{PathSafe}(\gamma)\}$$

$$\text{cut}(\gamma, T, f)$$

$$\{\text{Pose} = \text{end}(\gamma) \wedge S' = S \setminus \text{Sweep}(T, \gamma)\}.$$

This view is natural for preflight requirements, API contracts, and pass invariants.

1.3.2.4 Trace and temporal semantics

A controller protocol unfolds over time and includes concurrency. Temporal logic reasons about whole traces [R8, R34]. Typical safety properties use “always” $\Box$:

$$\Box(\text{Motion} \Rightarrow \text{Homed} \wedge \text{Authorized}).$$

Liveness properties use “eventually” $\Diamond$:

$$\Box(\text{AbortRequested} \Rightarrow \Diamond(\text{Stopped} \vee \text{Faulted})).$$

Artifact binding can also be temporal:

$$\Box(\text{ExecuteHash} = h \Rightarrow \text{StoredHash} = h).$$

A static list of canonical commands cannot establish these properties. They belong to the host-controller state machine.

1.3.2.5 Consistency among views

The semantic views should agree. If an operational interpreter executes a command from σ to σ' , then (σ, σ') should belong to the denotational relation. If a Hoare triple is proved, every operational execution starting in a state satisfying P should finish in a state satisfying Q , subject to its termination assumptions.

These consistency theorems make the semantics more than parallel documentation. They allow one view to justify a checker implemented in another.

1.3.2.6 Which view to use

Question	Best primary view
What final shapes are acceptable?	Denotational
What does this modal block do next?	Operational
What preflight is required?	Axiomatic
Can abort remain pending forever?	Temporal
Does a pass preserve all traces?	Denotational/refinement
Can the program reach a cut with spindle off?	Operational + abstract interpretation
Does the protocol execute the acknowledged bytes?	Temporal

1.3.2.7 Worked example: probe semantics

A probe command is a useful test of semantic completeness. Denotationally, it maps an initial state to a set of possible contact measurements. Operationally, it advances until contact, limit, or failure. Axiomatic reasoning requires a known probe direction, maximum distance, and valid probe state. Temporally, the host must eventually receive either a measurement or a terminal fault; a silent timeout is not a successful no-contact result.

A G-code emitter that treats the commanded endpoint as the measured endpoint confuses syntax with observation. The result of probing is produced by execution, not known at compile time.

1.3.2.8 Exercises

1. Give denotational and operational semantics for a dwell command.
2. Write a Hoare triple for tool selection.
3. State a temporal property for feed hold.
4. Explain why liveness generally requires fairness or environmental assumptions.
5. Describe a consistency theorem between a G-code interpreter and canonical IR.
6. Model a probe no-contact failure in all four semantic views.

1.3.3 Nondeterminism, Approximation, and Refinement

Section goals. The reader should be able to distinguish implementation choice from physical uncertainty, state exact and bounded refinement relations, and compose error bounds with explicit metrics.

1.3.3.1 Sources of nondeterminism

Nondeterminism enters a CAM system in several ways.

Specification freedom arises because an intent permits many implementations. The compiler may select any feasible strategy.

Algorithmic nondeterminism arises from parallel scheduling, hash iteration, randomized search, or solver tie-breaking. Reproducible builds should control or record it.

Controller nondeterminism includes asynchronous status messages, buffering, alarms, and timing-dependent transitions.

Physical uncertainty includes probe noise, backlash, tool runout, thermal drift, spindle regulation, and servo error.

These sources should not be represented by one generic “tolerance.” Choice, uncertainty, and error have different semantics.

1.3.3.2 Exact preservation

A semantics-preserving pass satisfies:

$$\text{Sem}(O) = \text{Sem}(I).$$

Examples include alpha-renaming, canonical data reordering, or modal compression when the interpreted canonical trace is identical. Exact preservation is a strong claim and should be reserved for passes whose semantics

really match.

1.3.3.3 Trace refinement

A lowering pass often resolves choices:

$$\text{Traces}(O) \subseteq \text{Traces}(I).$$

For example, a high-level traverse permits any safe free-space route. Lowering to retract, XY motion, and descent selects one route. The target is not equivalent to the source because it has fewer behaviors; it refines it.

Abadi and Lamport’s refinement mappings formalize how concrete states and traces implement abstract specifications [R9]. The same idea applies across CAM IR levels.

1.3.3.4 Bounded geometric refinement

A curve-linearization pass may satisfy:

$$d_H(\gamma_I, \gamma_O) \leq \varepsilon.$$

That is not automatically enough for physical safety. The checker must establish how centerline error affects the swept tool and holder, including orientation and frame uncertainty. A bound on an arc’s sagitta is useful only if it survives subsequent rounding and controller interpolation.

1.3.3.5 Quantitative error composition

Suppose a pass f is Lipschitz with constant L_f under the chosen metric and introduces local error ε_f . If the input already has error ε_{in} , then:

$$\varepsilon_{out} \leq L_f \varepsilon_{in} + \varepsilon_f.$$

For a sequence of passes:

$$\varepsilon_n \leq \left(\prod_{i=1}^n L_i \right) \varepsilon_0 + \sum_{k=1}^n \left(\prod_{j=k+1}^n L_j \right) \varepsilon_k.$$

A plain scalar sum is valid only under compatible metrics and amplification assumptions. Translation error, angular error, surface-normal error, and maximum gouge depth should remain distinct until a sound propagation rule combines them.

1.3.3.6 Approximation direction

A safety checker must know whether an approximation is an under- or over-approximation.

Let X be the true set. Maintain:

$$X^- \subseteq X \subseteq X^+.$$

- X^+ is an outer enclosure and is appropriate for proving absence of collision.
- X^- is an inner enclosure and is appropriate for proving guaranteed removal.
- A point sample with no enclosure theorem is neither.

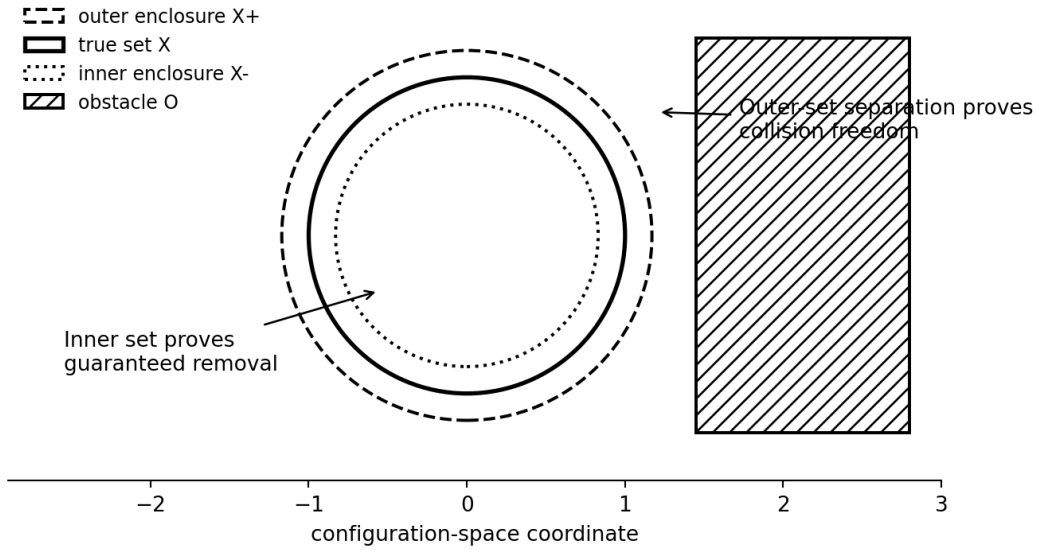


Figure 1.3: Inner and outer approximations support different proof directions.

1.3.3.7 Unknown is a valid result

A checker should have at least these outcomes:

- proved exact;
- proved within a bound;
- refuted with a counterexample;
- inconclusive because the available representation is too coarse;
- not checked;
- conditional on explicit assumptions.

Conflating inconclusive with safe is a semantic error. Refinement-based compiler design permits rejection. A pass may decline to lower an operation it cannot implement within its budget.

1.3.3.8 Worked example: arc lowering

For a circular arc of radius R approximated by chords with half-angle θ , the sagitta is:

$$s = R(1 - \cos \theta).$$

To keep $s \leq \varepsilon$, choose:

$$\theta \leq \arccos\left(1 - \frac{\varepsilon}{R}\right).$$

The arc pass can emit a witness containing R , total sweep, segment count, and maximum sagitta. An independent checker recomputes the bound and verifies endpoints. The postprocessor must then account for decimal round-

ing. If each coordinate is rounded by at most ρ , the final positional error needs an additional norm-dependent term.

1.3.3.9 Design rules

Design rule

Every approximation must declare its metric, direction, bound, assumptions, and artifact scope. A checker that cannot prove the bound must return inconclusive rather than upgrading sampled evidence into a guarantee.

1.3.3.10 Exercises

1. Distinguish source-level nondeterminism from physical uncertainty in a pocket operation.
2. Give an example of trace refinement that is not equivalence.
3. Derive the number of linear segments needed for a 20 mm radius half-circle with 0.01 mm sagitta tolerance.
4. Explain why a sampled toolpath is not automatically an outer approximation.
5. Compose two error bounds with Lipschitz constants 2 and 0.5.
6. Design a result type that separates refuted, inconclusive, and not checked.
7. Give an example where an under-approximation is unsafe for collision checking.

Checkpoint - semantics. Describe one cut denotationally, operationally, axiomatically, and temporally; then state what a bounded refinement must name besides a number ε .

1.4 Chapter synthesis: specifying the running pocket

We can now state the running example without mentioning a particular toolpath.

Let:

- S_0 be the initial stock solid;
- P be the target part that must remain after all operations;
- R_{rough} be the region that roughing is required to remove;
- A_{finish} be the roughing allowance intentionally left for finishing;
- O be the union of fixture and machine obstacles;
- B_t and B_h be the cutting-tool and holder solids;
- F_{WM} be the uncertain transform from work coordinates to machine coordinates;
- Q_{adm} be the admissible machine-configuration set.

A roughing intent can be expressed as separate propositions:

1. **Required removal:** the guaranteed removed volume contains R_{rough} .
2. **Target protection:** the cutting sweep does not enter P beyond the roughing allowance and geometric tolerance.
3. **Fixture protection:** the full tool-and-holder sweep is disjoint from O .
4. **Travel:** every transformed machine configuration lies in Q_{adm} .
5. **Process state:** cutting occurs only with the intended tool and spindle state.
6. **Uncertainty:** all five propositions hold for every transform and tool dimension inside the declared uncertainty sets.

Notice the separation. A successful required-removal check does not prove no gouge. A clear cutting-tool sweep does not prove the holder clears a clamp. A geometrically safe path does not prove that the controller uploaded and executed the certified bytes. The rest of the book preserves these distinctions.

1.4.1 A student-readable refinement statement

The compact theorem

$$\forall \tau \in \text{Exec}(P, A), \quad \alpha(\tau) \in N_\epsilon(\llbracket I \rrbracket)$$

can be read line by line:

- take **every** execution trace the compiled program can produce under assumptions A ;
- forget low-level details that do not matter by applying the abstraction α ;
- require the resulting manufacturing observation to lie within the declared tolerance neighborhood of the intent’s allowed outcomes.

The universal quantifier is the important part. Simulating one likely trace does not establish the statement.

1.4.2 Common misconceptions

1. “**The path is the machining result.**” A path is geometric. Material removal also depends on tool shape, time parameterization, process state, and stock.
2. “**Tolerance means approximate equality.**” A tolerance test is usually not transitive and cannot be used as mathematical identity without extra machinery.
3. “**A finer sampling grid is automatically safer.**” A grid parameter is not a continuous-domain bound unless the checker proves what can happen between samples.
4. “**A safe source program implies safe G-code.**” Every lowering, rounding, modal compression, upload, and runtime state change must preserve or re-establish the relevant claims.

1.4.3 Chapter exercises

1. Write an intent for drilling four holes that distinguishes required, permitted, and forbidden behavior. Do not specify a toolpath.
2. Give two different metrics for comparing a ball-end finishing path to a target surface. Explain which machining error each metric captures.
3. Model a probing operation as a relation. Identify at least three sources of nondeterminism.
4. Construct three points p, q, r showing that “distance less than ϵ ” is not transitive.
5. State one denotational, one operational, one Hoare-style, and one temporal property for spindle startup followed by a cut.

Chapter 2

A Language That Can Be Reasoned About

Chapter goal. Turn the semantic model from Chapter 1 into a practical JavaScript-facing language and a sequence of intermediate representations. By the end of the chapter, you should be able to explain why JavaScript is best used as a staged macro language, distinguish an abstract syntax tree (AST) from an intermediate representation (IR) and a dialect, model paths separately from actions, make effects and ordering explicit, and bind every generated object to provenance and identity.

The central engineering tension is convenience versus inspectability. JavaScript is expressive and familiar, but unrestricted JavaScript can read ambient state, invoke host APIs, construct opaque closures, and make compilation nondeterministic. A compiler IR is inspectable and serializable, but a raw IR is unpleasant for users to author directly.

The solution is staging: let JavaScript construct an inert plan, then end the JavaScript phase. From that boundary onward, the compiler operates on explicit data with defined semantics. This chapter explains the boundary and the IR ladder below it.

A second tension is geometry versus action. A curve only says where a point travels. It does not say whether the motion is a rapid, a cut, or a probe; it does not name the tool; it does not consume machine state; and it does not establish that one command may legally follow another. The IR must represent those distinctions instead of asking later passes to reconstruct them from context.

2.1 Staged JavaScript and the IR ladder

We begin with the authoring boundary because it determines what the rest of the compiler can trust.

Definition - staged programming. Staged programming divides computation into phases. An earlier phase constructs code or data that a later phase consumes. In this book, JavaScript runs in the authoring phase and must produce an inert, finite, serializable plan artifact.

Small example. A template program computes a list of fields and emits a JSON schema; the schema, not the live template environment, is the next phase's input.

CAM example. A user script loops over hole locations and calls plan-building functions. The resulting `PlanAst` contains holes, tools, parameters, and source locations but no executable closure.

Definition - abstract syntax tree (AST). An AST is a tree-shaped representation of source constructs after parsing or macro construction. It normally preserves source-oriented concepts and locations.

Example 1. `pocket({width: 30, height: 20})` becomes an AST node with named fields and a source span.

Example 2. A loop exists while JavaScript runs but need not remain in the resulting AST if it merely generated ten hole nodes.

Definition - intermediate representation (IR). An IR is a structured language used between compiler stages. It has explicit syntax, invariants, and semantics chosen for analyses and transformations rather than for end-user convenience.

CAM example. The intent IR may contain a `PocketIntent`; the geometric toolpath IR contains paths; the controller IR contains explicit controller operations. These are different languages because they answer different questions.

Definition - dialect. A dialect is a coherent family of operations and types at one abstraction level. A dialect has legality rules: after full conversion to a lower dialect, unsupported higher-level operations must not remain.

CAM example. A machine dialect may include `machine.traverse` and `machine.cut`, while a Z1 controller dialect includes the specific `upload`, `play`, `jog`, `probe`, and modal behaviors implemented by the target firmware profile.

Definition - elaboration and lowering. Elaboration resolves convenient but implicit source information into an explicit typed form. Lowering replaces higher-level operations with lower-level operations that refine their meaning.

Example 1. Elaboration resolves `3.175` plus a declared unit into a typed tool diameter.

Example 2. Lowering replaces an abstract safe traverse with a retract, lateral move, and descent that are valid for a specific machine and current stock state.

2.1.1 JavaScript as a Staged Macro Language

Section goals. The reader should be able to separate an authoring language from the stable CAM language, explain staging and elaboration, and design a deterministic capability-limited script boundary.

2.1.1.1 The convenience and danger of embedded JavaScript

JavaScript is attractive as a CAM authoring language. It has familiar control flow, functions, data structures, packages, editors, and a large ecosystem. Parametric geometry becomes concise:

```
for (let i = 0; i < holeCount; i++) {
  drill({
    at: p(origin.x + i * spacing, origin.y, topZ),
    diameter: mm(3),
    depth: mm(8),
  });
}
```

But unrestricted JavaScript is a poor stable semantic core. It can inspect time, randomness, environment variables, files, network state, prototypes, and host globals. It can loop forever, mutate shared objects, invoke asynchronous callbacks after lexical scopes have ended, or depend on implementation details of the runtime.

The solution is not to discard JavaScript. It is to give it a precise role: **JavaScript is a macro language that constructs an inert CAM abstract syntax tree.**

2.1.1.2 Two languages, not one

The authoring language includes JavaScript syntax and its computational power. The stable CAM language is a serializable AST with explicit constructors and no executable closures.

```
JavaScript source
  |
  | evaluate in isolated staged environment
  v
Immutable authoring AST
  |
  | elaborate names, units, frames, tools
  v
Plan IR
```

After staging, no compiler pass should call user-provided functions. The AST should contain only finite data, identifiers, source spans, and references to declared artifacts.

This resembles multi-stage programming: one program executes now to construct another program that will be analyzed and compiled later [R37]. The staging boundary is a semantic boundary and a security boundary.

2.1.1.3 Capability design

The script should receive an explicit capability object:

```
interface CamAuthoringCapabilities {
  units: UnitConstructors;
  geometry: GeometryConstructors;
  tools: ToolRegistryBuilder;
  plan: PlanBuilder;
  diagnostics: DiagnosticSink;
}
```

Capabilities prevent accidental ambient access, but ordinary lexical shadowing is not a security sandbox. Same-realm evaluation with `new Function` still exposes standard constructors and prototype chains. A hostile or simply defective script can also block the event loop indefinitely.

A production design should execute scripts in a separately terminable worker, process, or isolate with:

- no ambient network or filesystem access unless explicitly granted;
- a wall-clock deadline enforced outside the script;
- a memory limit;
- deterministic time and randomness sources;
- a versioned API module;
- structured input and output serialization;
- process termination on cancellation;
- no shared mutable object graph with the compiler.

2.1.1.4 Determinism and declared inputs

A reproducible authoring run should be modeled as:

$$\text{eval}(\text{extsource}, \text{apiVersion}, \text{inputs}, \text{seed}) = \text{AST}.$$

Every input must be declared and content-addressed. If a script reads a mesh, tool library, material table, or project parameter, the compilation record should include its hash.

```
interface ScriptEvaluationRecord {
  sourceHash: Hash;
  languageVersion: string;
  apiVersion: string;
  inputArtifacts: readonly ArtifactRef[];
  deterministicSeed?: bigint;
  resultAstHash: Hash;
  diagnostics: readonly Diagnostic[];
}
```

The same record should be sufficient to reproduce the AST in a clean environment.

2.1.1.5 Scope combinators

An authoring API may use scoped operations:

```
withTool(tool, () => {
  withFeed(mmPerMin(400), () => {
    pocket(feature);
  });
});
```

A synchronous implementation usually saves state, invokes the callback, and restores state in `finally`. If the callback returns a promise, restoration can happen before awaited work completes. The apparent lexical semantics are broken.

There are three sound choices:

1. Prohibit asynchronous callbacks and detect promises at runtime.
2. Make the combinator explicitly asynchronous and await the callback before restoration.
3. Avoid dynamic scopes in the AST builder and pass immutable context explicitly.

For a deterministic macro language, the third option is often simplest.

2.1.1.6 Mutability at the boundary

If a tool object is registered and then retained by reference, later user mutation can change the plan after validation:

```
const tool = { diameter: mm(3.175), fluteLength: mm(12) };
registerTool("T1", tool);
tool.diameter = mm(8); // changes meaning if reference was retained
```

The builder should deep-copy and freeze boundary objects, or convert them immediately into canonical immutable records. Hashing an object only helps if the object cannot change afterward.

2.1.1.7 Worked example: staged pocket program

A good authoring run produces data such as:

```

const ast: AuthoringProgram = {
  version: "cam-authoring/1",
  tools: [{
    id: "T1",
    geometry: { kind: "flatEndMill", diameter: 3.175, fluteLength: 12 },
    source: { file: "job.cam.js", line: 3, column: 14 },
  }],
  operations: [{
    kind: "rectPocket",
    featureId: "P1",
    frame: "work:G54",
    origin: [10, 10, 0],
    size: [30, 20],
    depth: 4,
    tool: "T1",
    radialAllowance: 0.2,
    axialAllowance: 0.2,
    source: { file: "job.cam.js", line: 8, column: 3 },
  }],
};

```

The AST contains no closure that can later inspect the host environment. It can be validated, displayed, serialized, hashed, diffed, and compiled by another implementation.

2.1.1.8 Design rules

Design rule

Use JavaScript to construct an inert, immutable, content-addressed AST. Do not make the JavaScript realm, closures, mutable objects, or ambient globals part of the CAM semantics.

2.1.1.9 Exercises

1. List five ambient JavaScript capabilities that damage reproducibility.
2. Design a JSON-serializable AST node for a probing operation.
3. Explain why a timeout checked after a script returns is not an enforced timeout.
4. Show how a synchronous scope combinator fails with an async callback.
5. Define an isolation protocol between a Studio UI and a worker process.
6. Specify which fields must be hashed to reproduce an authoring run.

2.1.2 A Multi-Level IR Architecture

Section goals. The reader should be able to assign concerns to IR levels, define legality at each stage, and avoid both a monolithic universal IR and premature lowering.

2.1.2.1 Why one IR is not enough

A universal node type such as:

```

interface Operation {
  kind: string;
  data: unknown;
}

```

appears flexible but erases the guarantees that make passes understandable. At the other extreme, lowering directly from authoring syntax to G-code forces every planner to know controller details and destroys manufacturing provenance.

A multi-level compiler retains the right information at each stage and defines explicit conversion points. MLIR demonstrates the value of multiple dialects and legality-controlled transformations for domain-specific compilation [R16]. A CAM compiler benefits even more because its abstraction levels correspond to distinct physical meanings.



Figure 2.1: A ladder of CAM intermediate representations.

2.1.2.2 Authoring AST

The authoring AST preserves:

- source spans;
- names and declarations;
- macros already expanded into data;
- user-level concepts;
- unresolved references where diagnostics benefit.

It should exclude arbitrary executable values after the staging boundary.

2.1.2.3 Elaborated Plan IR

Elaboration resolves:

- units;
- frames;
- tool references;
- geometry artifact references;
- defaults;
- scope-derived settings;
- identifier uniqueness;
- finite values and domain constraints.

The elaborated IR is the first stable, language-independent project representation. It should be possible to save it and compile it without re-running JavaScript.

2.1.2.4 Manufacturing Intent IR

Intent IR represents features and operations:

- pockets, holes, slots, surfaces, profiles;
- roughing and finishing requirements;
- allowances and tolerances;
- protected and permitted regions;
- dependencies and setup requirements;
- process constraints.

It deliberately excludes exact tool-center paths.

2.1.2.5 Geometric Toolpath IR

Toolpath IR contains continuous or piecewise-continuous curves with process semantics:

```
interface CuttingPath<F extends FrameId> {
  geometry: Path<F>;
  tool: ToolRef;
  feedPolicy: FeedPolicy;
  operation: OperationRef;
  phase: "entry" | "rough" | "finish" | "leadOut";
  directionality: "reversible" | "forwardOnly";
}
```

It should still be independent of a controller's modal syntax.

2.1.2.6 Scheduled Program IR

Scheduling introduces:

- a total or partially ordered operation sequence;
- path orientation choices;
- entries, retracts, and links;
- tool changes;
- probe dependencies;
- feed schedules or time laws;
- evolving stock-state references.

This is where semantics become stateful. A link that was safe before roughing may be unnecessary afterward; a path that was safe after roughing may be unsafe before it.

2.1.2.7 Machine IR

Machine lowering resolves:

- axis and kinematic capabilities;
- supported interpolation primitives;
- machine and work frames;
- travel limits;
- spindle and feed limits;
- tool-change behavior;
- probe and accessory capabilities;
- safe expansion of abstract traverses.

Every remaining operation must be supported by the selected machine profile or rejected.

2.1.2.8 Controller IR

Controller IR makes the controller's semantics explicit without committing to final formatting. It may contain:

- set units;
- select plane;
- select work offset;
- set absolute mode;
- linear and arc motion;

- probe cycle;
- spindle and accessory actions;
- upload metadata;
- program end.

The controller IR should have an interpreter. Final G-code text is then a serialization of this IR plus an optional modal-compression optimization.

2.1.2.9 Serialized Job Bundle

The final bundle contains more than .nc bytes:

- exact controller program bytes;
- content hash;
- machine and firmware profile hash;
- tool and holder manifest;
- stock, target, fixture, and frame artifacts;
- certificate graph;
- declared runtime assumptions;
- optional preview data generated from the same artifact.

2.1.2.10 Legality

Each IR has a legality predicate. Examples:

Authoring AST legal:

```
no executable closures; all nodes schema-valid
```

Plan IR legal:

```
all units/frames/references resolved; all values finite
```

Toolpath IR legal:

```
all paths continuous; operation provenance total
```

Machine IR legal:

```
every operation supported; all coordinates machine-resolved
```

Controller IR legal:

```
initial modal state explicit; no unknown raw effects
```

Job bundle legal:

```
hashes match; required claims present; assumptions well formed
```

A conversion should fail when it cannot produce legal output.

2.1.2.11 Validation is evidence, not a dialect

A type named `ValidatedProgram` is useful as an API guard, but validation does not create a new language. It creates evidence about an artifact.

```
interface Certified<T> {
  artifact: T;
  artifactHash: Hash;
  claims: readonly Claim[];
```

```
evidence: readonly Evidence[];
}
```

Different claims attach at different stages. Path continuity evidence belongs to a toolpath artifact; modal equivalence belongs to controller bytes; runtime identity belongs to an execution instance. One global boolean brand cannot express this graph.

2.1.2.12 Exercises

1. Assign each concern to an IR level: source span, scallop tolerance, tool-center curve, G17 plane, decimal formatting, upload filename.
2. Write legality predicates for a probing node in Plan IR and Machine IR.
3. Explain why a preview mesh should reference an artifact hash.
4. Give an example of information that must survive planning for later diagnostics.
5. Design an IR conversion that is partial and returns structured diagnostics.
6. Compare a multi-level IR with a universal stringly typed operation list.

Checkpoint - staging and IR. State what remains after JavaScript evaluation, why one universal IR is difficult to validate, and which facts belong in intent, toolpath, machine, and controller dialects.

2.2 Composable paths, effectful actions, and explicit state

A geometry API becomes unsafe when it silently treats every sequence of points as a complete machine program. The next concepts separate geometric composition from machine effects.

Definition - path. A path is a parameterized geometric curve in a named frame. It specifies positions or poses as a function of an abstract progress parameter, not of time.

Small example. $\gamma(u) = (10u, 0, 0)$ for $u \in [0, 1]$ is a line from $(0, 0, 0)$ to $(10, 0, 0)$.

CAM example. The same circular path may be used as a cutting contour, a probing path, or a preview guide. Those actions have different semantics even though their geometry is identical.

Definition - trajectory. A trajectory is a pose or machine configuration indexed by time. A path becomes a trajectory only after a time parameterization and machine-dynamics interpretation are supplied.

CAM example. Two trajectories can follow the same path with different feed schedules, accelerations, cycle times, and following errors.

Definition - action. An action is an operation that changes or observes machine or process state. It may contain a path but also includes effectful meaning such as cut, traverse, probe, select tool, or start spindle.

Definition - effect. An effect is an interaction with state or the environment that ordinary input/output types do not fully describe. Effects include changing the active tool, removing stock, reading a probe, altering modal state, uploading bytes, or consuming time.

Small example. `readFile()` returns bytes but also depends on external filesystem state.

CAM example. `probeSurface()` returns a measured coordinate and changes the knowledge state on which later work-offset commands depend.

Definition - typestate. Typestate represents an object's or protocol's state in the static type system so that only operations valid in that state can be expressed.

Small example. A file handle typed as `Closed` cannot be read until an open operation produces an `Open` handle.

CAM example. A program builder can require a `Homed` state before constructing motion.

Definition - indexed command. An indexed command records both its pre-state and post-state in its type, such as `Cmd<Before, After, Result>`.

CAM example. `Cut<SpindleRunning, SpindleRunning>` cannot follow `StopSpindle<SpindleRunning, SpindleStopped>` without a new start command.

Definition - single static assignment (SSA) state token. An SSA state token is a single-assignment value consumed and produced by state-changing IR operations. It makes effect order explicit in a dataflow graph.

CAM example. A probe result and the state token produced by probing both flow into `setWorkOffset`; a scheduler cannot move the offset command before the probe without breaking dependencies.

Student warning - types are not sensors. Typestate and state tokens prove facts about the *constructed program*. They do not prove that the physical machine is homed, that T1 is actually installed, or that the controller did not reboot after compilation. Those facts become runtime assumptions and checks in Chapter 4.

2.2.1 Paths and Curves as Composable Objects

Section goals. The reader should be able to model paths categorically, distinguish path image from parameterization, identify the failure of approximate equality, and define safe composition contracts.

2.2.1.1 Paths as arrows

A path has a start and end pose:

$$p : A \rightarrow B.$$

If $q : B \rightarrow C$, then composition is defined:

$$q \circ p : A \rightarrow C.$$

A stationary path $\text{id}_A : A \rightarrow A$ acts as an identity, and composition is associative. This is the free-category intuition behind a path builder.

```
interface Path<F extends FrameId> {
  frame: F;
  start: Point3<F>;
  end: Point3<F>;
  segments: readonly Segment<F>[];
}
```

A builder that derives end from appended segments establishes a valuable representation invariant: the declared endpoint cannot disagree with the final segment.

2.2.1.2 Geometry versus parameterization

A curve is often written:

$$\gamma : [0, 1] \rightarrow \mathbb{R}^3.$$

Two parameterizations can trace the same geometric image at different rates. For geometric path equivalence, curves are commonly considered modulo monotone reparameterization. Concatenating two curves changes parameterization even when the physical trace is unchanged.

This matters because a geometric path is not a trajectory. Time enters later through a monotone map:

$$s : [0, T] \rightarrow [0, 1], \quad x(t) = \gamma(s(t)).$$

Velocity, acceleration, jerk, and controller following error belong to $s(t)$ and machine dynamics, not to the curve alone.

2.2.1.3 Segment algebra

Useful primitive segments include:

- line segments;
- circular or helical arcs with geometric center, axis, and sweep;
- splines with declared basis and knot domain;
- polylines with explicit approximation provenance;
- stationary segments or dwells at a pose.

Controller-specific arc offsets such as I, J, and K do not belong in geometric path IR. They are one encoding of the same geometry under a selected plane and endpoint convention.

2.2.1.4 Exact joins and approximate joins

Suppose path composition accepts endpoints when:

$$d(a.end, b.start) < \varepsilon.$$

This relation is not transitive. It cannot serve as exact object identity in a category. Repeated approximate joins can drift, and topology construction can merge unrelated endpoints.

Safer designs include:

1. Canonical snapping to exact grid or symbolic point identities.
2. Explicit join witnesses recording the displacement.
3. A path result carrying accumulated endpoint uncertainty.
4. Exact rational or integer coordinates for topology decisions.
5. A metric-enriched algebra where composition returns a new error bound.

```
interface JoinWitness<F extends FrameId> {
    leftEnd: Point3<F>;
    rightStart: Point3<F>;
    displacement: Mm;
```

```
repair: "none" | "snap-right-to-left" | "insert-link";
}
```

An inserted link is not merely a numerical repair. It is a new motion that needs cut or traverse semantics and clearance checking.

2.2.1.5 Reversal and orientation

A geometric path can often be reversed, but a machining action may not be reversible. Direction affects:

- climb versus conventional milling;
- lead-in and lead-out geometry;
- cutter compensation;
- helical entry;
- probe direction;
- one-way process constraints;
- stock engagement.

Directionality belongs in metadata or the action type. An optimizer must not reverse paths solely because the point list is reversible.

2.2.1.6 Path continuity is not enough

A syntactically continuous path may still be invalid:

- an arc's declared endpoint may not lie on its circle;
- a polyline segment may include a first point inconsistent with the cursor;
- a spline may have a singular or non-finite evaluation;
- curvature may exceed process or dynamics limits;
- the path may cross forbidden material;
- the frame may be inconsistent.

Distinguish representation invariants from semantic properties.

2.2.1.7 Worked example: arc witness

An arc segment can be represented as:

```
interface ArcSegment<F extends FrameId> {
    to: Point3<F>;
    center: Point3<F>;
    axis: UnitVec3;
    sweep: Radians;
}
```

A checker given the start point verifies:

1. the axis is unit length within a certified bound;
2. start and end radial distances agree within tolerance;
3. the declared sweep rotates the start radius to the end radius;
4. any axial displacement is consistent with a helical interpretation;
5. all values are finite;
6. the chosen linearization, if any, satisfies its sagitta bound.

2.2.1.8 Design rules

Design rule

Use path composition to enforce endpoint discipline, but do not confuse numerical proximity with equality or structural continuity with machining safety. Make every repair an explicit, semantically classified operation.

2.2.1.9 Exercises

1. Prove associativity of exact segment-list concatenation.
2. Explain why parameterized-curve equality is too strict for geometric paths.
3. Design a metric-aware result type for approximate concatenation.
4. Give three reasons a path reversal may be illegal.
5. Specify checks for a helical arc.
6. Construct three endpoints that demonstrate non-transitivity of tolerance joins.
7. Explain why automatically inserting a line between discontinuous cut paths can be dangerous.

2.2.2 Actions, Effects, and Indexed Commands

Section goals. The reader should be able to model commands as effectful computations, explain Kleisli composition, and use indexed state to encode legal command sequencing.

2.2.2.1 A path is not an action

The same curve can denote a rapid traverse, cutting feed, probe motion, inspection scan, or air move. The geometry alone does not determine effects. A machining command changes machine and stock state, can fail, emits a trace, and may produce a value.

A simple state-and-error command type is:

$$M(A) = \Sigma \rightarrow \text{Result}(A \times \Sigma, E).$$

In TypeScript:

```
type Command<A> =
  (state: MachineState) =>
    Result<{ value: A; state: MachineState }, CommandError>;
```

A probe command returns a measurement; a cut commonly returns void but changes stock and pose.

2.2.2.2 Why ordinary composition fails

Let:

$$f : A \rightarrow M(B), \quad g : B \rightarrow M(C).$$

Ordinary composition $g \circ f$ is ill typed because f returns $M(B)$ while g expects B . A monad supplies `bind`:

$$\text{bind} : M(B) \times (B \rightarrow M(C)) \rightarrow M(C).$$

For a result-and-state computation, `bind` propagates errors and threads the updated state into the next command.

```
function bind<A, B>(ma: Command<A>, f: (a: A) => Command<B>): Command<B> {
  return state0 => {
    const ra = ma(state0);
    if (!ra.ok) return ra;
    return f(ra.value.value)(ra.value.state);
  };
}
```

This is Kleisli composition. Monads provide a disciplined way to compose effectful computations [R10, R11].

2.2.2.3 CNC effects

A realistic command effect includes more than mutable state:

```
interface CommandResult<A> {
  value: A;
  state: MachineState;
  trace: readonly Event[];
  assumptionsUsed: readonly AssumptionRef[];
  claimsProduced: readonly Claim[];
}
```

Potential effects include:

- failure and alarms;
- machine-state mutation;
- stock removal;
- logging and provenance;
- nondeterministic measurement;
- asynchronous controller interaction;
- resource use and time;
- uncertainty propagation.

An effect system can record which operations read or modify which resources [R14].

2.2.2.4 Indexed commands

A plain state monad does not express that the type of legal state changes. A parameterized or indexed command type does:

$$\text{Cmd}(S_{\text{before}}, S_{\text{after}}, A).$$

Composition requires the post-state index of one command to match the pre-state index of the next [R13]. Type-state uses the same idea to restrict operations by protocol state [R12].

```
type StartSpindle<S extends Homed & HasTool> =
  Cmd<S, S & SpindleRunning, void>;

type Cut<S extends Homed & HasTool & SpindleRunning & KnownWcs> =
  Cmd<S, S & AtPathEnd, void>;
```

The source API can make impossible sequences difficult to construct.

2.2.2.5 Limits of type-level safety

TypeScript types are erased. A forged value, unchecked deserialization, stale controller state, or physical mismatch can violate the index. Type-level typestate proves a property of the host program under its model; it does not prove the machine is physically in that state.

Runtime preflight must re-establish assumptions at the execution boundary.

2.2.2.6 Algebraic effects versus monolithic state

One large MachineState can become difficult to reason about. An alternative is to describe effects separately:

```
type Effect =
  | ReadPose
  | RequireHomed
  | SelectTool
  | SetSpindle
  | Move
  | RemoveStock
  | Probe
  | EmitDiagnostic;
```

Handlers interpret these effects for simulation, validation, preview, or controller execution. This can improve modularity, but it still requires a coherent ordering and state model. The key is not the syntax of monads versus effect handlers; it is the explicit representation of effects and their laws.

2.2.2.7 Worked example: probe then offset

A probe returns a measured coordinate:

```
type Probe = Cmd<ProbeReady, ProbeReady, Measurement>;
type SetWorkOrigin =
  (m: Measurement) => Cmd<ProbeReady, KnownWcs, void>;
```

Kleisli composition connects the returned value to the next operation:

```
const establishWcs = probeSurface(direction, maxTravel)
  .flatMap(measurement => setWorkOrigin(measurement));
```

The data dependency is explicit. A command list that merely places probe before set offset without binding the result is semantically incomplete.

2.2.2.8 Exercises

1. Implement pure and bind for a result-and-state command type.
2. State the three monad laws and explain their practical value for command sequencing.
3. Design indexed states for unhomed, homed, tool-selected, and spindle-running modes.
4. Explain one invariant that TypeScript typestate cannot establish about a physical machine.
5. Model a probe command that can return contact, no contact, or alarm.
6. Compare a state monad with an explicit effect algebra for CAM.

2.2.3 State Tokens, SSA, and Ordering

Section goals. The reader should be able to use an SSA-style state token to make effects explicit, construct def-use dependencies, and identify which transformations are legal around stateful com-

mands.

2.2.3.1 The machine state as a linear resource

A practical IR need not encode every typestate distinction in TypeScript generics. It can make ordering explicit with a state token. Each effectful operation consumes one token and produces the next.



Figure 2.2: An SSA-style machine state token threads through commands.

```
%s0 = machine.initial
%s1 = machine.require_homed %s0
%s2 = tool.select %s1 @T1
%s3 = spindle.start %s2 12000rpm
%s4 = motion.cut %s3 path=@p feed=400
%s5 = spindle.stop %s4
```

The token is not the full runtime state. It is an ordering and dependency witness.

2.2.3.2 Relation to SSA

Static single assignment form gives each value one definition and makes data dependencies explicit [R15]. A machine-state token applies the same discipline to side effects. Operations cannot be silently reordered across token dependencies.

Pure geometry computations need no token:

```
%p1 = path.offset %feature by=1.2mm
%p2 = path.reverse %p1
```

Effectful program construction does:

```
%s1 = motion.traverse %s0 to=%p1.start
%s2 = motion.cut %s1 path=%p1
```

2.2.3.3 Multiple resources

One global token is simple but can over-serialize independent work. A richer IR can use separate tokens for:

- controller state;
- stock state;
- tool library state;
- measurement environment;
- diagnostic/provenance stream.

However, splitting resources requires precise alias and commutativity rules. Two cuts that affect disjoint stock regions might commute geometrically but not mechanically if they change support or linking clearance.

For an initial implementation, a single machine/process token is usually safer.

2.2.3.4 Probe values and control flow

SSA makes measurements explicit values:

```
%m, %s1 = probe.toward %s0 direction=-Z max=10mm
%t      = frame.from_probe %m datum=@top_surface
%s2     = frame.install %s1 %t as=G54
```

A failed probe can branch:

```
switch %m.status:
  contact -> continue
  no_contact -> abort
  alarm -> quarantine_session
```

The IR can represent merge points with block arguments or phi-like values. A validator must ensure that state invariants hold along every incoming path.

2.2.3.5 Effect summaries

Optimization and scheduling require summaries:

```
interface EffectSummary {
  reads: readonly ResourceRegion[];
  writes: readonly ResourceRegion[];
  requires: readonly Predicate[];
  ensures: readonly Predicate[];
  mayFail: readonly FailureClass[];
}
```

A pass may commute two operations only if their effects are independent and the reordered preconditions remain valid.

2.2.3.6 Linear and affine use

A state token should usually be consumed exactly once. Duplicating it would fork the machine into two contradictory futures. Dropping it can omit required shutdown or recovery behavior.

A linear type system enforces exactly-once use; an affine system permits dropping but not duplication. Most mainstream TypeScript code cannot enforce this statically, but the IR verifier can check def-use counts.

2.2.3.7 Worked example: preventing an unsafe reorder

Suppose an optimizer sees:

```
%s1 = spindle.start %s0 12000rpm
%s2 = motion.cut %s1 @p
%s3 = spindle.stop %s2
```

Moving `spindle.stop` before the cut would require consuming `%s1` and producing a new state passed to the cut. The cut's `SpindleRunning` requirement fails. The token graph exposes the invalid transformation.

2.2.3.8 Exercises

1. Convert a small imperative command list into state-token SSA.
2. Explain why duplicating a machine state token is unsound.
3. Design a verifier for single-definition and single-consumption token use.
4. Show a legal reordering of two pure geometry operations.
5. Give a case where two cuts in disjoint regions still should not commute.

6. Model a branch after probing and the merge of its safe paths.

Checkpoint - effects. Distinguish a path, trajectory, and action; explain what an SSA state token prevents; and identify one fact that types cannot establish about the physical machine.

2.3 Provenance, identity, and an extensible API

Once a compiler has many artifacts and passes, two new questions appear: “Where did this value come from?” and “Is this exactly the same artifact that was checked?” Provenance and content identity answer them.

Definition - provenance. Provenance is structured information linking an output object to source spans, input artifacts, pass versions, parameters, and intermediate decisions.

Small example. A compiler diagnostic points from a machine-limit violation back through a generated toolpath to the source pocket call that requested it.

CAM example. A suspicious rapid can be traced to the linker pass, the two operations it connected, the stock snapshot used for planning, and the machine profile that supplied clearance rules.

Definition - content addressing. Content addressing identifies an artifact by a cryptographic digest of its exact bytes and encoding metadata. A change in content produces a different identity.

Example 1. Renaming an identical file need not change its content identity.

Example 2. Changing one emitted coordinate or line ending changes the serialized-job hash and invalidates claims that depend on those bytes.

Definition - capability. A capability is explicit authority to perform a class of operation. Capability-based APIs expose only the operations a phase or plugin is permitted to invoke.

CAM example. A planning plugin receives geometry construction and diagnostic capabilities but not controller transport or filesystem authority.

Definition - escape hatch. An escape hatch bypasses part of the normal typed or checked model, for example a raw controller command. It must be visibly marked, narrowly scoped, and treated conservatively by analyses.

CAM example. A raw M code whose effects are not derived by a trusted parser invalidates exact modal-state claims; a user-written effect annotation is an assumption, not proof.

2.3.1 Provenance, Identity, and Reproducibility

Section goals. The reader should be able to design total provenance, content-address artifacts, distinguish identity from display names, and bind diagnostics and certificates to exact inputs.

2.3.1.1 Provenance is semantic data

When a final move fails a check, the system should explain which source operation, feature, strategy, and compiler pass produced it. Provenance is therefore not an optional debug string. It supports:

- diagnostics;
- certificate dependency graphs;
- incremental recompilation;
- audit and review;

- visual selection in the UI;
- comparison between compiler versions;
- fault localization.

2.3.1.2 A provenance chain

```
interface Provenance {
  sourceSpan?: SourceSpan;
  authoringNode?: NodeId;
  operation?: OperationId;
  feature?: FeatureId;
  strategy?: { id: string; version: string };
  passHistory: readonly PassRecord[];
  parentArtifacts: readonly Hash[];
}
```

Every transformation either preserves provenance, refines it, or introduces a synthetic operation with an explicit reason such as `safety-retract` or `postprocessor-spindle-stop`.

2.3.1.3 Content addressing

An artifact hash should cover canonical serialization of:

- semantic content;
- schema version;
- relevant configuration;
- referenced artifact hashes.

A display filename is not identity. Two jobs named `part.nc` may differ; the same job may have several names.

Canonical serialization must define:

- map key order;
- number format;
- treatment of negative zero;
- Unicode normalization;
- omitted versus null fields;
- endianness for binary data;
- schema versioning.

2.3.1.4 Reproducible passes

A pass result should be a function of explicit inputs:

$$O = F(I, C, V, S),$$

where C is configuration, V identifies the pass implementation, and S is any declared random seed. Hidden time, locale, thread interleaving, or unordered iteration can break reproducibility.

Reproducibility does not guarantee correctness, but it makes evidence, debugging, and regression analysis tractable.

2.3.1.5 Machine and firmware identity

A machine profile named `z1` is not sufficient if firmware revisions change protocol or motion semantics. The executable bundle should identify:

- machine model and configured limits;
- controller dialect and firmware compatibility range;
- relevant capability flags;
- postprocessor version;
- calibration profile version;
- known semantic deviations.

Runtime preflight compares live identity with the bundle assumptions.

2.3.1.6 Provenance through optimization

An optimizer may merge, split, or reorder paths. It should retain a many-to-many mapping:

```
interface OriginMap {
  outputRange: CommandRange;
  inputs: readonly {
    artifact: Hash;
    operation?: OperationId;
    pathRange?: ParameterInterval;
  }[];
  transformation: string;
}
```

This allows a gouge counterexample at final block 813 to highlight the source finishing operation and the arc-fitting pass that changed it.

2.3.1.7 Worked example: cache key

A toolpath-planning cache key should include more than the feature geometry:

```
hash(
  intentIR,
  targetMeshHash,
  stockHash,
  toolAssemblyHash,
  strategyIdAndVersion,
  planningTolerance,
  machine-relevant constraints,
  deterministicSeed
)
```

Omitting the tool hash can reuse a path computed for a different cutter. Omitting the tolerance can attach an outdated witness to a stricter request.

2.3.1.8 Exercises

1. Design a canonical serialization rule for floating-point numbers.
2. Explain why a filename is not an artifact identity.
3. List the inputs to a deterministic contour-planning cache key.
4. Define provenance for an automatically inserted retract.

5. Describe how a certificate becomes stale after a machine-profile change.
6. Give one source of nondeterminism in JavaScript object iteration or parallel planning and show how to control it.

2.3.2 Designing an Extensible CAM API

Section goals. The reader should be able to design an API that is composable, explicit about context, extensible through strategies and kernels, and resistant to invalid states.

2.3.2.1 Separate stable concepts from plugins

The stable API should expose semantic concepts:

- frames and units;
- stock, target, fixtures, and tools;
- manufacturing features and operations;
- paths and motion classes;
- tolerances, assumptions, and diagnostics;
- compilation and certification stages.

Strategies should be plugins:

```
interface Strategy<I extends OperationIntent, W> {
  id: string;
  version: string;
  supports(intent: I, context: PlanningContext): SupportResult;
  plan(intent: I, context: PlanningContext): Result<{
    paths: readonly PlannedPath[];
    witness: W;
  }, Diagnostic[]>;
}
```

A strategy may propose output; it should not mint final safety claims.

2.3.2.2 Kernel independence

Geometry algorithms should depend on interfaces rather than one mesh library:

```
interface SurfaceOracle<F extends FrameId> {
  bounds(): Box3<F>;
  heightAtXY(x: Mm, y: Mm): Interval<Mm> | "outside";
  closestPoint(p: Point3<F>): ClosestPointBound<F>;
  raycast(ray: Ray3<F>): readonly HitBound<F>[];
}
```

Different kernels can implement exact B-rep queries, mesh acceleration structures, interval fields, or remote computation. The semantic contract says what bounds the result provides.

2.3.2.3 Explicit contexts

Avoid ambient mutable settings such as “current feed,” “current tool,” or “current frame” inside planners. Pass immutable contexts:

```
interface PlanningContext {
  machine: MachineProfileRef;
```

```

    setup: SetupRef;
    activeTool: ToolRef;
    frame: FrameId;
    toleranceBudget: ErrorBudget;
    stockState: StockStateRef;
    seed: bigint;
}

```

The authoring API may offer convenient scopes, but elaboration should convert them into explicit fields.

2.3.2.4 Result types and diagnostics

Exceptions are appropriate for programmer errors and violated internal invariants. User-level compilation failures should be values:

```

type CompileResult<T> =
  | { ok: true; value: T; diagnostics: readonly Diagnostic[] }
  | { ok: false; diagnostics: readonly Diagnostic[] };

```

A diagnostic should include severity, stable code, source provenance, artifact context, quantitative details, and suggested remediation.

2.3.2.5 Escape hatches

Advanced users may need raw controller operations. An escape hatch should be isolated:

```

interface RawControllerBlock {
  text: string;
  declaredDialect: DialectId;
  declaredEffects?: EffectSummary;
  provenance: Provenance;
}

```

Self-declared effects are assumptions, not proof. An independent parser may establish actual effects for a supported grammar. Otherwise, affected analyses become unknown and production certification should fail closed.

2.3.2.6 API layers

A pleasant system can offer three layers:

Feature API for most jobs:

```

roughPocket(feature, { tool: "T1", strategy: "adaptive" });
finishPocket(feature, { tool: "T2" });

```

Path API for custom machining geometry:

```

cut(path, { tool: "T1", feed: mmPerMin(350), intent: "finish" });

```

Machine API for exact canonical actions:

```

program.append(traverse(path, clearance));
program.append(dwel(seconds(1)));

```

Lower layers carry more responsibility and require stronger evidence.

2.3.2.7 Law-driven API testing

Algebraic laws become property tests:

- path identity and associativity under exact endpoints;
- frame-transform identity and inverse;
- serialization round trip;
- deterministic planning under fixed inputs;
- pass idempotence for canonicalization;
- effect-summary conservativeness;
- certificate invalidation under changed dependencies.

The API design should make laws visible enough to test.

2.3.2.8 Worked example: compiling explicitly

```
const authored = evaluateScript(source, scriptEnvironment);
const plan = elaborate(authored.ast, projectContext);
const intent = normalizeIntent(plan.value);
const proposed = strategyRegistry.plan(intent, planningContext);
const checkedPaths = checkPlanningWitness(intent, proposed);
const scheduled = schedule(checkedPaths, schedulingPolicy);
const machine = lowerToMachine(scheduled, z1Profile);
const controller = lowerToMakera(machine, makeraDialect);
const bytes = serialize(controller);
const final = validateFinalArtifact({
  intent,
  machine,
  controller,
  bytes,
  setup,
});
```

Each call has an explicit input and output. There is no hidden global “current job” whose meaning changes between preview and export.

2.3.2.9 Exercises

1. Design a strategy interface for constant-Z waterline finishing.
2. Define a geometry-kernel query with a sound outer bound.
3. Explain when to use an exception versus a diagnostic result.
4. Design three API layers for drilling.
5. Specify how a raw controller block affects certification.
6. Write five algebraic or metamorphic properties for the API.
7. Explain why plugin version identity belongs in provenance.

Checkpoint - identity and API. For any generated rapid, trace it to its source, pass, configuration, and artifact hashes; then explain how a raw command changes the analysis result.

2.4 Chapter synthesis: the pocket through the language boundary

The authoring script is not itself the manufacturing program. Its job is to create the first inert artifact.

```

const plan = cam.plan(({ stock, tool, pocket, finish }) => {
  const blank = stock.box({ x: mm(60), y: mm(40), z: mm(8) });
  const t1 = tool.endMill({ diameter: mm(3.175), flutes: 2 });

  const rough = pocket.rect({
    center: point2(mm(30), mm(20), WorkFrame),
    width: mm(30),
    height: mm(20),
    depth: mm(4),
    radialAllowance: mm(0.20),
    axialAllowance: mm(0.20),
    tool: t1,
  });

  finish.contour({ from: rough, allowance: mm(0.0), tool: t1 });
  return { blank, rough };
});

```

After isolated evaluation, a simplified AST might be:

```

{
  "kind": "Plan",
  "operations": [
    {
      "kind": "RectPocket",
      "frame": "work:g54",
      "widthMm": 30,
      "heightMm": 20,
      "depthMm": 4,
      "radialAllowanceMm": 0.2,
      "axialAllowanceMm": 0.2,
      "tool": "tool:t1",
      "source": { "file": "job.cam.js", "line": 5, "column": 17 }
    }
  ]
}

```

Elaboration then resolves defaults, tool references, units, frames, and target geometry. The result is a different artifact with a different hash. Later IRs may contain paths and actions, but they should still retain provenance to this source node.

2.4.1 The same pocket at three IR levels

An elaborated intent node answers *what result is required*:

```

PocketIntent {
  targetRegion: pocketInterior,
  floorZ: mm(-4),
  roughAllowance: { radial: mm(0.2), axial: mm(0.2) },
  protectedTarget: targetPart,
  requiredToolClass: "flat-end-mill",
}

```

A geometric toolpath node answers *which curves witness the operation*:

```

GeometricOperation {

```

```

feature: pocketIntentId,
cutPaths: [offsetLoop0, offsetLoop1, rasterFill],
coverageWitness: coverageCells,
chordErrorBound: mm(0.01),
}

```

A scheduled action program answers *in what order and machine state the curves execute*:

```

%s0 = requireHomed %machine
%s1 = selectTool %s0 @T1
%s2 = startSpindle %s1 12000rpm
%s3 = traverse %s2 @entryLink
%s4 = cut %s3 @offsetLoop0 feed=400mm/min
%s5 = cut %s4 @offsetLoop1 feed=400mm/min
%s6 = stopSpindle %s5

```

The representations are not redundant. Each makes one class of question explicit and postpones details that do not yet belong.

2.4.2 Why several IRs are easier than one universal IR

A universal node with dozens of optional fields appears flexible:

```

interface UniversalOperation {
  geometry?: unknown;
  feed?: number;
  spindle?: number;
  gcode?: string;
  target?: unknown;
  effects?: string[];
  // ...
}

```

It is difficult to know which combinations are legal and what the node means. Separate dialects move invalid combinations out of the representable space:

Intent IR	asks: what result is required?
Toolpath IR	asks: what geometric curves witness that result?
Scheduled IR	asks: in what order, with which entries and links?
Machine IR	asks: what trajectories and capabilities exist on this machine?
Controller IR	asks: what explicit controller operations realize them?
Job bytes	asks: what exact encoding will be transferred and executed?

2.4.3 Three representation invariants

1. Every coordinate and path carries a frame.
2. Every state-changing action consumes and produces an explicit state token or equivalent ordered effect.
3. Every artifact has content identity and provenance to its inputs and pass configuration.

These invariants do not prove machining safety. They make later proofs possible and prevent ambiguity from being created by the representation itself.

2.4.4 Common misconceptions

1. “TypeScript types prove the machine is in that state.” Types constrain program construction; runtime pre-flight must still compare the model with the physical/controller state.

2. **“A path builder proves the path is safe.”** It can prove a representation invariant such as endpoint consistency, not fixture clearance or target protection.
3. **“The compiler can infer effects from arbitrary JavaScript.”** General JavaScript has ambient effects. End the scripting phase at an inert AST boundary.
4. **“A filename identifies the job.”** Identity must bind exact content, schema, and relevant configuration.

2.4.5 Chapter exercises

1. Design an AST node for a probing operation. Separate source convenience fields from the elaborated fields needed by later passes.
2. Give one example in which the same path has three different action meanings.
3. Write an indexed-command type for `SelectTool`, `StartSpindle`, `Cut`, and `StopSpindle`; show one sequence that should fail to type-check.
4. Draw an SSA-style state-token graph for “probe Z, set G54 Z, start spindle, cut pocket.”
5. Define a content-addressed cache key for a toolpath-planning pass. List every input whose change must invalidate the result.

Chapter 3

Passes, Proof Obligations, and Certificates

Chapter goal. Turn a sequence of transformations into an assurance argument. By the end of the chapter, you should be able to write a semantic contract for a compiler pass, distinguish verification of a transformer from validation of one transformation result, derive local assertions and invariants, use abstract interpretation for whole-program properties, reason conservatively about geometry and numerical error, and design a certificate whose claims are precise and independently checkable.

A compiler can be beautifully typed and still be wrong. The difficult question is not whether a pass returned an object of the expected interface. It is whether the output preserves or refines the input's meaning under explicit assumptions and bounds.

This chapter follows one artifact through the middle and back end of the compiler. Each pass produces an output and a proof obligation. Some obligations are discrete and comparatively easy: units are resolved, modal state is explicit, serialization round-trips. Others are geometric and difficult: a swept volume avoids a fixture for every point on a continuous trajectory under transform uncertainty. The architecture must represent both honestly.

Definition - proof obligation. A proof obligation is a proposition that must be established before an artifact or transformation may be accepted. A pass contract generates proof obligations; a checker discharges or rejects them.

The chapter also separates the **producer** from the **checker**. The producer may be heuristic, optimized, parallel, and large. The checker should be smaller, deterministic, conservative, and based on a clear proposition. That separation is the practical route to certificate-carrying CAM.

3.1 Pass contracts, legality, and elaboration

A compiler pass is commonly written as a function $I \rightarrow O$. That type omits the reason the output should be trusted. We therefore enrich the concept.

Definition - compiler pass. A compiler pass is a transformation or analysis over an IR. In an assurance-oriented compiler, a pass also declares the accepted input language, produced output language, semantic relation, assumptions, diagnostics, error contribution, and evidence or checker.

Definition - pass contract. A pass contract is the explicit statement of what relation must hold between a particular input and output.

Small example. Constant folding must preserve expression value.

CAM example. Arc linearization must preserve endpoints and orientation while bounding geometric deviation under a named metric.

Definition - translation validation. Translation validation checks one actual pass result against its actual input. It differs from proving the transformation implementation correct for all possible inputs.

Example 1. After modal compression, independently interpret both controller programs and compare their canonical traces.

Example 2. After path reordering, check precedence, stock dependencies, and operation coverage for the produced schedule.

Definition - legality. An IR is legal for a stage when every operation, type, attribute, and dependency satisfies that stage's rules. A complete lowering pass must either convert an operation, retain it under an explicitly allowed mixed-dialect rule, or reject the program.

Definition - static semantics. Static semantics comprises rules checked without executing the physical job: name resolution, dimensional consistency, frame-graph consistency, finite values, supported operations, and representation invariants.

3.1.1 Complete example: validating arc linearization

Suppose the input artifact contains a circular arc a and the pass emits a polyline $p_0, \dots, p_n$.

Input. Arc center, radius, plane, start and end points, orientation, and an allowed Hausdorff error ε .

Output. A polyline with the same action label and process parameters.

Witness. For each line segment, the corresponding arc-parameter interval; a bound on maximum sagitta; exact endpoint correspondence; and the subdivision policy version.

Checker obligations.

1. The first and last polyline points equal the arc endpoints in the canonical numeric representation.
2. Parameter intervals are ordered, cover the complete arc once, and preserve orientation.
3. Every segment's sagitta bound is at most ε .
4. No action, tool, feed, frame, or provenance identity changed.
5. The output contains only finite coordinates and legal segment counts.

Claim. The polyline path is a bounded geometric refinement of the arc under Hausdorff distance, with bound ε .

The transformer may use any efficient subdivision algorithm. Acceptance depends on the checker, not on trusting that algorithm's implementation.

3.1.2 Pass Contracts and Translation Validation

Section goals. The reader should be able to define a compiler pass by a semantic relation, distinguish proof of a pass implementation from validation of one pass execution, and design witnesses and independent checkers.

3.1.2.1 A pass is more than a function

A software interface often presents a pass as:

```
transform(input: I): O
```

The semantic interface is richer. A pass claims that its output is related to its input by a relation R :

$$R(I, O).$$

A certifying pass should produce both output and evidence:

```
interface PassResult<O, W> {
  output: O;
  witness: W;
  diagnostics: readonly Diagnostic[];
}
```

An independent checker validates the relation:

```
check(input: I, output: O, witness: W): CheckResult
```

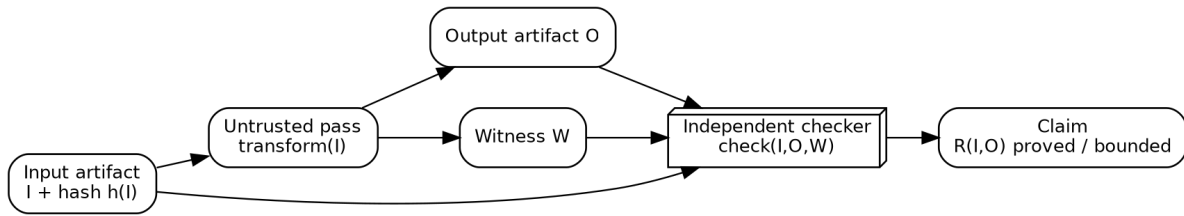


Figure 3.1: An untrusted pass produces output and a witness; a checker establishes the pass relation.

3.1.2.2 Classes of pass relation

Different passes require different correctness statements.

3.1.2.2.1 Exact semantic preservation

$$\text{Sem}(O) = \text{Sem}(I).$$

Appropriate for canonicalization, alpha-renaming, or modal compression under a fixed interpreter.

3.1.2.2.2 Trace refinement

$$\text{Traces}(O) \subseteq \text{Traces}(I).$$

Appropriate for scheduling or selecting one implementation allowed by an abstract operation.

3.1.2.2.3 Bounded geometric refinement

$$d(\text{Geom}(O), \text{Geom}(I)) \leq \varepsilon.$$

Appropriate for sampling, fitting, and numerical lowering.

3.1.2.2.4 Witness satisfaction

$$O \in \llbracket I \rrbracket.$$

Appropriate when a planner synthesizes one implementation of an intent.

3.1.2.2.5 Feasible optimization

$$\text{Feasible}(O) \wedge R(I, O) \wedge J(O) \leq J(I).$$

Appropriate for reordering, link shortening, or feed optimization. “Optimal” additionally requires a lower bound or proof of global optimality.

3.1.2.3 Verified compiler versus validating compiler

A verified compiler proves, once and for all, that the implementation of each pass satisfies its theorem for every accepted input. CompCert is the canonical example of pass-by-pass semantic preservation [R17]. This offers strong assurance but is expensive, particularly for evolving geometric algorithms.

Translation validation checks each actual compilation result. Pnueli and colleagues introduced the approach as an alternative to proving the optimizer itself [R18]. A validator receives I and O and decides whether the required relation holds. Alive2 applies this style to LLVM optimizations [R19].

For an experimental CAM compiler, translation validation is a pragmatic first target:

- strategy and optimization implementations remain ordinary TypeScript;
- each output carries a witness;
- a smaller checker validates the witness;
- failed validation rejects the artifact;
- frequently used checkers can later be mechanized.

3.1.2.4 Checker soundness

The key theorem is:

$$\text{check}(I, O, W) = \text{accept} \implies R(I, O).$$

The checker need not be complete. It may reject valid outputs when evidence is insufficient. In safety-oriented compilation, false rejection is usually preferable to false acceptance.

Checker implementation should be simpler than the producer. A path planner may search thousands of alternatives; its checker should verify one proposed path. A nonlinear optimizer may use complex heuristics; its checker should validate feasibility and objective value.

3.1.2.5 Pass descriptors

```
interface PassDescriptor {
  id: string;
  version: string;
  inputSchema: string;
  outputSchema: string;
  relation: RelationId;
  deterministic: boolean;
  configurationHash: Hash;
  implementationHash: Hash;
```



```
checker: CheckerRef;
}
```

The relation identity is as important as the code version. A pass called `refine` might change from a Hausdorff guarantee to a normal-deviation guarantee; the certificate must not treat the claims as interchangeable.

3.1.2.6 Local and global validation

Some pass properties are local:

- every arc endpoint lies on its circle;
- every formatted number reparses to the intended value within a bound;
- every operation is supported by the machine profile.

Others are global:

- path reordering preserves all precedence constraints;
- stock-dependent links remain collision-free in sequence;
- modal compression preserves the entire interpreted trace;
- cumulative error stays within budget.

A pass checker should be designed around the strongest dependencies of its claim, not around convenient implementation boundaries.

3.1.2.7 Worked example: arc linearization validator

The producer emits a polyline and witness:

```
interface ArcLinearizationWitness {
  radius: Mm;
  sweep: Radians;
  chordTolerance: Mm;
  segmentCount: number;
  roundingBound: Mm;
}
```

The checker verifies:

1. the input arc is valid;
2. the output endpoints match;
3. each output vertex lies on the intended parameterized chord sequence or within a numerical enclosure;
4. the analytic sagitta bound is at most `chordTolerance`;
5. coordinate rounding contributes no more than `roundingBound`;
6. the total metric bound is composed correctly.

It does not trust the producer's stated segment count or radius.

3.1.2.8 Exercises

1. State the pass relation for a path-reversal optimization.
2. Compare the trusted computing base of a verified compiler and a validating compiler.
3. Explain why a checker may be intentionally incomplete.
4. Design a witness for feed-rate clamping.
5. Identify a local and a global obligation for modal compression.

6. Write the soundness theorem for a scheduling validator.

3.1.3 Elaboration and Static Checking

Section goals. The reader should be able to define elaboration, distinguish normalization from validation, and derive a legal Plan IR from a permissive authoring AST.

3.1.3.1 What elaboration does

Elaboration converts convenient surface syntax into explicit semantic data. It resolves omissions and rejects ambiguity before expensive planning begins.

Typical tasks include:

- resolve tool, frame, feature, and geometry names;
- convert all units to canonical internal units;
- expand defaults and scope-derived settings;
- assign stable identifiers;
- freeze mutable values;
- validate finite numbers and domain restrictions;
- establish source provenance;
- reject duplicate or dangling references;
- normalize orientation and coordinate conventions;
- construct a frame graph and check consistency.

3.1.3.2 Static semantics

A typing judgment can be written:

$$\Gamma \vdash e : \tau,$$

where Γ contains declarations and context. For a point constructor:

$$\frac{\Gamma \vdash x : \text{Length} \quad \Gamma \vdash y : \text{Length} \quad \Gamma \vdash z : \text{Length} \quad \Gamma \vdash F : \text{Frame}}{\Gamma \vdash \text{point}(x, y, z, F) : \text{Point3}\langle F \rangle}.$$

A pocket operation may require a planar boundary, positive depth, compatible tool, and known workpiece frame.

3.1.3.3 Domain checking

Dimensional validity is weaker than domain validity. A feed of NaN mm/min, a negative tool diameter, or a zero spindle speed may carry the right unit but remain invalid.

Centralize domain predicates:

```
const Domain = {
  finiteLength(x: Mm): boolean,
  positiveLength(x: Mm): boolean,
  nonnegativeAllowance(x: Mm): boolean,
  positiveFeed(x: MmPerMin): boolean,
  validRpm(x: Rpm, profile: MachineProfile): boolean,
  validTolerance(x: Mm): boolean,
```

```
};
```

Do not rely on each strategy to rediscover these rules.

3.1.3.4 Frame graph consistency

A frame graph contains edges $T_{B \leftarrow A}$. Elaboration should reject:

- cycles whose composed transform is inconsistent beyond tolerance;
- multiple conflicting paths between frames;
- references to unknown frames;
- transforms with non-finite values;
- non-rigid matrices when rigid transforms are required;
- ambiguous work-coordinate selection.

When uncertainty is present, consistency becomes an enclosure question: do the alternative transform paths overlap within their declared bounds?

3.1.3.5 Defaults and provenance

Every default should become explicit in Plan IR and retain provenance indicating that it came from configuration rather than source text.

```
{
  feed: mmPerMin(400),
  provenance: {
    kind: "default",
    source: "material-library/aluminum-6061.json",
    rule: "slotting-feed:T1",
  }
}
```

This makes re-compilation under a changed material library correctly invalidate downstream artifacts.

3.1.3.6 Normal forms

Elaboration can establish a normal form:

- one canonical unit system;
- one orientation convention for polygon rings;
- explicit closed/open path flags;
- sorted map keys;
- explicit optional fields;
- unique identifiers;
- canonical frame names.

Normalization reduces the number of cases later passes and checkers must handle.

3.1.3.7 Static versus physical checks

Elaboration can prove that a tool reference exists. It cannot prove that the operator loaded that tool. It can prove that a work frame is defined in the project. It cannot prove that the live machine's G54 matches the definition.

The IR should therefore distinguish **compile-time facts** from **runtime assumptions**.

3.1.3.8 Worked example: resolving a pocket

Authoring node:

```
pocket({
  at: [10, 10],
  size: [30, 20],
  depth: 4,
  tool: "T1",
});
```

Elaborated node:

```
{
  kind: "rectPocket",
  id: "op:4f7c...",
  frame: "work:G54",
  origin: point(mm(10), mm(10), mm(0), "work:G54"),
  width: mm(30),
  height: mm(20),
  depth: mm(4),
  tool: { id: "T1", artifactHash: "sha256:..." },
  tolerance: mm(0.02),
  radialAllowance: mm(0),
  axialAllowance: mm(0),
  source: { file: "job.cam.js", line: 12, column: 1 },
}
```

Every omitted fact has become explicit or produced a diagnostic.

3.1.3.9 Exercises

1. Write static rules for a tool definition and a pocket operation.
2. Give three domain errors that unit types do not catch.
3. Design a frame-graph consistency diagnostic.
4. Explain why defaults need provenance.
5. Propose a normal form for polygon rings with holes.
6. Separate compile-time facts from runtime assumptions in a probing setup.

Checkpoint - passes. For one transformation, name its input language, output language, semantic relation, witness, assumptions, checker, and possible rejection diagnostics.

3.2 From intent to toolpaths, machine actions, controller IR, and bytes

The next group of passes turns intent into increasingly concrete motion and bytes. Their names sound procedural, so define them before using them.

Definition - planning. Planning chooses a geometric or process implementation that witnesses a high-level manufacturing intent.

CAM example. A pocket planner selects offsets, raster passes, or adaptive clearing and provides evidence for required coverage and protected boundaries.

Definition - scheduling. Scheduling chooses an order for operations while respecting precedence, resource, stock-state, and process constraints.

CAM example. Internal contours may be cut before an outer contour releases the part; roughing precedes finishing; probing precedes use of the measured offset.

Definition - linking. Linking constructs the motions that connect operations, including entries, retracts, traverses, and descents. A link is not semantically empty: it has a swept volume, duration, machine state, and collision obligations.

Definition - machine lowering. Machine lowering resolves abstract actions against a specific machine profile, kinematics, capabilities, limits, tools, and safe-motion policies.

Definition - controller IR. Controller IR is a structured, explicit representation of controller-level operations before text serialization. It models modal and protocol semantics without relying on string prefixes.

Definition - serialization. Serialization maps structured controller IR to exact bytes, including numeric formatting, comments, line endings, encoding, and checksums.

Small example. The real number $1/3$ must be rounded to a finite decimal representation.

CAM example. Choosing I/J rather than R, inserting modal words, and selecting line endings are semantic compiler decisions, not clerical formatting.

Definition - parse-back validation. Parse-back validation parses the emitted bytes with an independent target-language parser, interprets them, and compares that meaning with the pre-serialization controller IR.

CAM example. If coordinate rounding changes an arc enough to violate the bound, parse-back validation or a quantitative serialization checker must reject the bytes.

3.2.1 Planning Features into Toolpaths

Section goals. The reader should be able to model planning as constrained synthesis, preserve feature provenance, and design checkable witnesses for coverage and target protection.

3.2.1.1 Planner input and output

A planner receives intent, geometry, tools, stock state, process constraints, and an error budget. It returns candidate paths plus evidence.

```
interface PlanningInput<I> {
    intent: I;
    stock: StockStateRef;
    target: GeometryRef;
    fixtures: readonly GeometryRef[];
    tool: ToolAssemblyRef;
    context: PlanningContext;
}
```

The output should separate **cut paths** from **entry and linking**. A feature strategy can focus on material-removal geometry; a scheduler later connects paths under state-dependent clearance constraints.

3.2.1.2 Coverage and offset planning

For a two-dimensional pocket and a cylindrical cutter, centerline regions are often built by morphological offsets. If the pocket region is R and cutter radius is r , a safe center region is approximately:

$$R_c = R \ominus B_r.$$

Roughing passes may cover R_c with raster lines, inward offsets, or adaptive loops. A witness can include the arrangement of covered cells and a bound on uncovered required-removal regions.

Offset geometry must handle topology changes, narrow channels, corners, and holes. Exact predicates or robust integer/rational topology are preferable for decisions such as orientation and intersection.

3.2.1.3 Drop-cutter planning

For three-axis finishing, a drop-cutter method places the tool at sampled XY positions and finds the lowest collision-free Z against the target. The resulting cutter-location field is an approximation to a configuration-space surface.

A credible planner must state:

- tool geometry used in contact computation;
- target representation and its enclosure error;
- XY sampling strategy;
- interpolation between samples;
- residual or refinement criterion;
- behavior near boundaries and vertical walls;
- whether the field is an inner or outer safety bound.

A height sample alone does not prove continuous no-gouge between samples.

3.2.1.4 Constant-scallop intent

A true constant-scallop strategy controls maximum residual cusp height on the target surface. Merely tracing isolines of a distance-like field does not establish a physical scallop bound. The proof must connect:

1. target surface metric;
2. tool contact geometry;
3. field discretization error;
4. extracted contour error;
5. spacing between adjacent passes;
6. final swept material.

Names should not overstate guarantees. An experimental field-contour strategy can be useful while reporting simulation-only or unresolved scallop evidence.

3.2.1.5 Directionality and phase

Paths should carry phase and direction constraints:

```
interface PlannedPath {
    path: Path;
    operation: OperationRef;
    phase: "rough" | "semiFinish" | "finish";
    reversible: boolean;
    preferredDirection?: "climb" | "conventional";
    requiredPredecessors: readonly PathId[];
```

```
}

```

The scheduler should not infer reversibility from geometry alone.

3.2.1.6 Witness design

A raster pocket witness might include:

```
interface RasterPocketWitness {
  centerRegion: RegionRef;
  scanDirection: UnitVec2;
  stepover: Mm;
  strips: readonly StripCoverage[];
  unresolvedCells: readonly CellRef[];
  outerBoundaryError: Mm;
  guaranteedRemovalError: Mm;
}
```

The checker verifies coverage, boundary protection, and bound composition. If unresolved cells remain, the claim becomes inconclusive or the planner refines them.

3.2.1.7 Worked example: pocket coverage

Suppose a 20 mm wide center region is cut with a 3 mm tool and 1.2 mm stepover. A naive pass count of $\lceil 20/1.2 \rceil$ does not prove edge coverage because the first pass position and tool radius matter. A checker reconstructs the union of guaranteed inner swept strips and compares it with the required-removal region.

For a flat end mill whose radius has uncertainty $r \in [r^-, r^+]$, guaranteed removal uses r^- ; protected-boundary checking uses r^+ .

3.2.1.8 Exercises

1. Define a witness for offset-loop pocketing.
2. Explain why target height samples do not prove continuous no-gouge.
3. State the inputs required for a physical scallop-height claim.
4. Give a path property that should prevent reversal.
5. Design a refinement condition for adaptive drop-cutter sampling.
6. Explain why tool-radius uncertainty has opposite directions for removal and gouge checks.

3.2.2 Scheduling, Entry, and Linking

Section goals. The reader should be able to distinguish planning from scheduling, model evolving stock, derive safe links in configuration space, and preserve precedence and process constraints.

3.2.2.1 Planning produces islands; scheduling produces a program

Feature planners often produce several disconnected cut paths. The scheduler decides:

- order;
- orientation;
- tool grouping;
- entries and exits;
- traverses and retracts;

- probe and setup dependencies;
- feeds and time law;
- optional path merging.

These decisions are stateful because each cut changes stock.

3.2.2.2 Evolving stock state

Let S_i be stock before operation i and R_i its removed region:

$$S_{i+1} = S_i \setminus R_i.$$

A link is checked against the stock state that exists when the link executes. Checking every link against initial stock is conservative but may force excessive retracts. Checking every link against final stock is unsound because material may still be present.

Scheduled IR should reference the stock version:

```
interface ScheduledMotion {
  beforeStock: StockStateRef;
  command: Traverse | Cut | Probe;
  afterStock: StockStateRef;
}
```

3.2.2.3 Entry strategies

Common entries include:

- vertical plunge;
- linear ramp;
- helical ramp;
- predrilled entry;
- edge approach;
- rest-material entry.

Each entry has preconditions. A plunge may require a center-cutting tool and a region free of target or fixture material. A helix requires sufficient radial clearance. An edge approach requires an accessible exterior.

Entry selection is a constrained planning problem, not a cosmetic interpolation.

3.2.2.4 Traverse semantics

A traverse means “move through free space,” not “emit G0.” A machine lowering may choose:

```
retract in Z
move in XY
descend in Z
```

because some controllers implement multi-axis rapid as independent axis motion rather than a guaranteed straight segment. The expansion itself must be checked against machine and fixture geometry.

3.2.2.5 Configuration space

For fixed tool orientation, expand obstacles by the reflected tool assembly:

$$O_C = O \oplus (-T_{assembly}).$$

The tool reference point can then be treated as a point that must avoid O_C [R25]. Stock, fixtures, and machine no-go volumes contribute to the configuration obstacle.

A safe path γ satisfies:

$$\forall s \in [0, 1], \quad \gamma(s) \notin O_C.$$

3.2.2.6 Precedence constraints

Dependencies include:

- rough before finish;
- probe before dependent geometry;
- drill pilot before large drill;
- cut internal features before releasing surrounding material;
- maintain support for thin walls;
- tool-change grouping subject to process order.

Represent them as a directed acyclic graph when possible. A scheduler produces a topological order and a witness mapping each edge to positions in the schedule.

3.2.2.7 Worked example: two islands

Two finish paths lie in separate pocket islands. Reversing the second path shortens the free-space link, but the path is climb-only. The scheduler must choose among:

- retain direction and accept a longer link;
- add a safe traverse to the required start;
- select a different strategy that produces a legal orientation;
- reject the optimization.

A shortest endpoint distance is not the objective until direction and clearance constraints are included.

3.2.2.8 Exercises

1. Explain why final-stock collision checking is unsound for early links.
2. Specify preconditions for a helical entry.
3. Derive a configuration obstacle for a circular tool in 2D.
4. Design a precedence witness for a scheduled operation list.
5. Give an example where a shorter link violates process semantics.
6. Compare a conservative retract policy with free-space path planning.

3.2.3 Machine Lowering and Capability Resolution

Section goals. The reader should be able to describe machine capabilities algebraically, lower abstract actions safely, and reject unsupported semantics instead of guessing.

3.2.3.1 Machine profiles as semantic contracts

A machine profile should contain more than travel dimensions. It defines the target language and assumptions for lowering.

```
interface MachineProfile {
  id: string;
  version: string;
  axes: readonly AxisProfile[];
  kinematics: KinematicModel;
  interpolation: InterpolationCapabilities;
  spindle: SpindleCapabilities;
  probes: ProbeCapabilities;
  toolChange: ToolChangeCapabilities;
  controller: ControllerSemanticsRef;
  travelEnvelope: ConfigurationSetRef;
  dynamicLimits: DynamicLimits;
}
```

Capabilities should be structured data, not scattered booleans.

3.2.3.2 Algebraic capabilities

```
type ArcCapability =
  | { kind: "native"; planes: readonly Plane[]; helical: boolean }
  | { kind: "linearize"; maxChordError: Mm }
  | { kind: "unsupported" };
```

A lowering pass pattern-matches exhaustively. If a helical arc is requested and the controller supports planar arcs only, it may linearize within budget or reject. It must not silently emit an invalid arc.

3.2.3.3 Coordinate resolution

Machine lowering resolves work-frame paths to machine coordinates under a selected transform. It must also model whether the controller itself applies a work offset. There are two valid approaches:

1. Emit work coordinates and explicit WCS selection, then validate under the controller's offset semantics.
2. Pretransform all coordinates to machine coordinates and use machine-coordinate motion intentionally.

Mixing the two produces double offsets or unchecked travel.

3.2.3.4 Dynamic and geometric limits

Travel limits constrain configuration:

$$q(t) \in Q_{admissible}.$$

Dynamic limits constrain derivatives:

$$|\dot{q}_i| \leq v_i^{max}, \quad |\ddot{q}_i| \leq a_i^{max}, \quad |\dddot{q}_i| \leq j_i^{max}.$$

A geometric feed value does not by itself guarantee these limits; controller trajectory generation and path curvature matter. Machine IR may preserve a feed policy for later time parameterization or lower it to a verified schedule.

3.2.3.5 Safe expansion of abstract operations

An abstract `Traverse(A,B)` can lower to a sequence using a certified clearance plane. The pass relation is not textual equivalence. It is:

- same start and end;
- all intermediate motion belongs to certified free space;
- process state is preserved;
- added motion remains within machine limits;
- timing and side effects are acceptable.

Similarly, a tool change may expand into spindle stop, retract, move to change position, prompt, and tool-state update.

3.2.3.6 Unsupported and unknown behavior

A target profile should distinguish:

- unsupported by hardware;
- supported but not modeled;
- modeled but not verified;
- verified under constraints.

“Unknown” must not be treated as “probably supported.”

3.2.3.7 Worked example: rapid lowering

Suppose canonical IR contains a diagonal traverse from (0,0,10) to (100,100,10). If the controller guarantees coordinated linear rapid, one G0 may implement it. If rapid axes move independently and fixtures occupy a diagonal corridor, the actual dogleg may intersect them. A safe lowering either uses a known-clear route or emits coordinated feed motion under a verified speed policy.

3.2.3.8 Exercises

1. Design algebraic capabilities for probing.
2. Explain two consistent ways to handle work offsets.
3. State the pass relation for traverse expansion.
4. Give an example where a geometrically valid feed violates axis velocity limits.
5. Distinguish unsupported, unmodeled, and unverifiable.
6. Design a machine-profile hash and compatibility rule.

3.2.4 Controller IR and Modal Semantics

Section goals. The reader should be able to model modal state explicitly, interpret G-code blocks, and validate modal compression as a semantics-preserving optimization.

3.2.4.1 Modal syntax is compressed state

In RS274-like languages, words persist until replaced. A block omitting G1, F, or S may inherit them. Modal groups constrain which codes can be active simultaneously. Correct meaning depends on the initial state and controller dialect [R2].

Canonical IR should remain non-modal. Controller IR can introduce modal state in a controlled stage.

3.2.4.2 Controller state

A simplified state includes:

```
interface ModalState {
  units: "mm" | "inch" | "unknown";
  distanceMode: "absolute" | "incremental" | "unknown";
  motionMode: "rapid" | "linear" | "cwArc" | "ccwArc" | "none" | "unknown";
  plane: "XY" | "XZ" | "YZ" | "unknown";
  feedMode: FeedMode;
  feed?: number;
  spindle?: SpindleState;
  wcs?: WorkCoordinateSystem;
  toolLengthComp?: ToolCompState;
  position: AbstractPosition;
}
```

An interpreter maps each block and state to a successor state plus canonical events.

3.2.4.3 Blocks are evaluated as units

A G-code block is not necessarily a left-to-right sequence of words. Controller specifications define modal-group updates and execution order. A parser should produce a structured block; an interpreter applies dialect rules.

Raw string prefix classification is not semantic parsing. A line can contain several words, comments, checksums, or controller commands. Multiline payloads require complete grammar validation.

3.2.4.4 Initial state

A job should not depend on ambient modal state unless the dependency is an explicit runtime assumption. A preamble usually establishes:

- units;
- absolute/incremental mode;
- plane;
- feed mode;
- work offset;
- cutter and tool-length compensation policy;
- spindle and coolant baseline.

The final validator interprets the complete program from the declared initial state.

3.2.4.5 Modal compression

A simple serializer may emit every modal word in every block. A compression pass removes words whose values are already active. Its correctness statement is:

$$\text{Interpret}(B_{\text{compressed}}, m_0) = \text{Interpret}(B_{\text{explicit}}, m_0).$$

A translation validator can interpret both and compare canonical traces and final modal states.

Unknown raw text destroys this proof unless parsed. The safe response is to reset all affected modal components to unknown and require re-establishment before dependent commands, or to reject compression around the block.

3.2.4.6 Probes and result semantics

A probe block commands a search path; its actual contact point is a runtime result. Controller IR should distinguish commanded endpoint, maximum travel, success condition, and measurement register. Emitting the expected contact as though it were a deterministic absolute endpoint is incorrect.

3.2.4.7 Worked example: stale incremental mode

Consider:

```
G91
G1 X10 F300
...
X20
```

If the later block is intended as absolute X=20 but G90 was never restored, it moves an additional 20 mm. A canonical non-modal program would represent the intended target explicitly. The controller interpreter exposes the mismatch during validation.

3.2.4.8 Exercises

1. Define modal groups for motion and distance mode.
2. Interpret a short program under two different initial states.
3. State a validator for modal compression.
4. Explain how raw text should affect abstract modal state.
5. Model a probe block's command and result separately.
6. Design a complete, explicit preamble for a three-axis metric job.

3.2.5 Serialization, Round Trips, and Exact Bytes

Section goals. The reader should be able to define deterministic serialization, validate parse-back equivalence, account for rounding, and bind certificates to exact deployed bytes.

3.2.5.1 Serialization is a compiler pass

Formatting appears trivial until safety depends on it. The serializer chooses:

- decimal precision;
- suppression of trailing zeros;
- negative-zero handling;
- line endings;
- comments and metadata escaping;
- line numbers and checksums;

- program-end syntax;
- filename and upload encoding.

Each choice can change controller behavior or artifact identity.

3.2.5.2 Deterministic formatting

For a given Controller IR and configuration, serialization should produce exactly one byte sequence. Locale-dependent decimal commas, unstable comment order, timestamps, or platform line endings damage reproducibility.

```
interface SerializationConfig {
  decimals: number;
  trimTrailingZeros: boolean;
  lineEnding: "\n";
  emitComments: boolean;
  commentEscape: CommentEscapePolicy;
}
```

3.2.5.3 Rounding bounds

Rounding each coordinate to d decimal places introduces at most:

$$\rho = \frac{1}{2}10^{-d}$$

per scalar in the selected unit. In three dimensions, an independent-coordinate Euclidean bound is:

$$\rho_3 \leq \sqrt{3}\rho.$$

The actual path error also depends on interpolation and repeated endpoints. Arc center and radius formatting can amplify error or make an arc invalid.

3.2.5.4 Parse-back validation

After serialization, parse the exact bytes with an independent parser, interpret them under the target dialect, and compare with Controller IR:

$$\text{Interpret}(\text{Parse}(\text{bytes}), m_0) \approx_\epsilon \text{Sem}(\text{ControllerIR}).$$

This catches:

- omitted modal resets;
- wrong probe syntax;
- escaped comment failures;
- numeric overflow or scientific notation unsupported by the controller;
- program-end omissions;
- postprocessor bugs.

3.2.5.5 Injection through comments and metadata

User-controlled strings inserted into comments or header records must be escaped according to the dialect. Newlines or delimiters can terminate a comment and introduce executable words. The safest design uses structured metadata outside executable text; when comments are required, the serializer enforces a conservative character set.

3.2.5.6 Exact-byte binding

A certificate produced before serialization does not automatically apply afterward. The final validation artifact should include:

```
interface ByteArtifact {
  bytes: Uint8Array;
  sha256: Hash;
  dialect: DialectId;
  initialModalState: ModalStateRef;
  parseBackClaim: ClaimId;
}
```

Upload and execution must refer to this hash, not merely a filename.

3.2.5.7 Safety epilogue

A validator may require a final spindle stop, coolant stop, retract, or program end. The property must be checked on final interpreted bytes. A comment or validation diagnostic saying “M5 appended” is not evidence if the emitter did not actually append it.

3.2.5.8 Worked example: round-trip checker

1. Serialize explicit Controller IR.
2. Hash the bytes.
3. Parse with an independent grammar.
4. Interpret from the declared modal state.
5. Compare event traces, endpoints, process state, and final modal state.
6. Attach numeric bounds for rounded motion.
7. Verify the required epilogue.
8. Store the byte hash in every dependent claim.

3.2.5.9 Exercises

1. Derive the coordinate rounding bound for four decimal places in millimeters.
2. Design a comment-escaping policy.
3. Explain why parse equality is weaker than semantic equality.
4. Specify a final-byte certificate claim.
5. Give two ways an arc can become invalid after formatting.
6. Explain why the program filename cannot replace the content hash.

3.2.6 Optimization Without Breaking Meaning

Section goals. The reader should be able to state optimization legality, distinguish hard constraints from objectives, and validate local transformations and whole-program changes.

3.2.6.1 Optimization is constrained refinement

A CAM optimizer searches for a lower-cost implementation while preserving required behavior:

$$\min_O J(O)$$

subject to:

$$R(I, O) \wedge \text{Feasible}(O).$$

Safety constraints are not soft penalties. A path that is one second faster but intersects a fixture is not a worse solution; it is outside the feasible set.

3.2.6.2 Common optimizations

- remove zero-length and redundant motions;
- merge collinear segments;
- fit arcs or splines;
- compress modal words;
- reorder independent operations;
- choose path orientation;
- reduce retract height under certified clearance;
- group tool changes;
- schedule feed and spindle changes;
- select among strategy alternatives.

Each has a different pass relation.

3.2.6.3 Peephole validation

Local transformations can carry local proofs. Merging two collinear line segments is legal when:

- the intermediate point lies on the same line;
- feed and process state are compatible;
- no event at the intermediate point is semantically observable;
- controller interpolation of the merged move remains within limits;
- provenance is preserved.

A dwell, exact-stop marker, probe, or stock-dependent state change prevents the merge.

3.2.6.4 Reordering and commutativity

Two operations commute only when their effects are independent and their preconditions remain valid in either order. A sufficient condition resembles:

$$W_1 \cap (R_2 \cup W_2) = \emptyset,$$

with additional constraints for stock support, probing, tool state, and fixtures. Effect summaries provide a conservative test.

Geometric set subtraction is commutative, but the physical process is not necessarily so.

3.2.6.5 Objective evidence

A heuristic may report a candidate cost. A certificate should distinguish:

- feasibility proved;
- objective computed exactly or bounded;
- local improvement relative to a baseline;
- global optimality proved;
- optimality gap bounded by a lower bound.

Without a valid lower bound, call the result optimized or heuristic, not optimal.

3.2.6.6 Phase ordering

Passes interact. Arc fitting before machine lowering may create unsupported arcs. Rounding before collision checking may hide final-byte error. Reordering before entry generation may choose endpoints that later become infeasible.

A typical safe order is:

1. semantic planning;
2. scheduling and linking;
3. machine capability lowering;
4. bounded geometric optimization;
5. controller lowering;
6. modal optimization;
7. serialization;
8. final parse-back and geometric validation.

Final validation is necessary even when each pass has a local proof because bugs and omitted assumptions can occur at composition boundaries.

3.2.6.7 Worked example: arc fitting

A polyline fitting pass proposes an arc over points p_i . Its witness includes maximum deviation, endpoint tangents, sweep, and source range. The checker verifies:

- all covered points lie within the declared metric bound;
- the arc is supported by the target or can be lowered;
- no semantic event lies inside the replaced range;
- the outer swept-volume envelope remains safe;
- the optimization does not exceed remaining error budget.

3.2.6.8 Exercises

1. State legality conditions for removing a zero-length move.
2. Give an example where two material-removal operations do not commute.
3. Explain why safety cannot be a weighted objective term.
4. Design a witness for collinear segment merging.
5. Define an optimality gap.

6. Propose a pass order and justify two ordering constraints.

Checkpoint - lowering. Distinguish planning, scheduling, linking, machine lowering, controller lowering, and serialization; explain why parse-back validates bytes rather than formatting style.

3.3 Assertions, invariants, weakest preconditions, and abstract interpretation

Pass-specific checkers answer local questions. Whole programs also need statements that remain true across sequences and branches. Contracts, invariants, and abstract interpretation provide that layer.

Definition - assertion. An assertion is a proposition expected to hold at one point in a program or execution.

CAM example. Immediately before a cut: the machine is homed, the intended tool is active, the spindle is within an allowed range, and the work transform is known.

Definition - precondition. A precondition is a proposition that must hold before an operation may execute or before its contract applies.

CAM example. A cut requires a known work transform, a loaded compatible tool, and a valid spindle state.

Definition - postcondition. A postcondition is a proposition guaranteed after an operation terminates from a state satisfying its precondition.

CAM example. After a successful cut, the machine pose is at the path endpoint and the stock model has been updated by the declared removal semantics.

Definition - invariant. An invariant holds initially and is preserved by every relevant transition. It therefore holds at every reachable state.

Small example. A bank transfer preserves the sum of two account balances when no fee is charged.

CAM example. An ideal subtractive stock model preserves $S_{i+1} \subseteq S_i$: stock may be removed but not added.

Definition - weakest precondition. The weakest precondition $wp(c, Q)$ is the least restrictive condition sufficient to ensure postcondition Q after command c .

CAM example. Working backward from “the next rapid is clear” can derive requirements on current stock, fixture models, tool assembly, work transform, and clearance height.

Definition - soundness. An analysis is sound for a claim when every fact it reports as established is true for all concrete states or executions represented by its abstract result, under the stated assumptions. Soundness permits false alarms; it does not permit missed counterexamples.

CAM example. A sound travel analyzer may reject a path because transform uncertainty is too large, but it must not certify a path that can leave the machine envelope.

Definition - abstract interpretation. Abstract interpretation executes a program over a domain whose elements conservatively represent sets of concrete states. A sound abstract result may be imprecise, but it must not omit a possible concrete behavior relevant to the claim.

CAM example. Instead of one exact spindle RPM, the analysis may track off, on in $[11, 900, 12, 100]$, or unknown; every command updates this abstract state.

3.3.1 Do not confuse these statement kinds

Kind	Where it applies	How it is established	Pocket example
Representation invariant	One data representation	Constructors/checkers	A path endpoint equals the final segment endpoint.
Precondition	Before one command	Caller or weakest-precondition proof	Tool T1 is loaded before cut.
Postcondition	After one command	Command semantics	Stock is updated by the declared cutting sweep.
Inductive invariant	Every reachable state	Initial proof plus preservation	Stock never increases in the subtractive model.
Assumption	Outside the current proof	Measurement, attestation, operator check, or later discharge	Actual clamp position matches the fixture artifact.
Safety property	Every trace prefix	Temporal/model checking or runtime monitor	No execution starts an unauthorized hash.
Certificate claim	One named artifact/property	Named evidence and checker	Maximum target penetration is at most 0.020 mm.

A statement can play more than one role, but the role determines who must establish it and when.

3.3.2 Contracts, Hoare Logic, and Weakest Preconditions

Section goals. The reader should be able to distinguish assertions, invariants, assumptions, and guarantees; derive weakest preconditions; and use contracts to structure compiler and controller checks.

3.3.2.1 Vocabulary matters

Safety discussions become confused when different logical objects share the word “check.” The following distinctions should appear in code and certificate schemas.

An **assertion** is a proposition about one program point or artifact.

A **precondition** must hold before an operation.

A **postcondition** is guaranteed after successful execution, subject to assumptions.

An **invariant** holds initially and is preserved by all relevant transitions.

An **assumption** is an external fact not established by the current checker.

A **guarantee** is a proposition established when the assumptions hold.

A **witness** is data chosen by a producer to help establish a proposition.

A **proof or evidence object** is checked by a specified procedure.

An **attestation** establishes origin or integrity; it does not establish semantic truth.

3.3.2.2 Hoare triples

A Hoare triple has the form [R4]:

$$\{P\} c \{Q\}.$$

It means that if command c starts in a state satisfying P and terminates normally, the resulting state satisfies Q . Total-correctness variants also establish termination.

A traverse contract might be:

$$\{\text{Pose} = a \wedge \text{Homed} \wedge \text{Free}(\gamma, T, O, S)\}$$

$$\text{Traverse}(\gamma)$$

$$\{\text{Pose} = b \wedge S' = S \wedge \text{ProcessState}' = \text{ProcessState}\}.$$

The stock-equality postcondition distinguishes traverse from cut.

3.3.2.3 Sequential composition

If:

$$\{P\} c_1 \{R\}$$

and:

$$\{R\} c_2 \{Q\},$$

then:

$$\{P\} c_1; c_2 \{Q\}.$$

The intermediate assertion R is the interface between commands. Compiler IR design should expose enough state to state R .

3.3.2.4 Weakest preconditions

The weakest precondition $wp(c, Q)$ is the least restrictive condition that guarantees Q after c .

For sequential commands:

$$wp(c_1; c_2, Q) = wp(c_1, wp(c_2, Q)).$$

For an assignment $x := e$:

$$wp(x := e, Q) = Q[e/x].$$

For a conditional:

$$wp(\text{if } b \text{ then } c_1 \text{ else } c_2, Q) = (b \Rightarrow wp(c_1, Q)) \wedge (\neg b \Rightarrow wp(c_2, Q)).$$

A preflight engine can derive requirements backward from the program rather than maintaining an unrelated checklist.

3.3.2.5 CNC weakest-precondition example

Suppose the desired final condition is:

$$Q = \text{SpindleOff} \wedge \text{Pose} = p_{\text{safe}}.$$

The program is:

```
cut path P
retract to p_safe
stop spindle
```

Working backward:

1. `stop spindle` requires a valid controller session and guarantees `SpindleOff`.
2. `retract` requires a collision-free path from `end(P)` to p_{safe} .
3. `cut` requires homing, known WCS, selected tool, valid spindle state, safe cutting sweep, and feasible feed.

The derived precondition becomes the basis of compile-time claims and runtime preflight.

3.3.2.6 Invariant induction

To prove invariant I over a transition system:

1. **Initialization:** $I(\sigma_0)$.
2. **Preservation:** $I(\sigma) \wedge T(\sigma, \sigma') \Rightarrow I(\sigma')$.

Examples:

- stock monotonically decreases under cut and remains unchanged under traverse;
- every motion command has a known frame;
- cutting implies a selected tool and active spindle;
- an executing job has one content hash;
- an alarm state cannot transition directly to running without explicit recovery.

3.3.2.7 Separation of resources

Separation logic reasons about disjoint mutable resources [R38]. Its central intuition is useful even without a full separation-logic implementation. An operation that removes stock in region A should not affect a disjoint fixture region B . Effect summaries can state footprints, enabling local reasoning and safe parallel analysis.

3.3.2.8 Design by contract in APIs

Contracts should be executable where possible:

```
function cut<S extends ReadyToCut>(
  state: S,
  motion: CuttingMotion,
): Result<CutResult, Diagnostic[]> {
  requireFinitePath(motion.path);
  requirePositiveFeed(motion.feed);
  requireToolMatch(state.tool, motion.tool);
  // geometric and runtime obligations remain explicit claims
}
```

Do not pretend that a runtime assertion about metadata proves physical collision freedom. Contracts should state the boundary of what they establish.

3.3.2.9 Exercises

1. Write a Hoare triple for spindle start.
2. Derive wp for select tool; start spindle; cut with final condition stock conforms to roughing intent.
3. State and prove a stock-monotonicity invariant for a simplified semantics.
4. Distinguish an assumption from a precondition in a live tool check.
5. Give an example of a local resource footprint for two independent operations.
6. Explain partial versus total correctness for an abort command.

3.3.3 Abstract Interpretation for Machine Programs

Section goals. The reader should be able to define concrete and abstract domains, implement sound transfer functions, compute invariants over control flow, and use unknown states conservatively.

3.3.3.1 Simulating one trace is not analyzing all traces

A simulator executes one concrete state through one control path. A validator often needs to cover many possible states: unknown initial modal settings, uncertain positions, branches after probing, controller faults, or values from macros. Abstract interpretation computes over-approximations of these possibilities [R7].

Let C be a concrete domain and A an abstract domain. A concretization function:

$$\gamma : A \rightarrow \mathcal{P}(C)$$

maps an abstract value to the concrete states it represents.

A sound abstract transfer function $\widehat{F}$ satisfies:

$$F(\gamma(a)) \subseteq \gamma(\widehat{F}(a)).$$

3.3.3.2 Abstract machine state

```
interface AbstractMachineState {
  position: Box3 | "unknown";
  homing: "homed" | "unhomed" | "maybe";
}
```

```

tool: ToolRef | Set<ToolRef> | "unknown";
spindle: "off" | RpmInterval | "unknown";
wcs: TransformInterval | "unknown";
motionMode: Set<MotionMode>;
feed: Interval<number> | "unknown";
playing: "yes" | "no" | "maybe";
alarm: "yes" | "no" | "maybe";
}

```

The domain should be just expressive enough for the claims it checks.

3.3.3.3 Interval transfer

For a linear absolute move to $X = x$, the abstract X interval becomes $[x, x]$ after accounting for rounding and transform uncertainty. In incremental mode, it becomes:

$$X' = X + [x - \rho, x + \rho].$$

If distance mode is $\{\text{absolute}, \text{incremental}\}$, the successor joins both possibilities. Precision drops, but soundness remains.

3.3.3.4 Modal-state analysis

An abstract G-code interpreter can prove:

- units definitely metric;
- distance mode definitely absolute;
- spindle definitely on before each cut;
- position remains inside travel bounds;
- no motion occurs with unknown WCS;
- raw commands do not leave required modal components unknown;
- final spindle state is definitely off.

Each block records an invariant before and after:

```

interface BlockInvariant {
    blockIndex: number;
    before: AbstractMachineState;
    after: AbstractMachineState;
}

```

A small proof checker replays abstract transfers and verifies safety predicates.

3.3.3.5 Joins and loss of precision

If one branch selects tool T1 and another selects T2, the merge state contains $\{T1, T2\}$. A later operation requiring exactly T1 cannot be proved safe without a guard.

Widening may be necessary for loops. For example, repeated incremental motion can make position unbounded unless the loop count is bounded. A production-certifiable dialect may reject unbounded macros rather than analyze them imprecisely.

3.3.3.6 Reduced products

Different abstract domains can cooperate. A position box and a linear relation domain may jointly prove tighter bounds. A modal-state domain and typestate domain can reduce each other: if an alarm is definite, playing is false; if spindle is off, RPM interval is zero.

Such combinations increase checker complexity. Begin with simple domains tied to explicit claims.

3.3.3.7 Unknown raw effects

A raw block that is not parsed should conservatively set affected state to unknown. Self-declared effects can be recorded as assumptions, but they do not justify a proved claim.

```
function transferRaw(state: AbstractState, raw: RawBlock): AbstractState {
  return {
    ...state,
    position: "unknown",
    motionMode: allMotionModes,
    feed: "unknown",
    spindle: "unknown",
    wcs: "unknown",
  };
}
```

A later explicit preamble can re-establish some components.

3.3.3.8 Worked example: final spindle state

A program branches around an optional finishing pass. Both branches eventually execute M5. At the merge, spindle is definitely off. If one branch omits M5, the merge state is {off,on} and the final safety epilogue claim fails.

3.3.3.9 Exercises

1. Define a concretization function for an interval.
2. Write abstract transfer rules for G90, G91, G20, and G21.
3. Explain why joining branches loses precision.
4. Design a sound state update for an unknown raw command.
5. Show how a loop of incremental X moves can require widening.
6. Propose a reduced product relevant to probing and frames.

Checkpoint - program reasoning. Classify a statement as precondition, postcondition, invariant, assumption, temporal property, or certificate claim; then explain what soundness permits and forbids.

3.4 Robust geometry, swept volumes, and quantitative error

Discrete program reasoning is not enough for continuous geometry and floating-point computation. The checker must reason about what happens between samples and near degenerate geometric cases.

Definition - robust predicate. A robust predicate returns the mathematically correct discrete decision, such as orientation or intersection sign, despite finite-precision inputs and near degeneracy. Filtered, adaptive, exact, or interval methods are common techniques.

Definition - interval. An interval $[a, b]$ denotes every real value between its endpoints and can conservatively represent numerical uncertainty.

Small example. A measured diameter of 3.175 ± 0.005 mm is represented by $[3.170, 3.180]$ mm.

CAM example. Each component of a probed work transform may be enclosed by an interval.

Definition - enclosure. An enclosure is a set guaranteed to contain the exact value, path, transform, or solid of interest.

CAM example. A conservative swept-volume mesh or cell cover encloses every physical tool pose allowed by the path and uncertainty model.

Definition - inner approximation. An inner approximation X^- is guaranteed to satisfy $X^- \subseteq X_{true}$.

Small example. An inscribed polygon is an inner approximation of a disk.

CAM example. Required-removal claims use a volume certainly removed by the cutter, not merely a volume it might have touched.

Definition - outer approximation. An outer approximation X^+ is guaranteed to satisfy $X_{true} \subseteq X^+$.

Small example. A circumscribed polygon is an outer approximation of a disk.

CAM example. Fixture-clearance claims use an outer enclosure of both the tool-and-holder sweep and the uncertain fixture geometry.

Definition - swept volume. The swept volume of a solid along a trajectory is the union of every pose of that solid during the motion.

CAM example. Gouge analysis uses the cutting portion of the tool; fixture clearance uses the complete tool-and-holder assembly. Reusing one sweep for both claims can be unsound.

Definition - error budget. An error budget is a structured composition of quantified approximation and uncertainty bounds under named metrics and frames. It is not merely a list of tolerances added together.

CAM example. Path chord error, controller coordinate rounding, work-transform uncertainty, tool-radius uncertainty, and machine following error may contribute to different claims and may require different composition rules.

3.4.1 Two claims, two approximation directions

Fixture clearance. Let W_{true} be the true tool-and-holder sweep and O_{true} the true obstacle set. Construct outer enclosures W^+ and O^+ such that

$$W_{true} \subseteq W^+, \quad O_{true} \subseteq O^+.$$

If $W^+ \cap O^+ = \emptyset$, the true sweep and obstacle are also disjoint. The check may be conservative, but a reported success is meaningful.

Guaranteed removal. Let R_{true} be the material actually removed. Construct an inner approximation R^- such that

$$R^- \subseteq R_{true}.$$

If the required-removal region lies inside R^- , that material is guaranteed to be removed. An outer approximation would be unsuitable because it may contain material never actually reached.

The same sampled depth field cannot be silently reused for both claims unless it carries both sound inner and outer semantics.

3.4.2 Robust Numerics and Computational Geometry

Section goals. The reader should be able to distinguish topological predicates from metric constructions, use exact or adaptive predicates, apply interval enclosures, and identify failure modes of floating-point geometry.

3.4.2.1 Why geometric bugs are discontinuous

A small floating-point error in a distance often causes a small distance error. A small error in a sign predicate can change topology completely: two contours connect instead of remaining separate, a polygon changes orientation, or an intersection is missed.

Computational geometry therefore distinguishes:

- **predicates**, which decide discrete facts such as orientation or intersection;
- **constructions**, which compute coordinates.

Predicates deserve stronger numerical methods.

3.4.2.2 Orientation predicates

For points a, b, c in 2D, orientation is the sign of:

$$\text{orient2d}(a, b, c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

Near collinearity, floating-point cancellation can return the wrong sign. Adaptive exact predicates evaluate cheaply in ordinary cases and increase precision only near degeneracy [R22].

Use robust predicates for:

- polygon orientation;
- segment intersection;
- point-in-polygon decisions;
- Delaunay or Voronoi topology;
- contour stitching;
- mesh adjacency and ray intersections.

3.4.2.3 Exact topology, approximate geometry

A practical architecture can keep topology exact while allowing bounded approximate coordinates. For grid-generated contours, use grid-edge identifiers rather than re-quantized endpoint coordinates. For polygon kernels, use integer-scaled coordinates within a proven range or exact rational predicates.

The endpoint-key failure discussed later is a violation of this principle: a lossy packed coordinate is used as topological identity.

3.4.2.4 Interval arithmetic

An interval $[a, b]$ represents every real number between its endpoints. Arithmetic operations use outward rounding so the exact result is enclosed. IEEE 1788.1 specifies interval arithmetic over binary floating-point endpoints [R23]; Moore, Kearfott, and Cloud provide a broad treatment of interval analysis [R24].

If:

$$x \in [a, b], \quad y \in [c, d],$$

then:

$$x + y \in [a + c, b + d]$$

with outward-rounded endpoints. Multiplication takes the min and max of all endpoint products.

Intervals can bound:

- curve coordinates over a parameter range;
- transform uncertainty;
- distance to a surface;
- polynomial extrema;
- swept-volume occupancy;
- numerical residuals.

3.4.2.5 The dependency problem

Interval arithmetic can overestimate when the same variable appears repeatedly. For $x \in [0, 1]$:

$$x - x = [-1, 1]$$

under naive interval evaluation, although the true result is zero. Subdivision, affine arithmetic, Taylor models, or symbolic simplification can improve tightness.

An inconclusive wide interval is not a false result. The checker can subdivide or reject.

3.4.2.6 Conservative meshing

A triangle mesh is often treated as exact geometry, but it approximates a design surface. A certificate should record:

- source artifact hash;
- tessellation tolerance;
- orientation and watertightness status;
- whether the mesh encloses or is enclosed by the intended solid;
- repairs performed;
- unresolved non-manifold or self-intersection issues.

Without an enclosure relation, mesh-based collision checks prove facts about the mesh, not necessarily the intended CAD solid.

3.4.2.7 Tolerance and topology

Using one epsilon everywhere is dangerous. Distinct tolerances are needed for:

- numeric comparison;
- geometric construction;
- topological snapping;
- machining allowance;
- certification bound.

A topological snap modifies geometry. Its displacement must be recorded and charged to an error budget.

3.4.2.8 Worked example: collision of two nearly touching paths

Suppose a line passes $2\text{ }\mu\text{m}$ from a fixture in nominal double precision, while tool-radius uncertainty is $5\text{ }\mu\text{m}$ and frame uncertainty is $10\text{ }\mu\text{m}$. An exact floating-point distance does not make the move safe. The relevant outer enclosure intersects the obstacle. Numerical exactness and physical robustness are different dimensions.

3.4.2.9 Exercises

1. Compute `orient2d` for three nearly collinear points and explain cancellation risk.
2. Distinguish a predicate from a construction.
3. Give a use of interval subdivision in curve bounding.
4. Explain the interval dependency problem.
5. Design metadata for a mesh enclosure claim.
6. List four different tolerances in a CAM compiler and their roles.
7. Explain why exact arithmetic does not eliminate model uncertainty.

3.4.3 Swept Volumes, Stock Models, and Collision Proofs

Section goals. The reader should be able to define swept volume, choose conservative stock representations, distinguish cutting from holder collision, and formulate no-gouge and guaranteed-removal claims.

3.4.3.1 Swept volume

For a tool solid T and pose trajectory $x(t)$, the swept volume is:

$$\text{Sweep}(T, x) = \bigcup_{t \in [0, T]} x(t)T.$$

Swept-volume computation is central to collision detection, stock simulation, and manufacturing verification [R26]. Exact computation is difficult for general geometry and trajectories, so practical systems use conservative enclosures or discrete representations.

3.4.3.2 Stock evolution

For cutting command i with removed volume R_i :

$$S_{i+1} = S_i \setminus R_i.$$

A simulator may approximate R_i using the swept cutting geometry. The semantic distinction among commands remains important:

- cut updates stock;
- traverse must not update stock;
- probe ideally stops at first contact and may touch without intended removal;
- tool change and dwell preserve stock.

If a missing traverse is silently converted to a cut, the stock semantics change, not merely the visualization.

3.4.3.3 Tool versus holder

Let T_c be cutting geometry and T_a the full assembly. Then:

No target gouge:

$$\text{Sweep}(T_c, x)^+ \cap P_{\text{protected}} = \emptyset.$$

No fixture collision:

$$\text{Sweep}(T_a, x)^+ \cap O^+ = \emptyset.$$

Guaranteed removal:

$$R_{\text{required}} \subseteq \text{Sweep}(T_c, x)^-.$$

The superscripts indicate outer and inner enclosures.

3.4.3.4 Doxel and voxel models

A doxel model stores material intervals along parallel rays. A height field stores one surface value per XY cell and cannot represent undercuts or multiple layers. Triple-doxel models use three orthogonal directions and improve surface representation [R27, R28]. Voxels represent occupancy in three dimensions.

Tradeoffs include:

- memory;
- update speed;
- ability to represent overhangs and cavities;
- surface error;
- ease of conservative enclosure;
- suitability for cutter contact.

For a three-axis no-overhang part, a height field can be efficient, but its resolution must be connected to a continuous-domain bound.

3.4.3.5 A grid size is not a guarantee

Sampling motion at spacing h and stock on a grid of cell size g does not by itself prove “verified to $\max(h, g)$.” A narrow fixture or brief penetration can occur between samples.

A sound cell method needs one of:

- conservative occupancy of every cell touched by the continuous sweep;
- interval bounds over each cell and motion interval;
- a Lipschitz bound plus adaptive subdivision;
- exact primitive sweep tests;
- a certified distance-field enclosure.

Otherwise, label the result simulation-only.

3.4.3.6 Configuration-space collision

For fixed orientation, configuration-space obstacles reduce moving-solid collision to point-path collision [R25]:

$$O_C = O \oplus (-T_a).$$

A clearance checker can build an outer enclosure O_C^+ and prove that the tool-reference path is separated by a positive bound.

For rotating tools or multi-axis orientation, configuration space has more dimensions and the expansion varies with orientation.

3.4.3.7 Continuous path checking

Lines and arcs often admit analytic bounds. Splines and sampled paths can be recursively subdivided. A generic branch-and-bound checker operates on parameter intervals:

1. Bound path positions over interval I .
2. Expand by tool assembly and uncertainty.
3. If disjoint from obstacle enclosure, accept I .
4. If definitely intersecting, return a counterexample interval.
5. Otherwise subdivide until a limit.
6. If the limit is reached unresolved, return inconclusive.

This produces honest coverage of the continuous parameter domain.

3.4.3.8 Worked example: rapid through stock

A rapid from A to B at low Z is safe only if the entire outer swept assembly is disjoint from current stock and fixtures. Checking endpoints misses an obstacle between them. Checking sampled points can miss a thin wall. A segment-versus-expanded-obstacle test or conservative recursive enclosure addresses the whole move.

3.4.3.9 Exercises

1. Write set formulas for no-gouge, fixture collision, and guaranteed removal.
2. Compare height fields, dexels, triple dexels, and voxels.
3. Explain why one grid resolution cannot be attached blindly to all claims.
4. Design a recursive continuous collision checker.
5. Give a case where cutting geometry is clear but holder geometry collides.
6. Explain how stock state affects link verification.

3.4.4 Error Budgets and Quantitative Refinement

Section goals. The reader should be able to construct typed error budgets, compose bounds through passes, allocate tolerances, and distinguish numerical, model, calibration, and process uncertainty.

3.4.4.1 A budget is a proof structure

An error budget is not merely a UI sum. It records how a final bound follows from component bounds under explicit propagation rules.

Sources include:

- design-surface tessellation;
- tool geometry approximation;
- tool dimension tolerance and runout;
- path planning discretization;
- curve fitting and linearization;
- coordinate transform uncertainty;
- numeric rounding;
- controller interpolation;
- servo following error;
- probing and calibration;
- thermal and material effects.

Some are compiler-controlled; others are runtime assumptions.

3.4.4.2 Typed metrics

```
type ErrorBound =
| { metric: "hausdorff-position"; frame: FrameId; value: Mm }
| { metric: "normal-surface"; surface: Hash; value: Mm }
| { metric: "max-gouge-depth"; target: Hash; value: Mm }
| { metric: "transform-translation"; transform: Hash; value: Mm }
| { metric: "transform-rotation"; transform: Hash; value: Radians }
| { metric: "axis-following"; axis: AxisId; value: Mm };
```

Only compatible bounds can be combined directly. Rotational error becomes positional error only after multiplying by a radius or applying a more precise transform bound.

3.4.4.3 Budget allocation

Suppose a feature tolerance is 0.05 mm. The compiler may allocate:

CAD tessellation	0.010 mm
planning and refinement	0.012 mm
postprocessor rounding	0.003 mm
frame and probing	0.010 mm
machine following	0.010 mm
reserve	0.005 mm

This allocation is meaningful only if each number is a valid bound in a compatible metric and the composition theorem supports addition. A planner that consumes more than its allocation must refine or fail.

3.4.4.4 Correlation and worst-case composition

Worst-case scalar addition is conservative when errors can align. Root-sum-square composition assumes statistical independence and a probabilistic interpretation; it is not a deterministic maximum bound. Certificates must identify whether a claim is worst-case, probabilistic, empirical, or nominal.

3.4.4.5 Pass amplification

A small angular error $\delta\theta$ at radius r creates positional error approximately $r\delta\theta$. An offset operation can amplify boundary error near sharp corners or topology changes. A coordinate transform with scale should not exist in rigid machining frames, but a calibration map may have local condition numbers.

Each pass needs a sensitivity or Lipschitz bound when it propagates prior error.

3.4.4.6 Residual-driven refinement

An adaptive algorithm should report a residual related to the claimed metric. For a height field, comparing neighboring samples is not automatically a surface-error residual. A useful refinement loop is:

```

propose approximation
compute certified local bound
if bound <= allocated budget: accept cell
else if subdivision limit not reached: subdivide
else: inconclusive

```

The residual, subdivision depth, and unresolved regions become evidence.

3.4.4.7 Runtime budgets

Some budget components are checked at runtime. A tool measurement may tighten diameter uncertainty; a probe sequence may establish a frame bound; a machine calibration record may establish following-error limits. If live bounds exceed the bundle allocation, execution is refused or the job is recompiled.

3.4.4.8 Worked example: final surface error

Suppose:

- target mesh outer deviation: 0.008 mm;
- drop-cutter field bound: 0.012 mm;
- arc fitting: 0.004 mm;
- output rounding: 0.001 mm;
- frame translation: 0.010 mm;
- angular frame uncertainty: 0.0002 rad at 50 mm radius, contributing 0.010 mm.

A conservative additive position bound is 0.045 mm if the metrics and directions align. If the requested bound is 0.04 mm, the program is not certified merely because each individual pass met its local default.

3.4.4.9 Exercises

1. Classify six error sources as numerical, model, calibration, or process uncertainty.
2. Convert an angular bound to a positional bound at a given radius.
3. Explain when root-sum-square composition is inappropriate.
4. Design a typed budget for a planar pocket floor.
5. State a residual-driven refinement algorithm.
6. Explain how a runtime measurement can discharge a compile-time assumption.

Checkpoint - geometry. Choose inner or outer approximations for clearance, no-gouge, guaranteed removal, and residual stock; reject any tolerance statement that lacks a metric, frame, direction, or bound.

3.5 Certificate schemas, trusted checkers, and proof-carrying CAM

The previous analyses produce many facts. A certificate turns those facts into a checkable dependency structure.

Definition - claim. A claim is a precise proposition about a precise artifact. “Safe” is not a sufficiently precise claim.

Example 1. “Every machine-space point of trajectory artifact h1 lies in travel envelope artifact h2 under transform interval h3.”

Example 2. “The maximum penetration of the cutting sweep into protected target h4 is at most 0.020 mm.”

Definition - assumption. An assumption is an external fact required by a claim but not established by its evidence, such as measured tool diameter or firmware conformance.

Definition - evidence. Evidence is machine-checkable data used to establish a claim: interval bounds, abstract invariants, path correspondences, solver assignments, separating witnesses, or parse-back traces.

Definition - certificate. A certificate binds claims, subject artifacts, assumptions, evidence, dependencies, checker identities, methods, results, and quantitative bounds.

Definition - trusted computing base (TCB). The TCB is the code, semantics, numerical primitives, identity mechanisms, and assumptions that must be correct for the assurance argument to hold. A principal architectural goal is to keep it small and explicit.

Definition - proof-carrying CAM. Proof-carrying CAM is the producer/checker architecture in which a complex planner or compiler emits an artifact plus evidence, and a smaller independent checker accepts or rejects the named claims.

3.5.1 Certificate Schemas and Proof Graphs

Section goals. The reader should be able to write precise claims, bind them to artifacts and assumptions, organize dependencies as a DAG, and avoid ambiguous status labels.

3.5.1.1 A certificate is a graph of propositions

A certificate should not be one boolean or one table of reassuring labels. It is a graph connecting artifacts, assumptions, claims, evidence, and checkers.

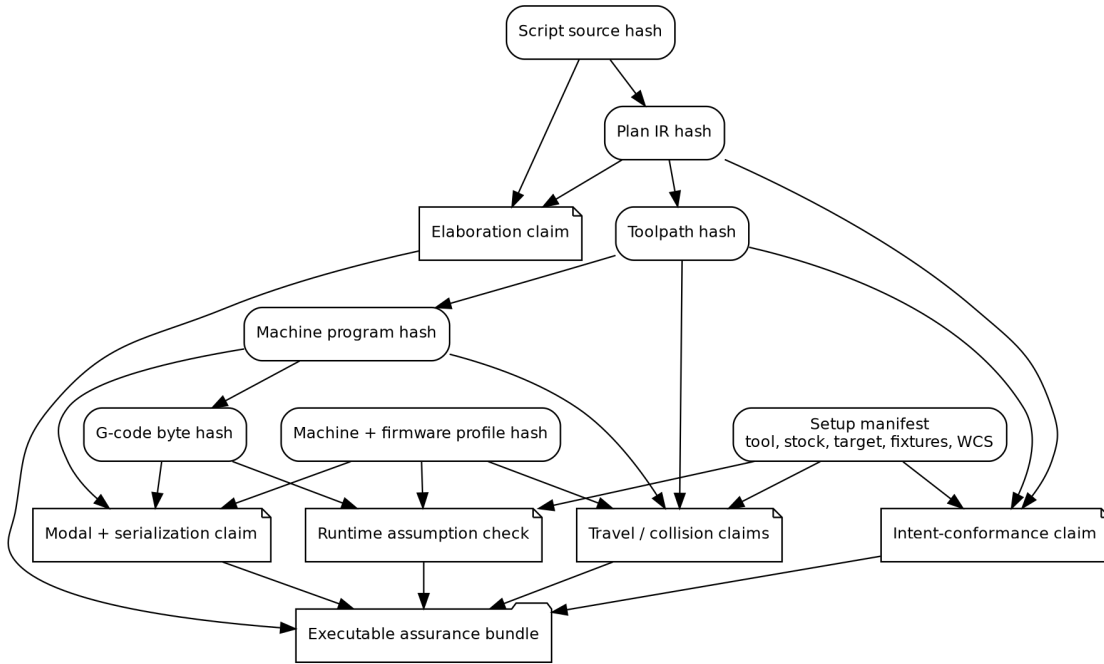


Figure 3.2: A certificate dependency graph binds source, IRs, final bytes, setup, and runtime state.

A change to any dependency invalidates downstream claims.

3.5.1.2 Structured claims

```

interface Claim {
  id: ClaimId;
  subject: ArtifactRef;
  proposition: StructuredPredicate;
  result:
    | "proved-exact"
    | "proved-bounded"
    | "translation-validated"
    | "exhaustive-finite-check"
    | "simulation-only"
    | "assumed"
    | "unknown"
    | "refuted";
  method: MethodRef;
  assumptions: readonly AssumptionRef[];
  evidence: readonly EvidenceRef[];
  dependencies: readonly ClaimId[];
  bound?: ErrorBound;
  checker: CheckerIdentity;
}

```

The proposition is not a string such as "gouge". It is structured:

```
{
  kind: "max-penetration",
  sweptArtifact: "sha256:...",
  protectedTarget: "sha256:...",
  maximum: mm(0.02),
  metric: "signed-normal-depth",
}
```

3.5.1.3 Results and methods are orthogonal

“Checked to resolution” combines method and conclusion. Separate:

- proposition;
- method;
- result;
- quantitative bound;
- coverage;
- assumptions.

A sampled simulation method may produce a counterexample and thus refute a claim. Failure to find a counterexample does not upgrade it to proof.

3.5.1.4 Assumption records

```
interface Assumption {
  id: AssumptionId;
  proposition: StructuredPredicate;
  source: "operator" | "calibration" | "machine-attestation" | "library";
  evidence?: ArtifactRef;
  validFrom?: Timestamp;
  validUntil?: Timestamp;
  runtimeCheck?: RuntimeCheckSpec;
}
```

Examples:

- tool T1 diameter lies in [3.170, 3.180] mm;
- fixture model hash matches setup QR code;
- machine firmware belongs to a compatible semantic profile;
- G54 transform lies inside an interval;
- stock dimensions meet a measured bound.

3.5.1.5 Evidence types

```
type Evidence =
  | AbstractInterpretationTrace
  | IntervalSubdivisionTree
  | CollisionSeparationWitness
  | CoverageWitness
  | ParseBackTraceComparison
  | SolverCertificate
  | ModelCheckingTrace
  | Counterexample
  | CalibrationRecord
```

```
| SignatureAttestation;
```

Evidence schemas should be stable, versioned, and independently checkable.

3.5.1.6 Claim granularity

Separate claims by meaning:

- target no-gouge;
- fixture collision;
- holder collision;
- rapid-through-stock;
- required removal;
- machine travel;
- feed and dynamics;
- modal equivalence;
- final spindle stop;
- exact upload hash;
- controller start acknowledgement.

One generic evidence object should not be able to mint unrelated rows. Strong typing can enforce this:

```
function certifyGouge(
  claim: GougeClaim,
  evidence: GougeEvidence,
): CertifiedClaim;
```

3.5.1.7 Completeness policy

A bundle policy states which claims are required for a use class:

Preview-only:

```
  schema, finite values, parse success
```

Attended air cut:

```
  travel, controller parse-back, exact bytes, runtime identity
```

Attended material cut:

```
  + target, fixture, holder, stock, tool and WCS assumptions
```

Unattended production:

```
  + protocol liveness, runtime monitor, calibrated dynamic bounds,
    recovery and interlock claims
```

A function such as `isFullyVerified` must derive from explicit policy, not from absence of severe diagnostics.

3.5.1.8 Worked example: false evidence reuse

Suppose a dextr simulation checks low rapids and spoilboard penetration but receives no target surface. It can support:

```
rapid-through-current-stock sampled simulation
spoilboard penetration sampled simulation
```

It cannot support:

maximum gouge into protected target ≤ 0.02 mm

because the proposition's target artifact is absent. A schema that requires `protectedTargetHash` and target-specific evidence prevents the incorrect promotion.

3.5.1.9 Exercises

1. Write a structured claim for machine travel.
2. Design separate evidence types for rapid collision and target gouge.
3. Explain why a digital signature is not a semantic proof.
4. Define a completeness policy for an attended air cut.
5. Draw a dependency graph for an arc-linearized final job.
6. Describe certificate invalidation after a tool change.

3.5.2 Trusted Checkers and Proof-Carrying CAM

Section goals. The reader should be able to minimize a trusted computing base, explain proof-carrying code, design producer-consumer separation, and evaluate checker independence.

3.5.2.1 The producer is complicated

CAM planners use spatial indices, heuristics, solvers, floating-point kernels, caches, parallelism, and UI state. Treating this entire implementation as trusted makes assurance fragile.

Proof-carrying code separates an untrusted producer from a consumer-side checker: code arrives with evidence that it satisfies a policy, and the consumer validates the evidence before execution [R20]. Foundational proof-carrying code reduces reliance on specialized verification-condition generators by grounding evidence in a smaller logic [R21].

The analogous CAM architecture is:

```

complex planner / optimizer / postprocessor
      |
      v
  artifact + evidence
      |
      v
small independent checker
      |
  accept or reject

```

3.5.2.2 Trusted computing base

A realistic trusted base includes:

- definitions of claim predicates;
- canonical artifact hashing and serialization;
- independent parsers and semantic interpreters;
- robust numeric primitives used by checkers;
- certificate dependency validation;
- runtime identity and upload-hash handshake;
- a small execution monitor;
- explicitly accepted physical assumptions.

The UI, heuristic planners, optimizers, and preview renderer should not need to be trusted.

3.5.2.3 Independence

A checker that calls the same buggy helper as the producer is not strongly independent. Independence can be improved by:

- separate implementations;
- different algorithms or representations;
- reduced feature set;
- smaller codebase;
- strict schemas;
- deterministic behavior;
- extensive property and differential testing;
- optional mechanized proofs for critical kernels.

Independence is a spectrum, not a binary property.

3.5.2.4 Proof objects versus recomputation

A checker can:

1. recompute the property independently;
2. verify a compact witness;
3. check a formal proof term;
4. combine these methods.

For contour coverage, a cell decomposition witness may be cheaper than re-running planning. For modal equivalence, replaying both programs is simple. For a linear or mixed-integer optimizer, a solver certificate or dual bound may support optimality.

3.5.2.5 Checker resource bounds

Evidence can be adversarial or malformed. Checkers need:

- schema validation;
- size and recursion limits;
- deterministic resource bounds;
- integer overflow protection;
- denial-of-service resistance;
- explicit version compatibility;
- no execution of producer-supplied code.

3.5.2.6 Compositional certificates

If pass P_1 establishes $R_1(A, B)$ and P_2 establishes $R_2(B, C)$, a composition theorem establishes $R(A, C)$:

$$R_1(A, B) \wedge R_2(B, C) \implies R(A, C).$$

The theorem may accumulate quantitative bounds. Certificate composition should be implemented by a checker, not inferred from matching stage names.

3.5.2.7 Mechanization strategy

Formalize the small semantic core first:

- canonical command semantics;
- controller modal semantics;
- pass-relation composition;
- certificate validity;
- state-machine authorization;
- numeric lemmas for common primitives.

Leave complex search algorithms unverified but validated. This yields more assurance per unit effort than attempting to prove an entire CAM application at once.

3.5.2.8 Worked example: validating a solver schedule

An untrusted optimizer returns operation order π , path orientations, total cost J , and a lower bound L . The checker verifies:

1. π is a permutation of required operations;
2. all precedence edges are respected;
3. every selected orientation is legal;
4. each transition cost is recomputed;
5. total cost equals the sum;
6. the lower-bound certificate is valid;
7. the reported gap $(J - L)/L$ is correct.

The optimizer's internal search does not enter the trusted base.

3.5.2.9 Exercises

1. List the trusted components in a minimal CAM checker.
2. Give an example where a checker is insufficiently independent.
3. Compare recomputation with witness checking.
4. Design resource limits for a certificate checker.
5. State a composition theorem for two bounded geometric passes.
6. Choose three components suitable for formal mechanization and justify them.

Checkpoint - certificates. Write one precise claim with subject artifact, assumptions, evidence, checker, method, result, and numeric bound; explain which elements belong to the trusted computing base.

3.6 Chapter synthesis: a certificate graph for the pocket

A single green badge hides too much. The running pocket needs a graph of separate claims.

```
source script hash
|
v
elaborated plan hash ---- unit/frame/static evidence
|
v
```

```

intent IR hash ----- target/allowance specification
  |
  v
planned toolpath hash --- coverage witness, boundary witness
  |
  v
scheduled program hash -- precedence and link witnesses
  |
  v
machine IR hash ----- travel/capability/dynamics assumptions
  |
  v
controller IR hash ----- modal and dialect validation
  |
  v
serialized byte hash ---- parse-back and rounding evidence
  |
  v
uploaded byte hash ----- controller acknowledgement

```

For the pocket, a minimal property set includes:

1. **Static legality:** all units, frames, tools, references, and operation forms are valid.
2. **Required removal:** an inner approximation of removed stock covers the roughing-required region.
3. **Target protection:** an outer cutting sweep does not penetrate protected target beyond allowance and tolerance.
4. **Fixture clearance:** an outer tool-and-holder sweep is disjoint from modeled fixtures.
5. **Travel:** every machine-space configuration lies inside the machine envelope.
6. **Process sequencing:** tool, spindle, probe, roughing, and finishing dependencies hold.
7. **Controller equivalence:** interpreted serialized bytes refine the checked controller IR.
8. **Content binding:** the uploaded and authorized job hash equals the certified byte hash.

3.6.1 Why evidence cannot be reused casually

Suppose a simulator samples the cutter and reports a depth map. That evidence may support a claim about sampled stock removal. It does **not** support a gouge claim unless the checker also compares the sweep or resulting stock with a target/protected geometry and covers the continuous domain between samples. Evidence is reusable only when the new proposition logically follows from what the evidence and checker establish.

3.6.2 A practical pass interface

```

interface CertifyingPass<I, O, W> {
  readonly id: string;
  readonly version: string;

  transform(
    input: Artifact<I>,
    config: Artifact<PassConfig>,
  ): Result<{
    output: Artifact<O>;
    witness: Artifact<W>;
  }, Diagnostic[]>;

```



```

check(
  input: Artifact<I>,
  output: Artifact<O>,
  witness: Artifact<W>,
  config: Artifact<PassConfig>,
): CheckResult;
}

```

The interface does not magically make the checker sound. It forces the architecture to name the relation and the evidence boundary.

3.6.3 Common misconceptions

1. “**The type `ValidatedProgram` means every property is proved.**” A brand records that a gate ran; only property-specific claims and evidence state what was established.
2. “**The generator and checker agree, so the output is correct.**” Shared algorithms and shared bugs can make agreement meaningless. Independence and a simpler checker matter.
3. “**Sampling at 0.1 mm proves error below 0.1 mm.**” Only if a theorem or conservative enclosure bounds unsampled behavior.
4. “**All errors can be summed.**” Bounds must use compatible metrics, frames, and amplification rules.
5. “**Optimization can run first and validation later.**” An optimizer should produce candidates inside a separately checked feasible set; the validator must reject any candidate that violates hard constraints.

3.6.4 Chapter exercises

1. Write a pass contract for fitting arcs to a polyline. State the semantic relation, metric, endpoint conditions, and checker inputs.
2. Derive a weakest precondition for `selectTool`; `startSpindle`; `cut(path)` under a postcondition that the target remains protected.
3. Design an abstract domain for modal motion mode, units, distance mode, active WCS, spindle state, and position bounds.
4. Explain why outer swept volume is appropriate for no-collision while inner swept volume is appropriate for guaranteed removal.
5. Create a certificate schema for final-byte parse-back equivalence. Identify the TCB.
6. Give a counterexample in which a valid path reordering changes process semantics because the intermediate stock state changes.

Chapter 4

Execution, Optimization, and Engineering the Whole System

Chapter goal. Close the gap between a checked file and a controlled physical execution. By the end of the chapter, you should be able to model the controller as a temporal protocol, discharge runtime assumptions, separate feasibility from optimization, choose operations-research models for sequencing and feed scheduling, organize an implementation around a small trusted core, and use the Dropcut/Z1 defects as architectural lessons rather than isolated bugs.

A compile-time certificate is conditional. It says, in effect: “If this machine profile, firmware semantics, tool assembly, stock, fixture model, work transform, and job identity match reality, then these claims hold.” The runtime system must establish enough of those conditions before execution. It must also handle events that do not fit a pure batch compiler: disconnects, ambiguous timeouts, feed holds, alarms, partial uploads, controller reboots, changed work offsets, and human intervention.

Only after feasibility and identity are secure should the system optimize. Operations research supplies the mathematical tools for choosing among many valid implementations, but an objective function is not a safety policy. Cycle time, rapid distance, wear, and tool changes can be optimized only inside a hard, independently checked feasible set.

4.1 Temporal controller semantics and runtime assurance

The controller is part of the target semantics, not a byte sink.

Definition - protocol state machine. A protocol state machine defines states, messages, transitions, guards, and effects for interactions between host and controller.

CAM example. Disconnected, Connected, Uploading, Stored, Authorized, Running, Held, Aborting, Stopped, and Alarmed are semantically distinct states even when a user interface compresses them into one status line.

Definition - safety property. A safety property says that a bad event never occurs; a finite bad trace prefix demonstrates violation.

CAM example. A job never starts unless its exact hash is authorized for the current epoch.

Definition - liveness property. A liveness property says that a desired event eventually occurs under stated fairness and environment assumptions.

CAM example. After an accepted abort request, the machine eventually reaches Stopped or an explicit fault state.

Definition - temporal logic. Temporal logic expresses properties over traces using operators such as “always” $\square$ and “eventually” $\Diamond$.

CAM example. $\square(\text{Running} \Rightarrow \text{AuthorizedHash} = \text{ExecutingHash})$.

Definition - runtime assurance. Runtime assurance uses a small trusted monitor to enforce a safety envelope around a more complex planner, controller client, or physical system.

CAM example. A monitor can reject state-changing commands outside a closed grammar, require hash-bound authorization, enforce command envelopes, and trigger a safe stop when assumptions change.

Definition - authorization epoch. An authorization epoch is a changing identifier that binds a preflight decision to the machine state on which it was based. If the work offset, tool, firmware profile, or relevant state changes, the epoch changes and stale authorization cannot be reused.

4.1.1 Temporal Semantics and Controller Protocols

Section goals. The reader should be able to model the host-controller interaction as a state machine, state safety and liveness properties, and reason about ambiguous failures and authorization races.

4.1.1.1 The protocol is part of the compiler target

A final G-code file is useless unless the correct bytes are transferred, stored, selected, and executed. The controller interface is therefore part of the compilation and deployment semantics.

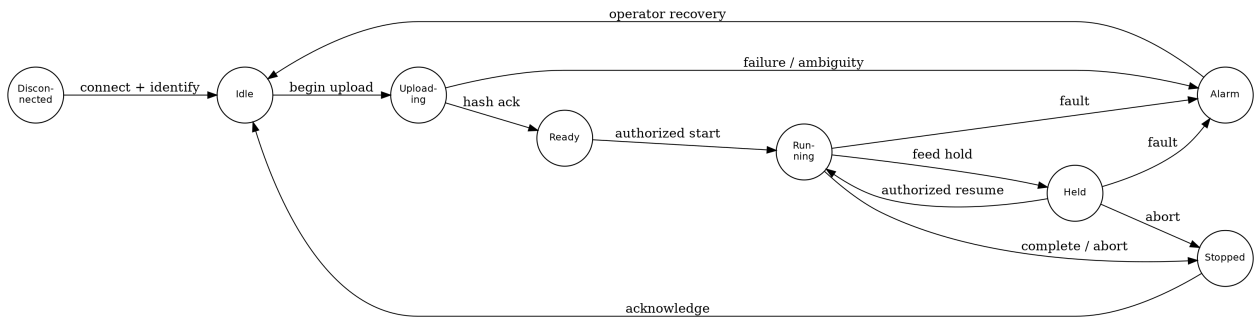


Figure 4.1: A simplified controller and job lifecycle state machine.

4.1.1.2 States and transitions

A controller model should include states such as:

- disconnected;
- connected but unidentified;
- idle;
- uploading;
- ready with a specific stored hash;

- running a specific execution instance;
- held;
- stopped;
- alarmed;
- ambiguous or quarantined after timeout.

Transitions have guards and effects. Resume is not a read-only command; it enables motion. FeedHold is a stop-class action and should remain available even when ordinary command admission is blocked.

4.1.1.3 Safety properties

Examples:

$$\Box(\text{Running}(h) \Rightarrow \text{Authorized}(h)).$$

$$\Box(\text{Start}(h) \Rightarrow \text{StoredHash} = h).$$

$$\Box(\text{Motion} \Rightarrow \neg \text{Alarm}).$$

$$\Box(\text{GenericReadPath} \Rightarrow \neg \text{MotionEffect}).$$

The last property requires complete parsing, not prefix classification.

4.1.1.4 Liveness properties

$$\Box(\text{UploadStarted} \Rightarrow \Diamond(\text{Ready} \vee \text{Failed} \vee \text{Quarantined})).$$

$$\Box(\text{AbortRequested} \Rightarrow \Diamond(\text{Stopped} \vee \text{Alarm})).$$

Liveness depends on assumptions about network delivery, controller scheduling, and machine responsiveness. Those assumptions must be stated.

4.1.1.5 Atomic admission and execution

A preflight that checks state, releases a lock, and later sends motion has a time-of-check/time-of-use gap. Another command can change the state between admission and execution.

A safer design serializes:

```

acquire session authority
refresh live state
check preconditions
bind authorization to command/job hash and state epoch
send command
observe acknowledgement or quarantine
release authority
```

The state epoch invalidates admission if intervening events occur.

4.1.1.6 Ambiguous timeout

After sending a command, a timeout does not imply that the machine did nothing. The command may have executed while the reply was lost. The correct successor state is a set of possibilities. The session should enter an ambiguous/quarantined state until re-synchronized through a trusted status query and protocol boundary.

Blind retry is especially dangerous for motion or state-changing commands.

4.1.1.7 Message streams and loss

Safety-relevant events should not share a lossy queue designed for UI telemetry. A dropped alarm, completion marker, or protocol sentinel can make the host state inconsistent. Separate durable control events from best-effort display updates.

Partial writes must also be handled. A transport `Write` may write fewer bytes than requested without a fatal error; command framing must loop until complete or fail before execution can be assumed.

4.1.1.8 Model checking

A finite abstraction of the protocol can be exhaustively checked for invariants and deadlocks. TLA-style specifications are well suited to concurrency and temporal properties [R8, R34]. Hybrid automata extend state machines with continuous dynamics when stop distance and watchdog timing matter [R35].

4.1.1.9 Worked example: compound command classification

A classifier that examines only the first whitespace token may label:

```
status\nG0 X100
```

as read-only because `status` is first. The semantic policy concerns every parsed command in the payload. A fail-closed grammar rejects embedded newlines or parses all blocks and proves that every effect belongs to the allowed class.

4.1.1.10 Exercises

1. Define states and transitions for upload, start, hold, resume, and abort.
2. State three safety and two liveness properties.
3. Explain the time-of-check/time-of-use race in preflight.
4. Model an ambiguous timeout as a set of successor states.
5. Explain why stop commands should bypass ordinary motion admission.
6. Design a fail-closed read-only command grammar.
7. Identify which messages require durable delivery.

4.1.2 Runtime Assurance and Physical Assumptions

Section goals. The reader should be able to bind compile-time evidence to live machine state, design a runtime assurance monitor, and distinguish logical proof from physical validation.

4.1.2.1 Conditional assurance

A compile-time certificate proves a conditional theorem:

if assumptions A hold, artifact B satisfies claims C

Runtime preflight must establish as much of A as possible. The remaining assumptions require operator acceptance, calibration records, or physical interlocks.

4.1.2.2 Hash-bound execution

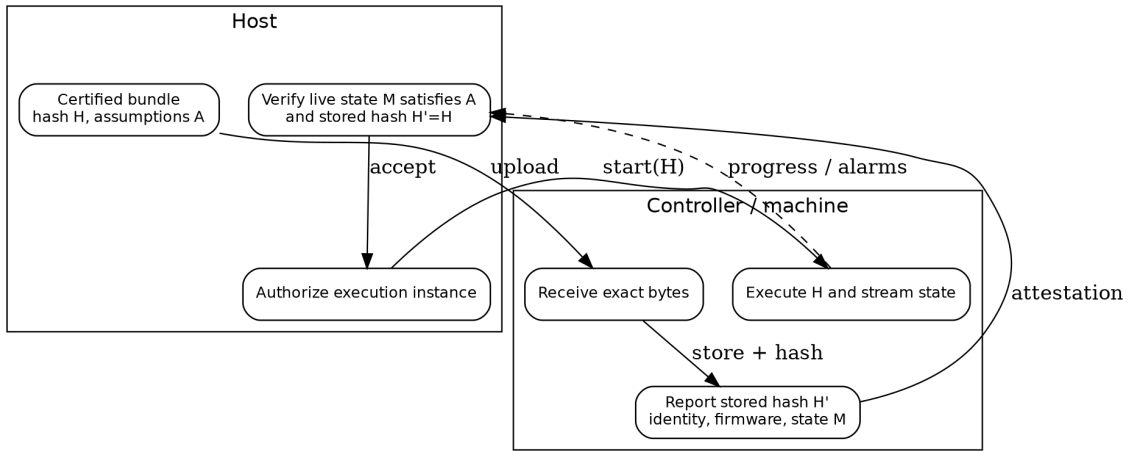


Figure 4.2: A runtime handshake binds authorization to exact bytes and live state.

A robust protocol is:

1. Host proposes certified bundle hash H and assumptions A .
2. Controller stores exact bytes and reports stored hash H' .
3. Controller reports identity, firmware, homing, WCS, tool state, alarm state, and job state M .
4. Host verifies $H' = H$ and $M \models A$.
5. Host authorizes one execution instance of H .
6. Controller acknowledges start of that exact instance.
7. Host monitors state until terminal completion, abort, or fault.

4.1.2.3 Runtime checks

Possible checks include:

- machine identity and firmware semantics profile;
- homed axes;
- active work coordinate system and measured transform;
- tool ID, length, and optional diameter measurement;
- fixture/setup identifier;
- stock presence and dimensions;
- door, probe, and accessory interlocks;
- no active alarm;
- exact stored job hash;
- controller idle or permitted paused state;

- calibration validity period.

4.1.2.4 Runtime assurance monitor

A runtime assurance architecture places a small trusted monitor around a more capable controller or host planner. Simplex-style systems switch to a trusted safety action when an advanced component approaches an unsafe region; modern runtime-assurance frameworks apply this to cyber-physical systems [R36].

For a Z1-class machine, a realistic monitor can enforce:

- authorized state transitions;
- command classes;
- job-hash binding;
- axis and feed envelopes;
- spindle/tool consistency;
- watchdog policy;
- communication-loss behavior;
- explicit confirmation of stop.

4.1.2.5 Stopping distance and latency

A stop request is not an instantaneous stop. If velocity is v and guaranteed deceleration is a , a simple lower bound on stopping distance is:

$$d_{stop} \geq \frac{v^2}{2a},$$

plus command, communication, and controller latency. A safety envelope must reserve this distance. Feed hold may preserve the job and decelerate differently from emergency stop or reset.

4.1.2.6 Physical model validation

Even perfect software cannot prove that the CAD fixture model matches reality. Useful physical validation practices include:

- setup identifiers and photographs;
- measured stock and tool records;
- probing routines with uncertainty;
- air cuts and reduced-feed first runs;
- witness marks or sacrificial stock;
- calibration artifacts;
- independent inspection of critical dimensions;
- bounded validity periods for calibration.

These produce assumptions and attestations, not mathematical facts from the compiler.

4.1.2.7 Degradation policies

A runtime system should define what happens when an assumption cannot be established:

- refuse execution;
- downgrade to preview or air-cut mode;

- require explicit operator override with recorded rationale;
- reduce feed and restrict motion envelope;
- re-probe or re-measure;
- recompile with wider uncertainty;
- quarantine the session after ambiguity.

Silent continuation is not a policy.

4.1.2.8 Worked example: changed G54

A job was certified under work transform T_0 with uncertainty set Ξ . Before execution, the live controller reports T_1 . The runtime checker computes whether T_1 lies inside the certified set. If not, the travel and collision claims are stale. The correct response is to revalidate or recompile, not to rely on the same file hash.

4.1.2.9 Design rules

Safety note

A certificate is valid only for the exact artifact and assumption set named by its claims. Live machine identity, tools, frames, setup, and controller state are inputs to execution, not incidental operator details.

4.1.2.10 Exercises

1. Design a runtime assumption manifest for a pocket job.
2. Explain why upload success is weaker than exact-byte execution.
3. Derive stopping distance for a given velocity and deceleration, then add latency distance.
4. List five monitor-enforceable properties and five physical assumptions it cannot prove.
5. Design a degradation policy for an unknown tool measurement.
6. Explain how a changed WCS invalidates geometric certificates.

Checkpoint - runtime. State one protocol safety property, one liveness property, and the facts an authorization epoch must invalidate when they change.

4.2 Operations research inside a checked feasible set

Optimization begins only after the feasible set has been defined and can be checked.

Definition - feasible set. The feasible set contains exactly the candidates satisfying all hard constraints. A candidate outside it is rejected, regardless of objective value.

Definition - objective function. An objective assigns a cost to a feasible candidate, such as estimated cycle time, rapid distance, tool changes, energy, or predicted wear.

Small example. Among routes visiting all required points, minimize total distance.

CAM example. Among schedules that respect rough-before-finish, tool, stock, fixture, and probing constraints, minimize cycle time and tool changes lexicographically.

Definition - precedence graph. A precedence graph is a directed acyclic graph whose edges require one operation to occur before another.

CAM example. Probe datum before using it; rough before finish; cut internal features before an outer contour releases the workpiece.

Definition - configuration space. Configuration space is the space whose points represent complete machine configurations. Collision of a moving solid becomes point avoidance in an expanded forbidden region.

CAM example. Expand clamps and stock by the reflected tool-and-holder shape, then plan a tool-center link through the remaining free region.

Definition - path parameterization. A path parameterization is a monotone function $s(t)$ that maps time to progress along a fixed geometric path.

CAM example. The same circular path can be traversed slowly at constant speed or with an acceleration-limited schedule; the geometry is unchanged but the trajectory differs.

Definition - optimal control. Optimal control chooses a time law and possibly other control variables to minimize an objective while satisfying dynamic and process constraints.

CAM example. Minimize cycle time subject to axis velocity, acceleration, jerk, tracking-error, spindle, and chip-load limits.

Definition - robust optimization. Robust optimization seeks candidates that remain feasible under a stated uncertainty set rather than only at nominal parameter values.

CAM example. Prefer a link with larger clearance margin when fixture and work-transform uncertainty make the nominally shortest link brittle.

4.2.1 State the optimization result honestly

Status	What has been established
Feasible	All checked hard constraints hold.
Locally optimal	No improving candidate exists in a defined local neighborhood.
Globally optimal	No feasible candidate has lower objective value. This requires a proof, exhaustive method, or solver certificate.
Bounded-suboptimal	A valid lower bound L and candidate cost J establish a stated gap, such as $(J - L)/L$.
Heuristic candidate	The producer found a candidate; feasibility and objective must still be checked independently.

For hobby and production CAM alike, “verified feasible with estimated cycle time” is often the most honest and useful result.

4.2.2 Feasibility Before Optimality

Section goals. The reader should be able to formulate CAM planning as constrained optimization, separate hard constraints from objectives, and state what evidence is needed for feasibility and optimality claims.

4.2.2.1 The feasible set comes first

Operations research gives a useful decomposition:

$$\min_{x \in \mathcal{F}} J(x),$$

where $\mathcal{F}$ is the feasible set and J is an objective.

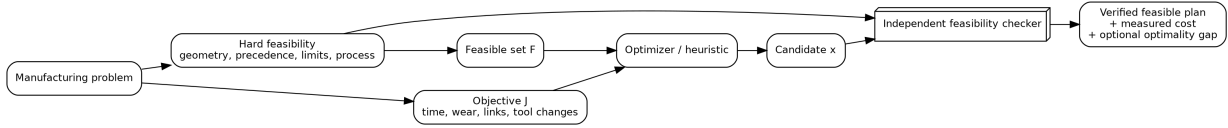


Figure 4.3: Feasibility is independently checked before objective claims are trusted.

For CAM, hard constraints may include:

- target and fixture separation;
- required material removal;
- machine travel and kinematics;
- tool compatibility;
- process precedence;
- entry feasibility;
- maximum velocity, acceleration, jerk, and contour error;
- stock-dependent support and clearance;
- probing and setup dependencies;
- controller capability.

The objective may combine cycle time, retract distance, tool changes, wear, energy, surface quality, or risk margin.

4.2.2.2 Why safety is not a penalty

A common heuristic objective is:

$$J = T + \lambda C_{collision}.$$

No finite λ makes collision categorically forbidden. A sufficiently large time benefit can outweigh the penalty. Safety belongs in constraints:

$$C_{collision}(x) = 0.$$

Soft penalties are appropriate for preferences, not invariant violations.

4.2.2.3 Decision variables

A full CAM problem may contain:

- discrete strategy choices;
- tool assignment;

- operation ordering;
- path orientation;
- entry selection;
- link route;
- feed schedule;
- spindle schedule;
- setup assignment;
- tolerances allocated to passes.

This creates mixed discrete-continuous optimization. Solving it monolithically is often impractical. A staged architecture solves subproblems while preserving feasibility certificates.

4.2.2.4 Constraint generation

Not every constraint must be enumerated upfront. A cutting-plane or lazy-constraint workflow can:

1. solve a relaxed problem;
2. check the candidate geometrically;
3. extract a violated constraint or conflict;
4. add it to the optimization model;
5. repeat.

The checker remains authoritative. The optimizer uses counterexamples to improve search.

4.2.2.5 Feasible, optimal, and near-optimal

These claims are distinct:

- **Feasible:** all hard constraints have been checked.
- **Locally optimal:** no move in a specified neighborhood improves the objective.
- **Globally optimal:** no feasible solution has lower objective.
- **Gap bounded:** $L \leq J^* \leq J$, where L is a valid lower bound.

The relative gap is:

$$\frac{J - L}{\max(|L|, \eta)},$$

with a small η to avoid division by zero.

4.2.2.6 Lexicographic objectives

Safety remains a constraint, but quality objectives may have strict priority:

1. meet dimensional and finish requirements;
2. minimize tool changes;
3. minimize cycle time;
4. minimize retract distance.

Lexicographic optimization avoids arbitrary weights between incomparable units.

4.2.2.7 Worked example: roughing schedule

Suppose three roughing operations can use T1, while a finish operation requires T2 and must follow all roughing. The optimizer chooses among roughing orders and path orientations to minimize links, then performs one tool change. A candidate schedule includes a precedence witness and independently checked links. The objective value is accepted only after feasibility is established.

4.2.2.8 Exercises

1. Formulate a CAM problem with three hard constraints and two objective terms.
2. Explain why a collision penalty is insufficient.
3. Give two discrete and two continuous decision variables.
4. Design a lazy-constraint loop using a collision checker.
5. Distinguish feasible, locally optimal, and globally optimal.
6. Propose a lexicographic objective for a hobby CNC job.

4.2.3 Sequencing and Precedence-Constrained Routing

Section goals. The reader should be able to model operation ordering as a routing problem with precedence, encode orientation choices, and validate a schedule independently.

4.2.3.1 From nearest neighbor to routing

Choosing the nearest next path is fast but ignores precedence, directionality, tool changes, and stock effects. A more complete abstraction treats operations or path endpoints as nodes in a routing problem.

The precedence-constrained traveling salesman problem has been applied directly to CNC toolpath optimization [R31]. Variants include generalized TSP, asymmetric TSP, and clustered routing.

4.2.3.2 Node and state expansion

A path with two legal orientations can be represented by two states:

P.forward: enter at A, exit at B
P.reverse: enter at B, exit at A

Selecting one excludes the other. Transition cost depends on exit and entry positions, stock state, tool state, and link feasibility.

For tool choices, create states (operation, tool, orientation, entry). This can grow rapidly, motivating decomposition and pruning.

4.2.3.3 Precedence graph

Let binary variable $y_{ij} = 1$ when operation i occurs before j . For each precedence edge (i, j) :

$$pos_i + 1 \leq pos_j.$$

A simpler schedule checker receives a permutation π and verifies:

$$\forall (i, j) \in E, \quad \pi^{-1}(i) < \pi^{-1}(j).$$

4.2.3.4 Asymmetric transition costs

Link costs are often asymmetric:

- a path endpoint may have different safe exits;
- climb-only orientation changes entry and exit;
- Z heights differ;
- stock removal makes later links easier;
- spindle or coolant transitions have direction-dependent costs.

Use a directed cost graph.

4.2.3.5 Mixed-integer model sketch

Let x_{ij} indicate transition from state i to j :

$$\min \sum_{i,j} c_{ij} x_{ij}$$

subject to one incoming and outgoing transition for each selected operation state, selection constraints, precedence constraints, and subtour elimination. Tool changes can add fixed transition costs.

For large jobs, exact MILP may be too slow. Heuristics, local search, dynamic programming over small precedence width, or branch-and-bound can provide good candidates. Verification remains the same.

4.2.3.6 Stock-dependent edges

A fixed edge cost c_{ij} assumes the link from i to j is independent of earlier operations. This is often false. One conservative approach computes links against initial stock. A more precise approach augments state with relevant stock features or recomputes edges during search.

The abstraction should state which stock model the cost and feasibility use.

4.2.3.7 Schedule witness

```
interface ScheduleWitness {
  permutation: readonly OperationId[];
  chosenVariants: readonly VariantId[];
  precedencePositions: ReadonlyMap<OperationId, number>;
  transitions: readonly {
    from: VariantId;
    to: VariantId;
    link: PathRef;
    cost: Seconds;
    stockState: StockStateRef;
  }[];
  objective: Seconds;
  lowerBound?: Seconds;
}
```

The checker recomputes every field.

4.2.3.8 Worked example: four contours

Four contour paths have two orientations each. Two inner contours must precede the outer contour so the part remains supported. A nearest-neighbor tour chooses the outer contour second and violates support. A precedence-aware solver excludes that tour, then selects orientations minimizing legal transition cost.

4.2.3.9 Exercises

1. Create a directed state graph for two reversible and one fixed-direction path.
2. Write a permutation-based precedence checker.
3. Explain why transition costs are asymmetric.
4. Formulate one-in/one-out routing constraints.
5. Give a case where stock-dependent links break a static cost matrix.
6. Design a schedule witness and checker.

4.2.4 Free-Space Linking and Configuration Space

Section goals. The reader should be able to formulate linking as path planning in configuration space, compare retract, graph-search, and field methods, and attach a continuous clearance certificate.

4.2.4.1 The link planner's problem

Given start configuration q_s , goal q_g , obstacles O_C , and admissible configurations Q , find:

$$\gamma : [0, 1] \rightarrow Q \setminus O_C$$

minimizing a cost such as length or time.

For a three-axis machine with fixed orientation, the configuration is usually (x, y, z) . Fixtures, stock, and machine exclusions are expanded by the tool assembly.

4.2.4.2 Baseline: certified retract policy

A simple policy is:

1. rise to a certified clearance Z ;
2. move in XY ;
3. descend.

Advantages:

- easy to reason about;
- predictable;
- often robust to stock complexity.

Disadvantages:

- unnecessary vertical motion;
- clearance plane may not exist around tall fixtures;
- machine rapids may not follow assumed paths;
- one global height can be overly conservative.

A baseline policy is valuable as a fallback and reference implementation.

4.2.4.3 Visibility graphs and A*

For polygonal or polyhedral configuration obstacles, graph methods connect visible vertices and search for a shortest route. A raster or voxel occupancy grid supports A* or Dijkstra search. The discretization must be conservative: blocked cells should outer-enclose obstacles, and diagonal corner cutting must be controlled.

A path found on a grid needs smoothing and continuous revalidation.

4.2.4.4 Distance fields and Eikonal equations

A weighted shortest-path field $T(x)$ can satisfy an Eikonal equation:

$$|\nabla T(x)|F(x) = 1,$$

where F is a speed or inverse cost field. Fast marching computes monotonically advancing arrival times efficiently [R29]. Backtracing the gradient yields a candidate path. Fast marching has also been used for geodesic computation on surfaces [R30].

The numerical field does not itself prove collision freedom. The extracted path must be checked against a conservative obstacle enclosure, and discretization error must be bounded.

4.2.4.5 Clearance-aware objectives

Shortest distance can graze obstacles. A robust cost may penalize low clearance:

$$J(\gamma) = \int_0^1 (1 + \lambda\phi(d(\gamma(s), O_C))) \|\gamma'(s)\| ds,$$

where ϕ grows as clearance decreases. Hard minimum clearance remains a constraint.

4.2.4.6 Dynamic feasibility

A geometric link with a sharp corner may be dynamically infeasible at rapid speed. The linker can:

- produce piecewise paths then rely on controller blending with a certified contour-error bound;
- round corners within free-space margins;
- generate a dynamically feasible spline;
- lower the speed near corners.

Geometry and time parameterization should remain separate but coordinated.

4.2.4.7 Link certificate

A link claim should include:

- path artifact hash;
- stock-state hash;
- fixture and tool-assembly hashes;
- frame and uncertainty assumptions;

- minimum certified separation or collision-free enclosure;
- machine travel claim;
- dynamics claim or explicit handoff to feed scheduling.

4.2.4.8 Worked example: low retract

A planner proposes a low Z link through a region cleared by the immediately preceding roughing pass. The link is shorter than a full retract. Its certificate depends on the post-roughing stock artifact. Reordering the link earlier invalidates the certificate even though the path geometry is unchanged.

4.2.4.9 Exercises

1. Formulate a three-axis link problem in configuration space.
2. Compare retract, visibility graph, A*, and fast marching approaches.
3. Explain why a grid path needs continuous revalidation.
4. Design a clearance-aware cost with a hard margin.
5. Give a dynamically infeasible but collision-free path.
6. List the dependencies of a stock-specific link certificate.

4.2.5 Feed Scheduling and Optimal Control

Section goals. The reader should be able to separate geometric path from time law, derive path-coordinate constraints, and validate a feed schedule under machine and process limits.

4.2.5.1 Path parameterization

Given geometric path $q(s)$ for $s \in [0, 1]$, choose monotone $s(t)$. Then:

$$\dot{q} = q'(s)\dot{s},$$

$$\ddot{q} = q''(s)\dot{s}^2 + q'(s)\ddot{s}.$$

Axis velocity and acceleration limits become inequalities in $\dot{s}$ and $\ddot{s}$. Jerk adds another derivative and coupling terms.

4.2.5.2 Time-optimal path parameterization

The objective is:

$$\min T = \int_0^1 \frac{1}{\dot{s}(s)} ds$$

subject to dynamic constraints. CNC minimum-time trajectory research includes velocity, acceleration, jerk, and tracking-error limits [R32]. TOPP-RA formulates reachability-based time-optimal path parameterization for general constraints [R33].

4.2.5.3 Curvature and contour speed

For planar curvature κ , normal acceleration is:

$$a_n = v^2 \kappa.$$

Thus:

$$v \leq \sqrt{\frac{a_n^{max}}{|\kappa|}}.$$

A sharp corner has unbounded ideal curvature and requires stopping, blending, or geometric smoothing. Controller lookahead determines the actual trajectory.

4.2.5.4 Process constraints

Machine dynamics are not the only limits. Cutting may constrain:

- chip load;
- spindle power and torque;
- tool deflection;
- engagement angle;
- material removal rate;
- surface finish;
- minimum feed to avoid rubbing;
- maximum plunge feed.

A feed schedule should reference process models and their uncertainty.

4.2.5.5 Controller realization

A host schedule expressed as continuously varying feed may be approximated by discrete feed changes or controller overrides. The postprocessor needs a realization theorem: the controller's trajectory remains within the certified envelope.

If the controller performs undocumented lookahead, host-side optimality and dynamic guarantees are conditional on a calibrated model.

4.2.5.6 Feed schedule witness

```
interface FeedScheduleWitness {
  pathHash: Hash;
  knots: readonly {
    s: number;
    speed: MmPerMin;
    accelerationBound: number;
  }[];
  axisVelocityMargins: readonly Interval<number>[];
  axisAccelerationMargins: readonly Interval<number>[];
  processMargins: readonly ProcessMargin[];
  estimatedTime: Seconds;
}
```

The checker evaluates constraints over each interval, not only at knots.

4.2.5.7 Worked example: circular path

For a circle of radius 10 mm and maximum normal acceleration 500 mm/s²:

$$\kappa = 0.1 \text{ mm}^{-1},$$

$$v \leq \sqrt{500/0.1} = 70.71 \text{ mm/s} = 4242.6 \text{ mm/min}.$$

A requested feed of 6000 mm/min violates the acceleration constraint even before axis limits and process force are considered.

4.2.5.8 Exercises

1. Derive $\dot{q}$ and $\ddot{q}$ from $q(s(t))$.
2. Compute a curvature-limited feed for a given radius and acceleration.
3. Explain why knot-only checking can miss a violation.
4. List four process constraints beyond machine dynamics.
5. Design a schedule witness for piecewise-constant feed.
6. Explain how controller lookahead complicates guarantees.

4.2.6 Robust and Multiobjective Optimization

Section goals. The reader should be able to formulate robust constraints, compare worst-case and chance constraints, construct Pareto tradeoffs, and record optimization evidence honestly.

4.2.6.1 Nominal optimization is brittle

A plan optimal for nominal tool diameter, transform, and stock may become unsafe under small deviations. Robust optimization requires constraints to hold for every parameter in an uncertainty set $\mathcal{U}$:

$$g(x, u) \leq 0 \quad \forall u \in \mathcal{U}.$$

This can be conservative but aligns with bounded safety claims.

4.2.6.2 Chance constraints

When uncertainty has a justified probability model, a chance constraint is:

$$\Pr[g(x, U) \leq 0] \geq 1 - \alpha.$$

A 99.9% success probability is not equivalent to a deterministic safety bound. The certificate must identify the distribution, evidence, and acceptable risk level. For collision and protected-target constraints, deterministic robust margins are generally preferable.

4.2.6.3 Scenario methods

When robust constraints are difficult, optimize over sampled scenarios and validate the final candidate with a deterministic checker. Scenario optimization can improve candidates but sampled feasibility alone should not be labeled universal proof.

4.2.6.4 Multiple objectives

Cycle time, surface quality, tool wear, energy, noise, and safety margin can conflict. The Pareto frontier contains solutions where no objective improves without worsening another.

Methods include:

- weighted sums;
- lexicographic optimization;
- epsilon constraints;
- interactive selection;
- goal programming.

The UI should expose meaningful tradeoffs rather than one opaque score.

4.2.6.5 Margin as an objective

After hard minimum clearance is satisfied, additional margin can be optimized:

$$\max_t \min d(\text{Assembly}(t), O).$$

This produces more robust links, though it may increase time. Margin evidence can also simplify runtime uncertainty accommodation.

4.2.6.6 Sensitivity analysis

A useful optimizer reports how objective and feasibility change with inputs:

- time sensitivity to feed limit;
- clearance sensitivity to WCS error;
- scallop sensitivity to stepover;
- tool-change sensitivity to operation grouping;
- optimality sensitivity to stock-dependent link costs.

High sensitivity identifies assumptions that deserve measurement or larger reserve.

4.2.6.7 Worked example: two link candidates

Candidate A takes 1.2 seconds with certified clearance 0.15 mm. Candidate B takes 1.35 seconds with clearance 2.0 mm. If frame uncertainty can reach 0.2 mm, A is infeasible under the robust model while B remains feasible. The nominal fastest candidate is not the robust optimum.

4.2.6.8 Exercises

1. Formulate a robust travel constraint under bounded WCS translation.
2. Distinguish robust and chance constraints.

3. Explain why scenario testing is not universal proof.
4. Construct a two-objective Pareto example for time and clearance.
5. Define clearance margin as an optimization objective.
6. Propose a sensitivity report for a finishing operation.

Checkpoint - optimization. Separate hard feasibility constraints from objectives, and describe the evidence needed to call a schedule globally optimal rather than merely feasible.

4.3 Implementation architecture, the complete example, and testing

A sound theory needs an implementation shape that preserves its boundaries.

Definition - reference semantics. A reference semantics is a simple, explicit interpreter or mathematical model used as the standard against which optimized implementations are compared.

CAM example. A pure interpreter for canonical commands can drive tests, translation validators, the time estimator, and parse-back comparison.

Definition - property-based testing. Property-based testing generates many inputs and checks general laws rather than only hand-written examples.

CAM example. Reversing a path twice should return an equivalent path; serializing and reparsing legal controller IR should preserve interpreted traces.

Definition - metamorphic testing. Metamorphic testing checks how outputs should relate when inputs are transformed in a meaning-preserving way.

CAM example. Translating the entire workpiece, fixtures, target, and path by the same rigid transform should not change clearance relationships.

Definition - differential testing. Differential testing compares independent implementations on the same inputs and investigates disagreement.

CAM example. Compare the optimized modal interpreter with a small reference interpreter, or compare two robust geometric kernels on adversarial cases.

Definition - acceptance gate. An acceptance gate is a machine-enforced condition that must pass before an artifact advances to a more hazardous operating mode.

CAM example. Preview may permit simulation-only results; an attended material cut requires stronger identity, travel, interlock, and setup checks; unattended operation demands a substantially higher assurance case.

4.3.1 Package Architecture for a TypeScript CAM Compiler

Section goals. The reader should be able to map semantic layers to packages, enforce dependency direction, isolate trusted checkers, and design compilation artifacts for browser and command-line frontends.

4.3.1.1 Architectural boundaries

A package graph should follow semantic dependency, not UI convenience. A practical decomposition is:

```

@cam/units, @cam/math
|
@cam/ir, @cam/machine
|
@cam/geometry, @cam/strategies
|
@cam/planner
|
@cam/compiler
|
@cam/post-*
|
@cam/analysis and checker packages

```

User interfaces depend on these packages. Core packages should not depend on React, Redux, CodeMirror, Three.js, browser globals, or controller sockets.

4.3.1.2 A semantics package

The reviewed architecture contains IR, compiler, analysis, and postprocessor packages, but a mature design should add a pure semantic center:

```

@cam/semantics
  canonical command interpreter
  machine-state transition relation
  stock-effect semantics
  trace events
  structured predicates

```

All of the following should use it:

- simulator;
- static analyzer;
- time estimator;
- pass validators;
- final G-code comparator;
- controller model;
- tests.

A viewer may approximate for speed but should not silently become the reference semantics.

4.3.1.3 Checker packages

Separate producer and checker namespaces:

@cam/geometry	untrusted algorithms
@cam/geometry-checker	robust predicates and witness checks
@cam/compiler	untrusted passes
@cam/pass-checker	translation validators
@cam/post-maker	serializer
@cam/gcode-semantics	independent parser and interpreter
@cam/analysis	simulations and diagnostics

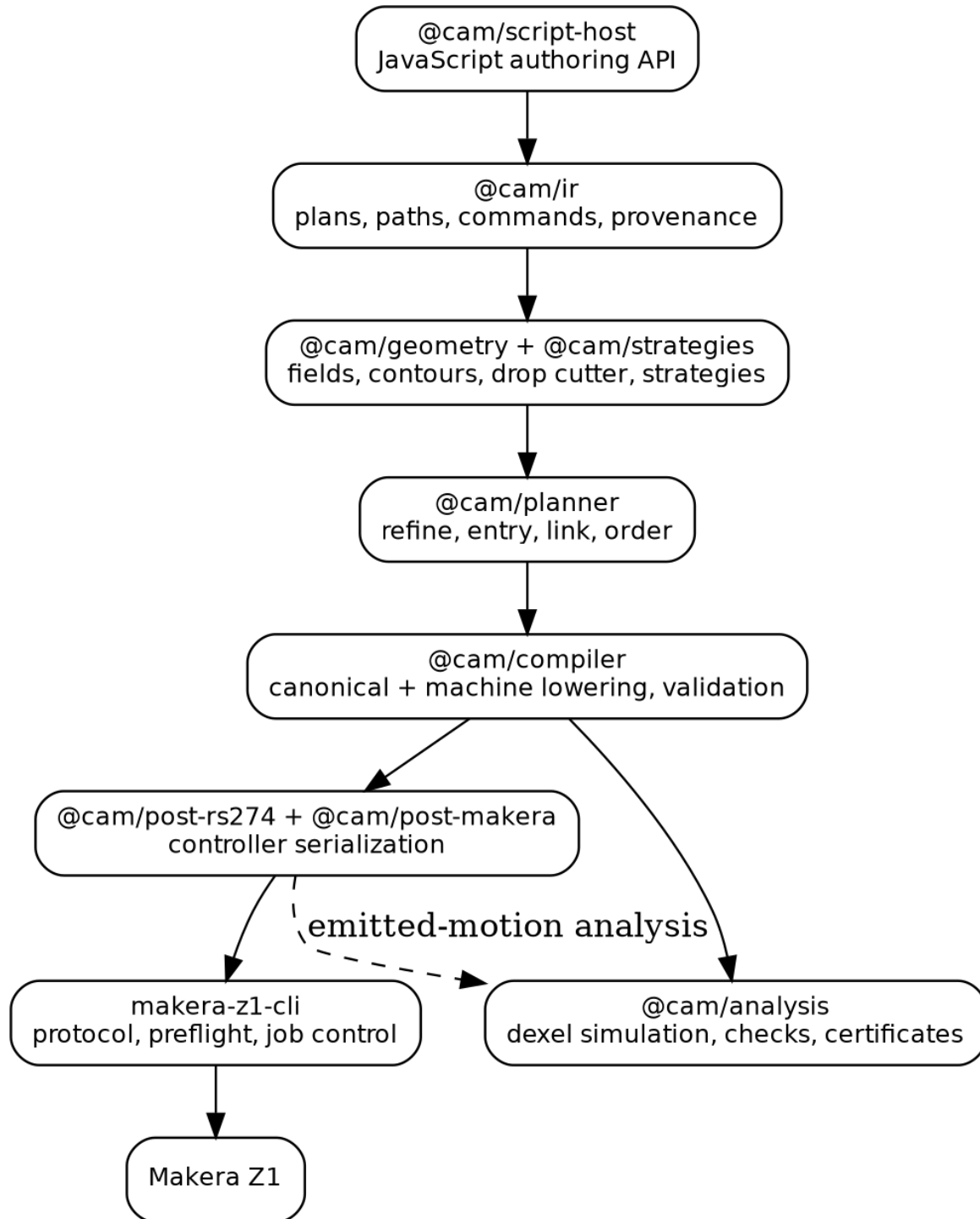


Figure 4.4: The reviewed Dropcut package architecture and controller boundary.

@cam/certificate-checker policy and proof graph verification

This separation helps detect accidental reuse of producer code in the checker.

4.3.1.4 Artifact store

Large arrays and meshes should not be duplicated through Redux or JSON. Use content-addressed immutable artifacts:

```
interface ArtifactStore {
  put<T>(schema: SchemaRef<T>, value: T): Promise<ArtifactRef<T>>;
  get<T>(ref: ArtifactRef<T>): Promise<Readonly<T>>;
  has(hash: Hash): Promise<boolean>;
}
```

The project state contains references. Derived artifacts can be cached by pass key and invalidated by dependency hashes.

4.3.1.5 Worker boundaries

Browser execution should isolate expensive or untrusted work:

- script evaluation worker or subprocess;
- planning worker;
- geometry/checker worker;
- viewer worker when practical.

Messages contain immutable artifact references and structured diagnostics. The UI thread never executes user scripts or long geometric loops.

4.3.1.6 CLI and Studio parity

The Studio and CLI should call the same compilation service over the same artifacts. A shared orchestration function prevents divergence:

```
interface CompileRequest {
  source: ArtifactRef<ScriptSource>;
  project: ArtifactRef<ProjectManifest>;
  target: TargetProfileRef;
  policy: AssurancePolicyRef;
}

interface CompileResponse {
  artifacts: CompilationArtifactGraph;
  diagnostics: readonly Diagnostic[];
  finalBundle?: ArtifactRef<ExecutableBundle>;
}
```

The Studio adds interactive visualization; the CLI adds reproducible automation. Neither should implement hidden pass logic.

4.3.1.7 Dependency rules

Enforce with linting or build tooling:

- @cam/ir imports only units, math, and schema utilities;
- strategies do not import postprocessors;
- postprocessors do not import UI packages;
- checkers do not import producer implementations except shared semantic definitions;
- controller code does not accept arbitrary CAM objects, only final bundles and explicit control operations;
- code that can move the machine is in a narrow package with mandatory authorization context.

4.3.1.8 Versioning

Version separately:

- schemas;
- semantic relations;
- pass implementations;
- checkers;
- controller dialect profiles;
- machine profiles;
- certificate policies.

A pass bug fix may change implementation version without changing its relation. A revised geometric metric changes the relation and requires a new claim type.

4.3.1.9 Worked package layout

```
packages/  
  units/  
  math/  
  schemas/  
  ir/  
  semantics/  
  geometry/  
  geometry-checker/  
  strategies/  
  planner/  
  compiler/  
  pass-checker/  
  machine/  
  controller-ir/  
  gcode-parser/  
  gcode-semantics/  
  post-rs274/  
  post-maker/  
  certificate-schema/  
  certificate-checker/  
  runtime-assurance/  
apps/  
  studio/  
  cli/  
  zl-controller/
```

4.3.1.10 Exercises

1. Draw an allowed import graph for the proposed packages.

2. Identify three producer/checker code-sharing risks.
3. Design an artifact-store cache key for machine lowering.
4. Specify a worker message for planning progress and cancellation.
5. Explain how Studio and CLI can share the exact same compiler.
6. Define a version-compatibility rule for a certificate checker.

4.3.2 An End-to-End Certified Pocket Example

Section goals. The reader should be able to trace one operation through every IR, state pass relations, construct a certificate graph, and identify compile-time and runtime obligations.

4.3.2.1 Source program

Consider a rectangular pocket in aluminum:

```
const T1 = tool.flatEndMill({
  id: "T1",
  diameter: mm(3.175),
  fluteLength: mm(12),
  stickout: mm(18),
});

const P1 = feature.rectPocket({
  id: "P1",
  frame: "work:G54",
  origin: p(mm(10), mm(10), mm(0)),
  width: mm(30),
  height: mm(20),
  depth: mm(4),
  wallTolerance: mm(0.04),
  floorTolerance: mm(0.04),
});

job.rough(P1, {
  tool: T1,
  radialAllowance: mm(0.20),
  axialAllowance: mm(0.20),
  strategy: "offset-pocket",
});

job.finish(P1, {
  tool: T1,
  strategy: "wall-and-floor",
});
```

The source leaves pass count, stepover, path orientation, entries, and links unspecified.

4.3.2.2 Authoring and elaboration

The staged evaluator emits an immutable AST. Elaboration resolves:

- the G54 frame;
- tool geometry artifact;
- stock, target, and fixture references;

- material defaults;
- finite and positive domains;
- explicit tolerances and allowances;
- source spans and operation IDs.

Claims:

C1: Plan IR is schema-valid and all references resolve.
 C2: Every numeric field is finite and satisfies its domain.
 C3: Every geometric quantity has a unit and frame.

4.3.2.3 Intent IR

The roughing protected solid is the target dilated by allowance. The finishing protected solid is the nominal target eroded only by the allowed gouge tolerance. Required-removal regions are defined separately.

```
interface PocketIntent {
    requiredRemoval: SolidRef;
    forbiddenRemoval: SolidRef;
    permittedRemoval: SolidRef;
    wallMetric: SurfaceDeviationMetric;
    floorMetric: SurfaceDeviationMetric;
    dependencies: readonly OperationId[];
}
```

4.3.2.4 Planning

The roughing strategy computes an inward tool-center region and offset loops. It emits:

- cut paths;
- coverage witness;
- boundary protection witness;
- directionality;
- local geometric error bounds;
- unresolved cells, ideally empty.

The finishing strategy emits one floor path family and one wall contour family with feature provenance.

Claims:

C4: guaranteed inner sweep covers roughing required-removal region.
 C5: outer cutting sweep avoids roughing protected region.
 C6: finishing path deviation meets wall and floor metrics.

4.3.2.5 Scheduling and linking

The scheduler chooses:

1. roughing passes from high to low depth;
2. floor finish;
3. wall finish;
4. safe entries and links against each evolving stock state;
5. spindle start before first cut and stop after final retract.

A schedule witness verifies precedence and recomputed transition costs.

Claims:

C7: schedule respects all operation and stock-support dependencies.
 C8: each entry is feasible for the tool and current stock.
 C9: each traverse outer assembly sweep avoids stock and fixtures.

4.3.2.6 Machine lowering

The Z1 profile resolves:

- machine travel;
- G54 semantics;
- supported spindle range;
- arc capability;
- maximum feeds;
- traverse expansion policy;
- probe and accessory semantics.

If the profile does not guarantee coordinated diagonal rapid, abstract traverses lower to explicit Z/XY/Z moves.

Claims:

C10: every scheduled operation is supported or soundly transformed.
 C11: all transformed poses remain within machine travel under WCS uncertainty.
 C12: feeds and spindle requests lie within static profile limits.

4.3.2.7 Controller lowering and serialization

Controller IR establishes a known modal state, emits explicit motion semantics, and ends in a defined safe state. The serializer generates deterministic bytes.

A final independent parser interprets the exact bytes and compares them to Controller IR.

Claims:

C13: parsed final bytes produce the same canonical trace within rounding bound.
 C14: no raw or unknown controller effect remains.
 C15: final interpreted spindle state is off.
 C16: byte artifact hash is H.

4.3.2.8 Geometric final validation

Final interpreted motion is checked, not merely pre-postprocessor paths. The checker uses:

- outer tool and holder sweeps for forbidden intersections;
- inner cutting sweeps for guaranteed removal;
- evolving stock for traverses;
- target-specific deviation metrics;
- typed error budgets including formatting and transform uncertainty.

A local counterexample invalidates the corresponding claim and links back through provenance.

4.3.2.9 Executable bundle

```
interface ExecutableBundle {
    jobBytes: ArtifactRef<Uint8Array>;
```

```

jobHash: Hash;
intent: ArtifactRef<IntentIR>;
setup: ArtifactRef<SetupManifest>;
machineProfile: ArtifactRef<MachineProfile>;
certificate: ArtifactRef<CertificateGraph>;
runtimePolicy: AssurancePolicyRef;
}

```

4.3.2.10 Runtime discharge

Before start, the host establishes:

- live machine and firmware compatible with profile;
- homing complete;
- G54 inside certified interval;
- T1 loaded and measured within bounds;
- fixture/setup identity correct;
- no alarm;
- exact stored hash equals H .

Only then does it authorize one execution instance.

4.3.2.11 Failure example

Suppose coordinate formatting consumes 0.003 mm and the live G54 uncertainty is 0.020 mm rather than the budgeted 0.010 mm. The finishing wall budget exceeds 0.04 mm. The same .nc bytes may remain syntactically valid, but the executable assurance bundle is not valid under the live setup. Revalidation or re-probing is required.

4.3.2.12 Exercises

1. Draw the artifact and claim DAG for the pocket.
2. Identify which claims are exact, bounded, assumed, and runtime-checked.
3. Design a coverage witness for the roughing loops.
4. State the machine-lowering relation for a traverse.
5. Explain why final geometric checks use interpreted bytes.
6. Give three changes that invalidate the bundle without changing source code.

4.3.3 Testing: Properties, Fuzzing, Differential, and Metamorphic Methods

Section goals. The reader should be able to build a layered test strategy, derive property and metamorphic tests from semantics, and target numerical and protocol edge cases.

4.3.3.1 Tests are not proofs, but semantics improve tests

Testing cannot establish universal correctness over continuous geometry and unbounded programs. It remains essential for finding defects, validating checkers, and protecting implementations. Formal semantics supply high-value oracles and transformations.

4.3.3.2 Unit tests

Unit tests cover:

- constructors and domain checks;
- individual transfer functions;
- parser productions;
- numeric primitives;
- artifact hashing;
- diagnostic provenance;
- known controller replies;
- known geometry regressions.

Every reported production defect should become a permanent regression.

4.3.3.3 Property-based tests

Properties include:

path identity:

```
concat(empty(start(p)), p) == p
```

exact transform inverse:

```
inverse(T) * (T * p) == p within numeric bound
```

serialization determinism:

```
serialize(ir, c) has stable bytes
```

round trip:

```
parse(serialize(ir)) semantically equals ir
```

certificate dependency:

```
changing any hashed input invalidates dependent claims
```

contour chaining:

```
optimized result equals a collision-free tuple-key reference
```

Generate data across the full machine envelope, not only near the origin.

4.3.3.4 Metamorphic testing

When an exact expected output is difficult, transform the input in a way that predicts a relation between outputs.

Examples:

- translating the entire setup and machine envelope should translate paths and preserve relative collision results;
- rotating a symmetric problem should rotate the output and preserve objective value;
- subdividing a line should preserve interpreted motion;
- adding redundant modal words should preserve canonical trace;
- reordering independent declarations should not change artifact semantics;
- increasing a tolerance should not turn a previously feasible geometric approximation infeasible, absent a strategy change.

Metamorphic tests are particularly effective for CAM algorithms.

4.3.3.5 Differential testing

Compare independent implementations:

- optimized contour chaining versus slow tuple-key reference;
- postprocessor parse-back versus canonical interpreter;
- analytic line/arc bounds versus dense high-precision sampling;
- TypeScript checker versus a Python or exact-kernel reference;
- controller protocol parser versus captured traces;
- simulator representations at increasing resolution.

Agreement does not prove correctness, but disagreement is highly informative.

4.3.3.6 Fuzzing parsers and protocols

Fuzz:

- embedded newlines;
- compound G-code blocks;
- comments and delimiters;
- truncated frames;
- duplicate or late replies;
- partial writes;
- invalid lengths and checksums;
- random byte loss;
- protocol switches;
- huge numeric fields;
- negative zero and non-finite values.

Stateful protocol fuzzing should track the model state and search for invariant violations.

4.3.3.7 Numerical adversaries

Generate cases near:

- collinearity and tangency;
- grid nodes exactly on contour levels;
- saddle cells;
- coordinate quantization boundaries;
- machine travel limits;
- arc sweeps near 0, π , and 2π ;
- very large and very small scales;
- negative coordinates;
- repeated points;
- tool/feature widths differing by tiny amounts.

4.3.3.8 Checker mutation testing

Deliberately corrupt artifacts and witnesses:

- alter one coordinate after certification;
- change tool hash;
- remove a precedence edge;
- change modal preamble;
- truncate bytes;

- swap evidence between claim types;
- lie about resolution;
- modify a runtime assumption.

The checker should reject every mutation that affects the proposition.

4.3.3.9 Physical test ladder

A physical bring-up sequence should progress through:

1. offline semantics and checker tests;
2. protocol tests against a fake machine;
3. live read-only status;
4. stop and hold tests;
5. bounded jog tests;
6. spindle/accessory tests without a tool where appropriate;
7. air cuts above stock;
8. sacrificial soft material;
9. attended real material at conservative feeds;
10. only then broader operation.

Each step has explicit pass/fail criteria and recovery procedures.

4.3.3.10 Exercises

1. Write five property tests for path and frame algebra.
2. Design a translation metamorphic test for a pocket job.
3. Build a differential oracle for contour chaining.
4. List ten protocol fuzz cases.
5. Design a certificate mutation suite.
6. Define pass/fail criteria for an air-cut test.

Checkpoint - implementation. Identify the reference semantics, independent checkers, worker boundaries, and acceptance gates in your architecture; name a property, metamorphic, differential, and fuzz test for one pass.

4.4 The Dropcut/Z1 lessons and a staged migration path

The Dropcut/Z1 implementation is useful because its defects cross semantic boundaries. Each bug is a concrete example of a term defined earlier in the book:

- a sampled stock-removal result was promoted into a target-gouge claim: **claim/evidence mismatch**;
- same-realm new Function execution escaped the intended API boundary: **failed staging and capability isolation**;
- command authorization classified only a lexical prefix and defaulted unknown text to read-only: **open-world protocol semantics**;
- a packed endpoint key repeated within the machine envelope and merged unrelated contours: **alias collision and invalid approximate identity**.

The purpose of the case study is not to criticize one codebase. It demonstrates why the architecture needs semantic boundaries, closed grammars, independent checkers, content identity, and counterexample-driven tests.

4.4.1 The Dropcut/Z1 Case Study: What the Defects Teach

Section goals. The reader should be able to connect concrete implementation defects to missing semantic distinctions, derive architectural remedies, and avoid local patches that leave the underlying proof obligation undefined.

4.4.1.1 Scope of the reviewed snapshot

The case study refers to the Dropcut Studio and Makera Z1 controller snapshot reviewed in August 2026, including the `task/cnc-control-dropcut` branch near commit `e82bed1e5a00f38f4441e6ea13e1265edc775928`. The repository already contains several strong decisions:

- non-modal canonical machining commands;
- explicit path objects and provenance;
- separate planning, machine lowering, postprocessing, and analysis packages;
- capability-driven lowering;
- emitted-motion analysis;
- risk classes and fresh preflight in the controller;
- visible distinctions between exact, sampled, unchecked, and unverifiable states.

The critical defects are valuable because each reveals a missing theory boundary.

4.4.1.2 Case A: evidence for one claim minted another claim

In `packages/analysis/src/checks.ts`, sampled checks simulate emitted moves against stock and record rapid-through-material and spoilboard penetration. The inputs do not include a protected target surface or allowance model. In `packages/analysis/src/certificate.ts` and `packages/compiler/src/recertify.ts`, the same generic sampled status is attached to both `rapidCrash` and `gouge`.

The failure is not primarily a bad conditional. It is an evidence-type failure:

```
SampledEvidence
  -> generic status
  -> any sampled certificate row
```

A correct architecture is:

```
RapidCollisionEvidence -> RapidCollisionClaim
GougeEvidence          -> GougeClaim
RequiredRemovalEvidence -> RequiredRemovalClaim
```

The claim schema should require the target artifact hash. Without that dependency, a gouge proposition is not even well formed.

Theory lesson: propositions, methods, and evidence types must be separated. Sampled simulation without a continuous enclosure is not a bounded proof.

4.4.1.3 Case B: same-realm script execution was described honestly but remained unsafe

`packages/script-host/src/sandbox.ts` shadows obvious globals and documents that this is not a security boundary. It still compiles with `new Function` in the current realm. The `timeoutMs` option is not enforced inside `runScript`; it relies on a caller to terminate a worker, while Node execution remains in-process.

A constructor/prototype path can recover ambient authority, and a nonterminating script can block an in-process CLI or worker until external termination.

The architectural fix is staging:

1. execute JavaScript in a separately terminable environment;
2. give it only serializable capabilities and inputs;
3. return an inert AST;
4. destroy the execution environment;
5. compile the AST without invoking user code.

Theory lesson: a capability API is not a sandbox unless the runtime enforces the capability boundary. The authoring language and CAM language must be distinct.

4.4.1.4 Case C: free-text command classification used lexical prefixes instead of semantics

The controller classifier in `makera-z1-cli/pkg/makera/safety.go` splits on whitespace, classifies `parts[0]`, and returns read-only for unknown plain verbs. The protocol encoder strips only trailing newlines, so embedded separators can remain. A payload beginning with an allowed read verb can contain a later motion command.

The local patch “inspect more tokens” is not enough because controller syntax is block-structured and dialect-specific. The correct boundary is a fail-closed parser:

```
bytes -> complete payload grammar -> command/effect AST -> risk policy
```

Unknown syntax is not read-only. It is denied or routed through explicit authorization.

Theory lesson: authorization is a semantic property of the complete parsed payload. Prefix classifiers are not interpreters.

4.4.1.5 Case D: a packed endpoint key changed topology

`packages/geometry/src/contours.ts` quantizes coordinates at 10^{-6} mm and takes each coordinate modulo 2^{26} . The key repeats after:

$$2^{26} \times 10^{-6} \text{ mm} = 67.108864 \text{ mm}.$$

That period lies inside the machine work envelope. Unrelated endpoints separated by the period can share identity, causing disconnected contour segments to merge or appear closed.

The optimization was motivated by avoiding string allocation. It silently replaced a tuple with a non-injective encoding.

Repairs include:

- nested maps keyed by exact quantized integers;
- string tuple keys if performance is sufficient;
- signed integer pairing with a proven range;
- `BigInt` pairing;
- best of all for marching squares, grid-edge topology IDs.

Theory lesson: topological identity must be injective over the declared domain. Approximate numerical equality and hash keys require domain proofs, not comments.

4.4.1.6 Cross-boundary findings

Several high-severity issues in the same snapshot follow the same pattern:

- work offsets were accepted in the API but not consistently emitted or incorporated into machine travel checks;
- a validator documented a safety epilogue that the final emitter did not append;
- probe commands confused the commanded endpoint with the measured result;
- a missing traverse could be replaced by a cut, changing stock semantics;
- raw text could invalidate modal compression;
- machine-profile identity was not fully bound to validation;
- entry and linking used incomplete stock and holder models;
- an error budget added heterogeneous geometric numbers without a formal metric;
- controller preflight and execution were not one atomic state transition;
- timeout and message-stream behavior could leave ambiguous session state.

Each is a boundary mismatch: API versus emitter, canonical versus modal semantics, planned versus final motion, nominal versus physical frame, admission versus execution.

4.4.1.7 What should be preserved

The correct response is not a rewrite that discards the strong structure. Preserve:

- canonical non-modal commands;
- path and provenance types;
- capability-based lowering;
- exact-versus-sampled status distinctions;
- emitted-motion timeline;
- typed controller operations;
- stop commands that remain available;
- fresh preflight;
- no blind retry after ambiguous motion failure;
- dead-man jog behavior.

Then strengthen the semantic contracts and checker boundaries.

4.4.1.8 Counterexample-driven architecture

Each critical bug should become more than a regression test. It should shape a type or schema:

Counterexample	Architectural response
Gouge row without target	Target hash required by GougeClaim; only GougeEvidence can discharge it
Constructor escape	Process/worker isolation; AST-only output
Multiline command hidden after read verb	Complete fail-closed payload parser
67.108864 mm key alias	Injective topology identity over machine envelope
WCS not reflected in travel	Frame-explicit machine lowering and runtime transform assumption
Safety stop promised but absent	Final-byte parse-back claim
Raw command changes modality	Unknown abstract state or rejection
Timeout reply arrives late	Quarantined protocol state and resynchronization

4.4.1.9 Exercises

1. For each critical bug, identify the false proposition the implementation could appear to establish.
2. Redesign the evidence types for sampled analysis.
3. Specify a fail-closed command grammar and policy.
4. Prove that a proposed endpoint-key scheme is injective over a declared coordinate range.
5. Explain why a final-byte validator catches an omitted epilogue.
6. Design a quarantined session state after timeout.
7. Choose one high-severity finding and state its missing pass relation.

4.4.2 A Staged Migration Roadmap

Section goals. The reader should be able to prioritize semantic risk, introduce certificates incrementally, and define acceptance gates for progressively stronger operating modes.

4.4.2.1 Principles for migration

A running codebase cannot become formally verified in one step. The migration should:

- remove false claims before adding stronger claims;
- make unknown states explicit;
- introduce independent checkers at stable boundaries;
- preserve productive planners and UI work;
- bind artifacts and versions early;
- move physical control behind a narrow assurance interface;
- define operating modes with clear evidence requirements.

4.4.2.2 Phase 0: stop overclaiming

Immediate changes:

1. Remove the automatic gouge upgrade when no target-specific evidence exists.
2. Rename sampled statuses to `simulation-only` unless a continuous enclosure theorem exists.
3. Mark holder, fixture, WCS, and machine identity dependencies explicitly.
4. Fail closed around raw controller effects and unknown commands.
5. Bind every certificate to the exact final byte hash.
6. Disable production-cut labels for bundles missing required claims.

This phase improves honesty even before new algorithms exist.

4.4.2.3 Phase 1: harden language and identity

- execute scripts in a separately terminable environment;
- produce an immutable authoring AST;
- centralize finite/domain checks;
- deep-freeze or copy boundary objects;
- make units and frames explicit;
- content-address all major artifacts;
- record pass, strategy, machine, firmware, and checker versions;
- establish reproducible CLI compilation.

Acceptance gate: identical declared inputs produce identical Plan IR and final bytes.

4.4.2.4 Phase 2: executable semantics and final-byte validation

- create @cam/semantics;
- create an independent controller parser and interpreter;
- define canonical trace events;
- validate modal compression;
- parse back exact bytes;
- compare interpreted final motion to Machine IR;
- verify preamble and epilogue on bytes;
- propagate rounding bounds.

Acceptance gate: every emitted production file has a successful parse-back equivalence claim.

4.4.2.5 Phase 3: pass witnesses

Introduce translation validators for:

- unit/frame elaboration;
- path continuity and arc validity;
- traverse expansion;
- arc linearization and fitting;
- feed clamping;
- operation scheduling and precedence;
- machine capability lowering;
- serialization.

Acceptance gate: each production pass either emits a checked witness or is inside the trusted core with an explicit test/proof plan.

4.4.2.6 Phase 4: robust geometric kernel

- exact/adaptive predicates for topology;
- interval and conservative bounding primitives;
- injective contour identity;
- tool and holder geometry;
- target, stock, and fixture artifact schemas;
- inner and outer swept-volume approximations;
- continuous line/arc collision checks;
- adaptive subdivision for general paths;
- claim-specific evidence.

Acceptance gate: no-gouge, fixture, holder, and required-removal claims are separate and target-specific.

4.4.2.7 Phase 5: controller state machine

- replace generic free-text read paths with parsed operations;
- serialize admission and execution under one authority;
- add state epochs and exact job hashes;
- quarantine after ambiguous timeout;
- handle partial writes;
- separate durable control events from lossy telemetry;

- model upload, start, hold, resume, abort, and alarm;
- model-check key safety and liveness properties.

Acceptance gate: every motion-enabling transition is authorized against fresh state and exact artifact identity.

4.4.2.8 Phase 6: runtime assurance

- live machine and firmware identity;
- tool/setup/WCS checks;
- calibration records and validity;
- exact upload hash acknowledgement;
- execution-instance binding;
- watchdog and stop confirmation;
- bounded dynamic envelopes;
- explicit degradation modes.

Acceptance gate: compile-time claims and runtime assumptions compose into one checked execution authorization.

4.4.2.9 Phase 7: optimization maturity

Only after feasibility checking is dependable:

- precedence-aware sequencing;
- stock-dependent link planning;
- clearance-aware free-space search;
- feed scheduling under dynamics and process limits;
- solver certificates and objective gaps;
- robust/multiobjective optimization.

Optimization should consume and produce certified artifacts, not bypass the assurance pipeline.

4.4.2.10 Operating-mode gates

4.4.2.10.1 Development and preview

- schema-valid AST and IR;
- deterministic build record;
- simulation diagnostics;
- no machine connection required.

4.4.2.10.2 Attended air cut

- exact bytes and parse-back;
- machine travel and controller-state claims;
- known tool and WCS;
- machine identity and fresh preflight;
- operator present with stop access.

4.4.2.10.3 Attended material cut

- target, stock, fixture, holder, and tool models;

- no-gouge and collision evidence;
- required-removal or explicit limitation;
- runtime assumption discharge;
- conservative feeds and physical setup verification.

4.4.2.10.4 Unattended or production use

- validated protocol liveness and recovery;
- runtime assurance monitor;
- calibrated dynamic and stopping envelopes;
- durable event logging;
- interlocks and physical risk assessment;
- organizational release and incident process.

4.4.2.11 Definition of done

A production compiler is not done because it emits plausible G-code. A meaningful definition of done is:

1. Every stage has explicit syntax, semantics, and legality.
2. Every pass names a refinement relation.
3. Every approximation names metric, direction, bound, and assumptions.
4. Every safety claim has claim-specific evidence and an independent checker.
5. Every certificate is bound to exact artifacts and versions.
6. Every controller transition belongs to a checked state machine.
7. Runtime authorization establishes live assumptions and exact job identity.
8. Unknowns cause rejection or explicit degradation, never silent success.

4.4.2.12 Final perspective

The theoretical framework is not decoration around CAM algorithms. It tells the implementation where facts live and which conclusions follow from them. Category theory clarifies path composition and effectful sequencing. Denotational semantics identifies acceptable physical outcomes. Operational semantics makes modal and machine behavior executable. Hoare logic derives preconditions. Abstract interpretation covers families of states. Robust geometry distinguishes samples from enclosures. Translation validation makes evolving algorithms practical. Proof-carrying architecture keeps the trusted base small. Operations research improves performance after feasibility is established. Temporal logic and runtime assurance close the gap between files and machines.

The result is not a claim that software can eliminate physical uncertainty. It is a system that states exactly what it knows, what it assumes, how it knows it, which artifact the statement concerns, and what must still be checked before motion begins.

4.4.2.13 Exercises

1. Convert the roadmap into a twelve-month engineering plan with dependencies.
2. Define acceptance criteria for Phase 2 final-byte validation.
3. Choose three claims to implement first and justify their risk reduction.
4. Design an operating-mode policy for a hobby workshop.
5. Explain why optimization maturity is deliberately last.
6. Write a one-page assurance case for an attended air cut.

Checkpoint - case study. Map each reviewed defect to a violated concept: claim/evidence relation, staging boundary, closed-world protocol grammar, or artifact identity/topological identity.

4.5 A failure trace that filenames cannot prevent

Host verifies bytes H1 and saves them as job.nc.
 Host uploads job.nc; controller stores H1.
 Operator edits local job.nc, producing H2 with the same filename.
 UI still displays "job.nc verified" from the earlier result.
 A retry or controller-side selection starts a different stored file H3.

Every filename comparison succeeds while artifact identity fails. A safe protocol binds the certificate to the exact byte hash, binds upload acknowledgement to the stored hash, and binds start authorization to that hash plus the current machine-state epoch.

4.6 Chapter synthesis: from certificate to controlled execution

A practical runtime handshake can be summarized as follows:

1. Host presents certified job hash H and required assumptions A.
2. Controller reports machine identity, firmware profile, state epoch, active tool, homing state, work offsets, alarms, and stored job hash H'.
3. Host verifies $H = H'$ and checks that reported state discharges A.
4. Host authorizes H for the current epoch E.
5. Controller starts only the pair (H, E), acknowledges the execution instance, and invalidates authorization when relevant state changes.
6. Runtime monitor observes the execution envelope and stop policy.

This protocol does not prove the fixture model matches the real clamp. That remains a physical assumption or a fact to be checked by sensing and operator procedure. The protocol does prevent a class of software identity and stale-state errors.

4.6.1 Feasibility before optimization in the running pocket

The schedule optimizer may choose contour direction, island order, tool changes, and links. It may minimize

$$J = T_{\text{cycle}} + \lambda_1 N_{\text{tool changes}} + \lambda_2 D_{\text{rapid}},$$

but only subject to hard constraints:

- roughing precedes finishing;
- probe-derived transforms precede dependent motion;
- every link is clear in the stock state that exists at that point in the schedule;
- the tool and holder remain clear of fixtures;
- machine velocity, acceleration, jerk, and controller limits are respected;
- target-protection and required-removal claims remain valid;
- the exact final bytes remain bound to the certificate and execution instance.

The optimizer may be heuristic. A certificate can honestly say “verified feasible with estimated objective J .” It should say “optimal” only when the solver provides a valid lower bound or optimality certificate.

4.6.2 A small trusted core

A sensible first trusted core contains:

1. structured claim definitions;
2. artifact hashing and canonical serialization;
3. a simple canonical-command and controller interpreter;
4. final-byte parse-back comparison;
5. closed command grammar and protocol-state checker;
6. interval and robust-predicate primitives used by geometric checkers;
7. certificate-DAG validation;
8. runtime hash/epoch authorization.

Toolpath strategy generation, visualization, editor logic, search heuristics, and most optimizers should remain outside the trusted core and produce candidates plus evidence.

4.6.3 Common misconceptions

1. **“The file was verified, so runtime state no longer matters.”** Work offsets, tools, firmware, stored bytes, and alarms can change after compilation.
2. **“An abort request succeeded, so the machine stopped.”** Completion requires an observed stopped or faulted state and must account for latency and stopping distance.
3. **“Unknown commands are probably harmless.”** A machine-control authorization policy must be closed-world and fail closed.
4. **“The shortest link is best.”** A slightly longer link with larger certified clearance may be preferable under uncertainty.
5. **“Tests replace proofs.”** Tests find defects and validate implementations; they do not establish universal continuous-domain properties unless connected to a proof argument.

4.6.4 Capstone exercises

1. Specify a complete upload/start/hold/resume/abort protocol and state three safety and three liveness properties.
2. Formulate the running pocket’s operation ordering as a precedence-constrained routing problem. Identify state-dependent edge costs.
3. Design a runtime preflight record that binds machine profile, firmware semantics, tool assembly, stock, fixtures, work transform, and byte hash.
4. Propose acceptance gates for preview, attended air cut, attended material cut, and unattended operation.
5. Choose one Dropcut/Z1 defect and redesign the relevant representation, checker, and tests so the defect class becomes difficult to reintroduce.

Appendix A

Mathematical and Notational Reference

The main chapters introduce concepts in the order they are needed. This appendix collects notation in one place for later lookup. It is a reference, not a substitute for the motivating examples in Chapter 1.

This appendix collects the notation used throughout the book. It is intended as a working reference rather than a substitute for a course in analysis, geometry, logic, or optimization.

A.0.0.1 Sets and functions

Symbol	Meaning
$x \in A$	x is an element of set A
$A \subseteq B$	every element of A belongs to B
$A \cup B$	union
$A \cap B$	intersection
$A \setminus B$	set difference
$A \times B$	Cartesian product
$\mathcal{P}(A)$	powerset of A
$f : A \rightarrow B$	total function from A to B
$R \subseteq A \times B$	relation between A and B
$R; S$	relational composition

A partial function can be modeled as a total function into an option or result type, or as a relation with no successor for some inputs.

A.0.0.2 Logic

Symbol	Meaning
$\neg P$	not P
$P \wedge Q$	P and Q
$P \vee Q$	P or Q
$P \Rightarrow Q$	implication
$P \Leftrightarrow Q$	equivalence
$\forall x. P(x)$	for every x , $P(x)$

Symbol	Meaning
$\exists x.P(x)$	there exists an x satisfying $P(x)$
$\Box P$	always P in temporal logic
$\Diamond P$	eventually P in temporal logic

A safety property states that a bad event never occurs. A liveness property states that a good event eventually occurs. Many controller requirements combine both.

A.0.0.3 Orders and lattices

A preorder $\sqsubseteq$ is reflexive and transitive. A partial order is also antisymmetric. In compiler refinement, $Q \sqsubseteq P$ commonly means that Q has no behaviors outside those allowed by P .

A lattice provides least upper bounds $a \sqcup b$ and greatest lower bounds $a \sqcap b$. Abstract interpretation uses joins to combine control-flow paths.

A.0.0.4 Metric spaces

A metric d satisfies:

1. $d(x, y) \geq 0$;
2. $d(x, y) = 0$ iff $x = y$;
3. $d(x, y) = d(y, x)$;
4. $d(x, z) \leq d(x, y) + d(y, z)$.

The open ball of radius ε is:

$$B_\varepsilon(x) = \{y \mid d(x, y) < \varepsilon\}.$$

The distance from point x to set A is:

$$d(x, A) = \inf_{a \in A} d(x, a).$$

The Hausdorff distance between compact sets is:

$$d_H(A, B) = \max \left(\sup_{a \in A} d(a, B), \sup_{b \in B} d(b, A) \right).$$

A.0.0.5 Geometry and morphology

Symbol	Meaning
$\mathbb{R}^n$	n -dimensional Euclidean space
$SO(3)$	three-dimensional rotation group
$SE(3)$	rigid motions in three dimensions
$A \oplus B$	Minkowski sum
$A \ominus B$	erosion / Minkowski difference in morphological usage

Symbol	Meaning
∂A	boundary of A
$\text{int}(A)$	interior of A
A^-	guaranteed inner approximation
A^+	guaranteed outer approximation

For sets $A, B \subseteq \mathbb{R}^n$:

$$A \oplus B = \{a + b \mid a \in A, b \in B\}.$$

The configuration obstacle for a translating tool T and obstacle O is $O \oplus (-T)$.

A.0.0.6 Paths and trajectories

A geometric path is:

$$\gamma : [0, 1] \rightarrow Q,$$

where Q is configuration space. A time law is a monotone function:

$$s : [0, T] \rightarrow [0, 1].$$

The trajectory is $q(t) = \gamma(s(t))$.

For a tool solid T and pose $X(t)$:

$$\text{Sweep}(T, X) = \bigcup_t X(t)T.$$

A.0.0.7 Error and uncertainty

A deterministic bound states:

$$|x - \hat{x}| \leq \varepsilon.$$

An interval enclosure states:

$$x \in [x^-, x^+].$$

A probabilistic statement has the form:

$$\Pr(|X - \hat{x}| \leq \varepsilon) \geq 1 - \alpha.$$

These are not interchangeable.

A.0.0.8 Optimization

A constrained optimization problem is:

$$\min_x J(x) \quad \text{subject to} \quad g_i(x) \leq 0, \quad h_j(x) = 0.$$

A lower bound L and feasible candidate cost J satisfy:

$$L \leq J^* \leq J.$$

The gap quantifies the possible distance from global optimality.

Appendix B

Reference IR and Certificate Schemas

The following schemas are intentionally explicit. They illustrate a coherent design; a production implementation may split fields or use generated codecs.

B.0.0.1 Primitive identities and units

```
type Hash = string & { readonly __brand: "sha256" };
type SchemaId = string & { readonly __brand: "schema" };
type ArtifactId = string & { readonly __brand: "artifact-id" };
type FrameId = string & { readonly __brand: "frame" };
type ToolId = string & { readonly __brand: "tool-id" };
type FeatureId = string & { readonly __brand: "feature-id" };
type OperationId = string & { readonly __brand: "operation-id" };
type ClaimId = string & { readonly __brand: "claim-id" };
type AssumptionId = string & { readonly __brand: "assumption-id" };

type Mm = number & { readonly __unit: "mm" };
type MmPerMin = number & { readonly __unit: "mm/min" };
type Rpm = number & { readonly __unit: "rpm" };
type Radians = number & { readonly __unit: "rad" };
type Seconds = number & { readonly __unit: "s" };

interface ArtifactRef<T = unknown> {
  readonly id: ArtifactId;
  readonly hash: Hash;
  readonly schema: SchemaId;
  readonly mediaType: string;
  readonly byteLength: number;
  readonly _type?: T;
}
```

B.0.0.2 Frames and transforms

```
interface Point3<F extends FrameId = FrameId> {
  readonly x: Mm;
  readonly y: Mm;
  readonly z: Mm;
  readonly frame: F;
}
```

```

interface Quaternion {
  readonly x: number;
  readonly y: number;
  readonly z: number;
  readonly w: number;
}

interface RigidTransform<From extends FrameId, To extends FrameId> {
  readonly from: From;
  readonly to: To;
  readonly translation: readonly [Mm, Mm, Mm];
  readonly rotation: Quaternion;
}

interface TransformUncertainty {
  readonly translationBox: readonly [
    readonly [Mm, Mm],
    readonly [Mm, Mm],
    readonly [Mm, Mm],
  ];
  readonly angularRadius: Radians;
  readonly confidence: "deterministic-bound" | "probabilistic";
}

```

B.0.0.3 Geometry and setup

```

type GeometryRef = ArtifactRef<GeometryArtifact>;

interface GeometryArtifact {
  readonly kind:
    | "mesh"
    | "brep"
    | "implicit"
    | "voxel"
    | "dexel"
    | "height-field"
    | "primitive";
  readonly frame: FrameId;
  readonly enclosure?: {
    readonly direction: "inner" | "outer" | "two-sided" | "none";
    readonly metric: ErrorMetric;
    readonly bound?: ErrorBound;
  };
}

interface ToolAssembly {
  readonly id: ToolId;
  readonly cuttingGeometry: GeometryRef;
  readonly assemblyGeometry: GeometryRef;
  readonly fluteLength: Mm;
  readonly stickout: Mm;
  readonly measurementUncertainty: readonly ErrorBound[];
}

```

```

interface SetupManifest {
  readonly stock: GeometryRef;
  readonly target: GeometryRef;
  readonly fixtures: readonly GeometryRef[];
  readonly tools: readonly ArtifactRef<ToolAssembly>[];
  readonly frameGraph: ArtifactRef<FrameGraph>;
  readonly workSystem: string;
  readonly material?: ArtifactRef;
  readonly setupIdentity?: string;
}

```

B.0.0.4 Paths

```

interface LineSegment<F extends FrameId> {
  readonly kind: "line";
  readonly to: Point3<F>;
}

```

```

interface ArcSegment<F extends FrameId> {
  readonly kind: "arc";
  readonly to: Point3<F>;
  readonly center: Point3<F>;
  readonly axis: readonly [number, number, number];
  readonly sweep: Radians;
}

```

```

interface SplineSegment<F extends FrameId> {
  readonly kind: "spline";
  readonly basis: "bezier" | "bspline" | "nurbs";
  readonly degree: number;
  readonly controlPoints: readonly Point3<F>[];
  readonly knots?: readonly number[];
  readonly weights?: readonly number[];
}

```

```

type Segment<F extends FrameId> =
  | LineSegment<F>
  | ArcSegment<F>
  | SplineSegment<F>;

```

```

interface Path<F extends FrameId = FrameId> {
  readonly frame: F;
  readonly start: Point3<F>;
  readonly end: Point3<F>;
  readonly segments: readonly Segment<F>[];
  readonly identityPolicy: "exact" | "snapped" | "witnessed-approximate";
  readonly approximation?: readonly ErrorBound[];
}

```

B.0.0.5 Intent IR

```

interface RegionSpecification {
  readonly geometry: GeometryRef;
}

```

```

    readonly interpretation:
      | "required-removal"
      | "forbidden-removal"
      | "permitted-removal";
  }

interface SurfaceRequirement {
  readonly surface: GeometryRef;
  readonly metric: "normal-deviation" | "hausdorff" | "scallop-height";
  readonly maximum: Mm;
}

interface OperationIntentBase {
  readonly id: OperationId;
  readonly feature: FeatureId;
  readonly frame: FrameId;
  readonly toolConstraints: readonly ToolConstraint[];
  readonly predecessors: readonly OperationId[];
  readonly regions: readonly RegionSpecification[];
  readonly surfaceRequirements: readonly SurfaceRequirement[];
  readonly provenance: Provenance;
}

interface PocketIntent extends OperationIntentBase {
  readonly kind: "pocket";
  readonly floor: GeometryRef;
  readonly boundary: GeometryRef;
  readonly roughingAllowance: {
    readonly radial: Mm;
    readonly axial: Mm;
  };
};

type OperationIntent = PocketIntent | HoleIntent | SurfaceIntent | CustomIntent;

interface IntentProgram {
  readonly schemaVersion: string;
  readonly setup: ArtifactRef<SetupManifest>;
  readonly operations: readonly OperationIntent[];
}

```

B.0.0.6 Toolpath and schedule IR

```

interface PlannedPath {
  readonly id: string;
  readonly path: ArtifactRef<Path>;
  readonly operation: OperationId;
  readonly tool: ToolId;
  readonly phase: "entry" | "rough" | "semi-finish" | "finish" | "exit";
  readonly directionality:
    | { readonly kind: "reversible" }
    | { readonly kind: "fixed"; readonly reason: string };
  readonly feedPolicy: FeedPolicy;
  readonly provenance: Provenance;
}

```



```

}

interface ScheduledStep {
  readonly id: string;
  readonly beforeState: string;
  readonly afterState: string;
  readonly beforeStock: ArtifactRef;
  readonly afterStock: ArtifactRef;
  readonly command: CanonicalCommand;
  readonly dependencies: readonly string[];
}

interface ScheduledProgram {
  readonly setup: ArtifactRef<SetupManifest>;
  readonly steps: readonly ScheduledStep[];
  readonly finalState: string;
}

```

B.0.0.7 Canonical commands

```

type CanonicalCommand =
  | {
    readonly kind: "selectTool";
    readonly tool: ToolId;
  }
  | {
    readonly kind: "startSpindle";
    readonly rpm: Rpm;
    readonly direction: "cw" | "ccw";
  }
  | {
    readonly kind: "stopSpindle";
  }
  | {
    readonly kind: "traverse";
    readonly path: ArtifactRef<Path>;
    readonly clearanceClaim?: ClaimId;
  }
  | {
    readonly kind: "cut";
    readonly path: ArtifactRef<Path>;
    readonly tool: ToolId;
    readonly feed: FeedPolicy;
    readonly operation: OperationId;
  }
  | {
    readonly kind: "probe";
    readonly path: ArtifactRef<Path>;
    readonly expectedContact: ContactModel;
    readonly outputBinding: string;
  }
  | {
    readonly kind: "dwell";
    readonly duration: Seconds;
  }

```

```

    }
  | {
    readonly kind: "pause";
    readonly reason: string;
  };
};

```

B.0.0.8 Machine and controller IR

```

interface MachineCommandBase {
  readonly beforeToken: string;
  readonly afterToken: string;
  readonly provenance: Provenance;
}

type MachineCommand =
  | (MachineCommandBase & {
    readonly kind: "axisMove";
    readonly interpolation: "rapid" | "linear" | "cwArc" | "ccwArc";
    readonly target: Point3<"machine">;
    readonly center?: Point3<"machine">;
    readonly feed?: MmPerMin;
  })
  | (MachineCommandBase & {
    readonly kind: "setSpindle";
    readonly rpm: Rpm;
    readonly direction: "cw" | "ccw" | "off";
  })
  | (MachineCommandBase & {
    readonly kind: "probeMove";
    readonly direction: readonly [number, number, number];
    readonly maximumTravel: Mm;
    readonly feed: MmPerMin;
    readonly outputBinding: string;
  });

interface ControllerBlock {
  readonly words: readonly ControllerWord[];
  readonly comments: readonly string[];
  readonly provenance: Provenance;
}

interface ControllerProgram {
  readonly dialect: string;
  readonly declaredInitialState: ModalState;
  readonly blocks: readonly ControllerBlock[];
}

```

B.0.0.9 Diagnostics and provenance

```

interface SourceSpan {
  readonly artifact: Hash;
  readonly start: { readonly line: number; readonly column: number };
  readonly end: { readonly line: number; readonly column: number };
}

```

```

interface PassRecord {
  readonly passId: string;
  readonly passVersion: string;
  readonly relation: string;
  readonly inputHashes: readonly Hash[];
  readonly configurationHash: Hash;
}

interface Provenance {
  readonly source?: SourceSpan;
  readonly feature?: FeatureId;
  readonly operation?: OperationId;
  readonly parents: readonly Hash[];
  readonly passHistory: readonly PassRecord[];
}

interface Diagnostic {
  readonly code: string;
  readonly severity: "info" | "warning" | "error";
  readonly message: string;
  readonly provenance?: Provenance;
  readonly claim?: ClaimId;
  readonly counterexample?: ArtifactRef;
  readonly remediation?: string;
}

```

B.0.0.10 Quantitative claims

```

type ErrorMetric =
  | "hausdorff-position"
  | "normal-surface"
  | "max-gouge-depth"
  | "minimum-clearance"
  | "transform-translation"
  | "transform-rotation"
  | "axis-following";

interface ErrorBound {
  readonly metric: ErrorMetric;
  readonly value: number;
  readonly unit: "mm" | "rad";
  readonly frame?: FrameId;
  readonly subject?: Hash;
  readonly interpretation: "deterministic" | "probabilistic" | "empirical";
  readonly confidence?: number;
}

type StructuredPredicate =
  | {
    readonly kind: "disjoint";
    readonly left: Hash;
    readonly right: Hash;
    readonly minimumSeparation?: Mm;
  }

```

```

    }
  | {
    readonly kind: "max-penetration";
    readonly swept: Hash;
    readonly protectedTarget: Hash;
    readonly maximum: Mm;
    readonly metric: "signed-normal-depth" | "euclidean-depth";
  }
  | {
    readonly kind: "contains";
    readonly outer: Hash;
    readonly inner: Hash;
    readonly tolerance?: Mm;
  }
  | {
    readonly kind: "trace-refines";
    readonly source: Hash;
    readonly target: Hash;
    readonly relation: string;
  }
  | {
    readonly kind: "runtime-state-satisfies";
    readonly state: Hash;
    readonly assumptionSet: Hash;
  };
};

```

B.0.0.11 Certificate graph

```

interface CheckerIdentity {
  readonly id: string;
  readonly version: string;
  readonly binaryHash: Hash;
  readonly relationVersion: string;
}

interface Claim {
  readonly id: ClaimId;
  readonly subject: ArtifactRef;
  readonly proposition: StructuredPredicate;
  readonly result:
    | "proved-exact"
    | "proved-bounded"
    | "translation-validated"
    | "exhaustive-finite-check"
    | "simulation-only"
    | "assumed"
    | "unknown"
    | "refuted";
  readonly method: string;
  readonly assumptions: readonly AssumptionId[];
  readonly evidence: readonly ArtifactRef[];
  readonly dependencies: readonly ClaimId[];
  readonly bound?: ErrorBound;
  readonly checker: CheckerIdentity;
}

```

```
}

interface Assumption {
  readonly id: AssumptionId;
  readonly proposition: StructuredPredicate;
  readonly source: "operator" | "calibration" | "machine" | "library";
  readonly evidence?: ArtifactRef;
  readonly runtimeCheck?: string;
  readonly validUntil?: string;
}

interface CertificateGraph {
  readonly schemaVersion: string;
  readonly rootSubject: ArtifactRef;
  readonly artifacts: readonly ArtifactRef[];
  readonly assumptions: readonly Assumption[];
  readonly claims: readonly Claim[];
  readonly policy: string;
}
```

Appendix C

Checker Algorithms

This appendix gives reference algorithms. They emphasize the shape of soundness obligations rather than a particular implementation language.

C.0.0.1 Certificate graph checker

```
checkCertificate(graph, policy):
  validate schema and version compatibility
  verify every artifact hash against its bytes
  build maps for artifacts, assumptions, and claims

  for each claim:
    require unique claim id
    require subject exists and hash matches
    require all dependencies exist
    require all evidence artifacts exist
    require all assumptions exist
    require checker identity allowed by policy

  require claim dependency graph is acyclic
  process claims in topological order:
    require dependencies have acceptable results
    invoke claim-specific checker
    compare returned proposition, result, and bound
    reject if producer metadata overstates checker result

  require every policy-required proposition exists
  require no required claim is unknown, assumed-only, or simulation-only
  return accepted graph plus normalized summary
```

Soundness depends on claim-specific checkers and policy definitions. The graph checker prevents missing, stale, cyclic, or cross-applied evidence.

C.0.0.2 Translation-validation skeleton

```
validatePass(input I, output O, witness W, relation R):
  verify hash(I), hash(O), hash(W)
  verify schemas and pass versions
  switch R:
```

```

ExactTraceEquality:
    ti = interpret(I)
    to = interpret(0)
    return compareExact(ti, to)

TraceRefinement:
    return checkSimulationRelation(I, 0, W)

BoundedGeometry(metric, epsilon):
    return checkGeometricWitness(I, 0, W, metric, epsilon)

IntentSatisfaction:
    return checkIntentWitness(I, 0, W)

FeasibleOptimization:
    f = checkFeasibility(0, W.feasibility)
    j = recomputeObjective(0)
    l = checkLowerBound(W.lowerBound)
    return combine(f, j, l)

```

C.0.0.3 Abstract-state proof checker

```

checkAbstractTrace(program, proof):
    require proof has one entry per block
    state = abstractInitialState(program.initialAssumptions)

    for i in 0 .. program.blocks.length-1:
        require state is included in proof[i].before
        computed = abstractTransfer(program.blocks[i], proof[i].before)
        require computed is included in proof[i].after
        require localSafetyPredicates(proof[i].before, program.blocks[i])
        state = proof[i].after

    require finalPolicy(state)
    return success

```

“Included” uses the abstract-domain order. The producer may provide less precise states than the checker computes, but they must remain conservative.

C.0.0.4 Continuous collision checker

```

checkPath(path gamma, assembly T, obstacles 0, interval I, budget):
    P = boundPath(gamma, I)           // outer pose enclosure
    W = sweepEnclosure(P, T, uncertainty)

    if disjoint(W, outer(0)) with separation delta > 0:
        return ProvedSafe(I, delta)

    if definitelyIntersects(inner(W), inner(0)):
        return Refuted(I, counterexampleRegion)

    if width(I) <= budget.parameterLimit
       or diameter(P) <= budget.spatialLimit
       or recursionDepth >= budget.maxDepth:

```

```

    return Inconclusive(I, W)

split I into I1, I2
r1 = checkPath(gamma, T, 0, I1, budget)
r2 = checkPath(gamma, T, 0, I2, budget)
return combine(r1, r2)

```

The `definitelyIntersects` branch requires inner enclosures. If these are unavailable, the checker may prove safety or return inconclusive but cannot always prove collision.

C.0.0.5 Required-removal checker

```

checkRequiredRemoval(required R, cutMotions M):
    guaranteed = empty set
    for motion in M:
        innerSweep = computeInnerSweep(motion.tool, motion.path)
        guaranteed = union(guaranteed, innerSweep)

    residual = difference(R, guaranteed)
    if residual is empty by exact/conservative test:
        return proved
    if a definitely nonempty inner subset of residual is found:
        return refuted with residual witness
    return inconclusive with unresolved cells

```

C.0.0.6 Modal parse-back checker

```

checkFinalBytes(controllerIR C, bytes B, dialect D, initialState M0):
    parsed = independentParse(B, D)
    if parse error: reject

    sourceTrace = interpretControllerIR(C, M0)
    byteTrace = interpretParsedBlocks(parsed, M0)

    compare:
        event kinds and ordering
        tool/spindle/accessory state
        path endpoints and interpolation semantics
        probe command/result bindings
        final modal state
        numeric deviation against formatting budget

    verify required preamble and epilogue predicates
    return trace-equivalence or bounded-refinement claim

```

C.0.0.7 Schedule checker

```

checkSchedule(intent, candidate, witness):
    require candidate contains each required operation exactly once
    require no unknown operations

    positions = map operation -> index
    for each precedence edge (a,b):
        require positions[a] < positions[b]

```



```

state = initial process state
cost = 0
for each step:
    require step preconditions hold in state
    verify chosen orientation and entry
    verify link against referenced stock state
    state = apply certified effects(step, state)
    cost += recompute transition and operation cost

require state satisfies final policy
require cost == witness.objective within numeric bound
optionally verify lower-bound certificate

```

C.0.0.8 Error-budget composition

```

composeBound(inputBound e, passLocal p, sensitivity L, metric m):
    require e.metric == m or verified metric conversion exists
    require p.metric == m
    require L is a valid upper Lipschitz/sensitivity bound
    return L * e.value + p.value

```

For vector or transform bounds, use a matrix or interval propagation rule rather than a scalar function.

C.0.0.9 Runtime authorization checker

```

authorize(bundle B, liveState M, storedHash H'):
    verify certificate graph of B under requested operating policy
    require H' == B.jobHash
    require live machine identity matches B.machineProfile
    require firmware semantics compatible
    require no alarm and permitted controller state
    require homing state satisfies assumptions
    require active WCS lies inside certified transform set
    require tool and setup identity satisfy assumptions
    require calibration records valid

epoch = read controller state epoch
token = sign(jobHash, epoch, assumptionSnapshotHash, expiry)
return one-use authorization token

```

Execution consumes the token only if the state epoch remains unchanged.

Appendix D

A Temporal Controller Model

The following TLA+-style pseudocode illustrates a model suitable for refinement and model checking. It is not tied to a specific syntax version.

D.0.0.1 Variables

```
VARIABLES
  conn,                \* disconnected | connected
  identity,            \* unknown or machine/profile identity
  ctrlState,           \* idle | uploading | ready | running | held | alarm | quarantined
  storedHash,          \* hash or none
  executingHash,       \* hash or none
  authorizedHash,      \* hash or none
  stateEpoch,         \* monotonically increasing integer
  commandQueue,
  durableEvents,
  telemetry,
  homed,
  activeWcs,
  tool,
  alarm,
  lastOutcome
```

D.0.0.2 Initial state

```
Init ==
  /\ conn = "disconnected"
  /\ identity = "unknown"
  /\ ctrlState = "idle"
  /\ storedHash = None
  /\ executingHash = None
  /\ authorizedHash = None
  /\ stateEpoch = 0
  /\ commandQueue = << >>
  /\ durableEvents = << >>
  /\ telemetry = << >>
  /\ homed = FALSE
  /\ activeWcs = Unknown
  /\ tool = Unknown
```

```

/\ alarm = FALSE
/\ lastOutcome = None

```

D.0.0.3 Connection and identification

```

Connect ==
  /\ conn = "disconnected"
  /\ conn' = "connected"
  /\ identity' = "unknown"
  /\ UNCHANGED <<ctrlState, storedHash, executingHash, ...>>

```

```

Identify(id) ==
  /\ conn = "connected"
  /\ identity' = id
  /\ stateEpoch' = stateEpoch + 1
  /\ UNCHANGED <<conn, ctrlState, storedHash, ...>>

```

D.0.0.4 Upload

```

BeginUpload(h) ==
  /\ conn = "connected"
  /\ ctrlState = "idle"
  /\ ctrlState' = "uploading"
  /\ storedHash' = None
  /\ lastOutcome' = None
  /\ stateEpoch' = stateEpoch + 1

```

```

UploadSucceeded(h) ==
  /\ ctrlState = "uploading"
  /\ ctrlState' = "ready"
  /\ storedHash' = h
  /\ Append(durableEvents, [kind |-> "upload-ack", hash |-> h])
  /\ stateEpoch' = stateEpoch + 1

```

```

UploadAmbiguous ==
  /\ ctrlState = "uploading"
  /\ ctrlState' = "quarantined"
  /\ storedHash' = Unknown
  /\ lastOutcome' = "ambiguous-upload"
  /\ stateEpoch' = stateEpoch + 1

```

D.0.0.5 Authorization and start

```

Authorize(h, epoch) ==
  /\ ctrlState = "ready"
  /\ storedHash = h
  /\ epoch = stateEpoch
  /\ homed
  /\ ~alarm
  /\ RuntimeAssumptionsHold(identity, activeWcs, tool)
  /\ authorizedHash' = h
  /\ UNCHANGED <<ctrlState, executingHash, stateEpoch, ...>>

```

```

Start(h) ==

```

```

/\ ctrlState = "ready"
/\ storedHash = h
/\ authorizedHash = h
/\ ctrlState' = "running"
/\ executingHash' = h
/\ authorizedHash' = None          \* consume one-use authorization
/\ stateEpoch' = stateEpoch + 1

```

D.0.0.6 Hold, resume, abort, and alarm

FeedHold ==

```

/\ ctrlState = "running"
/\ ctrlState' = "held"
/\ stateEpoch' = stateEpoch + 1

```

Resume(h, epoch) ==

```

/\ ctrlState = "held"
/\ executingHash = h
/\ authorizedHash = h
/\ epoch = stateEpoch
/\ ctrlState' = "running"
/\ authorizedHash' = None
/\ stateEpoch' = stateEpoch + 1

```

Abort ==

```

/\ ctrlState \in {"running", "held", "uploading", "ready"}
/\ ctrlState' = "idle"
/\ executingHash' = None
/\ authorizedHash' = None
/\ Append(durableEvents, [kind |-> "aborted"])
/\ stateEpoch' = stateEpoch + 1

```

Fault(reason) ==

```

/\ ctrlState \in {"running", "held", "uploading", "ready"}
/\ ctrlState' = "alarm"
/\ alarm' = TRUE
/\ lastOutcome' = reason
/\ authorizedHash' = None
/\ stateEpoch' = stateEpoch + 1

```

D.0.0.7 Next-state relation

Next ==

```

\ / Connect
\ / \E id : Identify(id)
\ / \E h : BeginUpload(h)
\ / \E h : UploadSucceeded(h)
\ / UploadAmbiguous
\ / \E h, epoch : Authorize(h, epoch)
\ / \E h : Start(h)
\ / FeedHold
\ / \E h, epoch : Resume(h, epoch)
\ / Abort
\ / \E reason : Fault(reason)

```

```

\ / StatusUpdate
\ / Disconnect

```

D.0.0.8 Invariants

```

TypeInvariant ==
  ctrlState \in {"idle", "uploading", "ready", "running", "held", "alarm", "quarantined"}

RunningIsAuthorized ==
  ctrlState = "running" => executingHash # None

RunningMatchesStored ==
  ctrlState = "running" => executingHash = storedHash

NoMotionInAlarm ==
  alarm => ctrlState # "running"

UnknownUploadCannotRun ==
  storedHash = Unknown => ctrlState # "running"

OneUseAuthorization ==
  ctrlState = "running" => authorizedHash = None

```

D.0.0.9 Temporal properties

```

AbortEventuallyTerminal ==
  [] (AbortRequested => <> (ctrlState \in {"idle", "alarm", "quarantined"}))

UploadEventuallyResolves ==
  [] (ctrlState = "uploading" => <> (ctrlState \in {"ready", "alarm", "quarantined", "idle"}))

NoHashSubstitution ==
  [] (ctrlState = "running" => executingHash = storedHash)

```

These liveness properties require fairness and environment assumptions. A model checker should search for deadlocks, unauthorized transitions, stale-epoch authorizations, and dropped durable events.

Appendix E

Selected Solution Sketches

These are sketches, not the only valid answers. The purpose is to demonstrate the expected form of reasoning.

E.1 Chapter 1, Exercise 3

A probe can be modeled as a relation from an initial state to possible contact states. Nondeterminism comes from surface uncertainty, probe repeatability, controller sampling or debounce behavior, and the possibility of no contact before the travel limit. A useful result type therefore distinguishes `contact(positionInterval)`, `noContact`, `alarm`, and communication failure rather than returning one nominal number.

E.2 Chapter 1, Exercise 4

Let $p = 0$, $q = 0.75\varepsilon$, and $r = 1.5\varepsilon$ on the real line. Then $d(p, q) < \varepsilon$ and $d(q, r) < \varepsilon$, but $d(p, r) > \varepsilon$. Consequently, tolerance matching cannot serve as an equivalence relation without snapping, canonical representatives, symbolic identity, or explicit accumulated-error semantics.

E.3 Chapter 2, Exercise 3

A conceptual sequence is:

```
SelectTool<Idle, ToolLoaded<T1>>  
StartSpindle<ToolLoaded<T1>, ReadyToCut<T1>>  
Cut<ReadyToCut<T1>, ReadyToCut<T1>>  
StopSpindle<ReadyToCut<T1>, ToolLoaded<T1>>
```

`StopSpindle`; `Cut` fails because the post-state of `StopSpindle` does not match the required pre-state of `Cut`.

E.4 Chapter 2, Exercise 5

A planning-cache key must include at least the elaborated intent hash, target and stock geometry, tool and holder model, planner version, numerical-kernel version, tolerances, strategy parameters, frame graph, machine capability assumptions used by planning, and deterministic seed if randomized search is allowed. Omitting any semantic input risks reusing a witness for the wrong problem.

E.5 Chapter 3, Exercise 1

An arc-fitting relation can require identical endpoints, preserved traversal orientation, a Hausdorff deviation no greater than ε , and no change to action labels or process parameters. A checker can evaluate the fitted arc and original polyline with conservative segment/arc distance bounds and verify the endpoints exactly in a canonical numeric representation.

E.6 Chapter 3, Exercise 4

For collision absence, the checker must ensure the *true* sweep cannot reach an obstacle. If $W_{true} \subseteq W^+$ and $O_{true} \subseteq O^+$, then $W^+ \cap O^+ = \emptyset$ implies the true sets are disjoint. For guaranteed removal, the checker needs a set certainly removed: $R^- \subseteq R_{true}$. Required material contained in R^- is therefore guaranteed to be removed.

E.7 Chapter 4, Exercise 1

One safety property is that Running implies the executing hash equals the authorized hash for the current epoch. A second is that only a closed set of state-changing commands is admitted. A third is that Resume is accepted only from Held. Liveness properties can require accepted upload to reach either Stored or an explicit failure, accepted abort to reach Stopped or Faulted, and a held job to eventually produce either resume, abort, or operator-visible timeout under fairness assumptions.

E.8 Chapter 4, Exercise 4

A preview gate may accept simulation-only evidence but must label it. An attended air-cut gate should require exact-byte identity, machine-profile compatibility, homing and WCS checks, bounded travel, and closed protocol control. An attended material-cut gate additionally requires modeled stock, tool, fixture, target, spindle/process constraints, and operator setup confirmation. Unattended operation requires substantially stronger runtime monitoring, recovery semantics, validated physical models, and organizational controls.

Appendix F

Glossary

This glossary uses the meanings intended throughout the book. Several terms have broader meanings in compilers, formal methods, computational geometry, or manufacturing; the definitions below identify the interpretation relevant to a certificate-carrying CAM system.

Abstract domain. A mathematical domain whose elements conservatively represent sets of concrete program or machine states. Examples include intervals, axis-aligned boxes, finite sets of modal states, and symbolic frame relations.

Abstract interpretation. A framework for computing sound approximations of program behavior by interpreting a program over an abstract domain rather than enumerating concrete executions.

Abstraction function. A mapping from detailed physical or controller traces to the observations relevant at a higher semantic level, such as final stock, protected-surface penetration, or feature dimensions.

Acceptance gate. A machine-enforced condition that an artifact must satisfy before it can advance to a later lifecycle stage, such as preview, air cut, attended cutting, or production execution.

Acyclic dependency graph. A directed graph with no cycles. Certificate dependencies should normally form such a graph so that artifacts and claims can be checked in a finite topological order.

Admissible set. The subset of states, configurations, paths, or schedules satisfying all hard constraints.

Affine transform. A transform consisting of a linear map and a translation. Rigid frame transforms are the special case preserving distances and orientation or reflection, depending on the determinant.

Alarm state. A controller or machine state in which ordinary execution is blocked or modified because a fault, interlock, or exceptional condition has been detected.

Alias collision. Two distinct semantic objects receiving the same identifier or lookup key. In geometry this can merge unrelated endpoints, contours, cells, or topology components.

Artifact. An immutable, content-addressed input or output of a compiler stage: source, AST, IR, mesh, tool model, machine profile, controller program, serialized bytes, evidence, or certificate.

Artifact identity. The stable identity of an artifact, normally a cryptographic content hash plus a schema or media type. A filename or in-memory object address is not sufficient.

Assertion. A proposition intended to hold at one program point, controller state, geometric stage, or runtime checkpoint.

Assumption. An external fact required by a proof but not established by that proof. Tool measurement, machine calibration, fixture identity, controller conformance, and work-coordinate registration are typical assumptions.

Assumption discharge. The act of replacing an assumption with evidence or a runtime check that establishes it for a specific execution.

Attestation. Signed evidence about provenance, identity, configuration, or execution. Attestation authenticates a statement; it does not by itself establish the statement's mathematical truth.

Authorization epoch. A monotonically changing identifier binding an authorization decision to the machine state on which it was based. It prevents stale preflight results from authorizing later, changed states.

Backend. The portion of a compiler that lowers machine-independent operations to a target machine, controller dialect, serialization, or transport protocol.

Bounded refinement. A refinement relation that permits a quantified deviation under a named metric, such as Hausdorff distance, normal-direction surface error, or timing error.

Canonical command. A machine-oriented action with explicit machining meaning but without target-controller syntax or accidental modal dependence.

Certificate. A machine-checkable package binding claims, artifact identities, assumptions, evidence, dependency claims, checker identities, and quantitative bounds.

Certificate checker. A comparatively small program that validates evidence and decides whether a certificate claim follows for the named artifact under the named assumptions.

Certificate DAG. The dependency graph connecting source artifacts, transformed artifacts, claims, evidence, runtime facts, and final execution authorization.

Certified artifact. An artifact accompanied by one or more successfully checked claims. Certification is property-specific; it does not mean that every desirable property has been proved.

Checker independence. The degree to which a checker avoids sharing the producer's algorithms, data normalizations, hidden state, and defect modes. Independent implementation is often more valuable than nominal modular separation.

Claim. A precise proposition about a precise subject artifact, such as a travel-envelope inclusion or a maximum target-penetration bound.

Clearance. A distance or separation condition between the tool assembly and stock, target, fixtures, machine structures, or forbidden configuration-space regions.

Closed-world command language. A command language in which every permitted operation is explicitly enumerated and unknown operations are rejected. This is preferred for machine-control authorization.

Compiler pass. A transformation or analysis from one IR stage to another, ideally accompanied by an explicit semantic relation and a checker or proof obligation.

Concrete semantics. The detailed semantics of actual states and transitions, as opposed to an abstract analysis that represents sets of them.

Configuration space. A space whose points represent complete machine configurations. Collision of a moving solid can be transformed into point avoidance in an expanded forbidden region of configuration space.

Conformance. Satisfaction of a specification, standard, target intent, controller dialect, or certificate proposition.

Conservative approximation. An approximation oriented so that a successful check implies the desired property. Collision checks use outer obstacle and sweep enclosures; guaranteed-removal checks use inner removal enclosures.

Constraint. A hard condition defining feasibility. A safety constraint must not be converted into a soft objective penalty unless the resulting violations are categorically rejected later.

Content-addressing. Naming an artifact by a digest of its exact bytes and relevant encoding metadata, making changes observable and dependency invalidation mechanical.

Controller dialect. The syntax and operational interpretation implemented by a particular controller and firmware profile, including modal groups, commands, extensions, numeric ranges, and protocol behavior.

Controller IR. A structured representation of controller-level operations after machine lowering but before textual serialization.

Controller semantics profile. A versioned, content-addressed description of how a target firmware interprets commands, modes, uploads, acknowledgements, and lifecycle operations.

Counterexample. A concrete input, state, trace, geometry, or schedule showing that a claimed invariant or transformation relation is false.

Cutting sweep. The swept volume of the material-removing portion of the tool along a trajectory.

Cyber-physical compiler. A compiler whose target semantics includes physical mechanisms, continuous dynamics, sensors, actuators, material state, faults, and communication rather than only abstract machine instructions.

Denotation. The mathematical meaning assigned to a syntax element or IR object. A manufacturing operation often denotes a set of acceptable outcomes rather than one path.

Denotational semantics. A compositional mapping from programs or operations to mathematical objects such as functions, relations, traces, or sets of acceptable workpiece states.

Dexel. A depth element representing intervals of material along a ray. A triple-dexel model uses three orthogonal ray families.

Diagnostic. A structured explanation of an error, warning, unproved obligation, assumption, or source-to-output provenance relationship.

Dialect conversion. A checked transformation from one IR dialect or abstraction level to another under explicit legality and semantic-preservation rules.

Dimensional type. A type carrying physical dimension, such as length, angle, speed, acceleration, or spindle rate, so dimensionally invalid operations are rejected.

Dijkstra monad. A specification-indexed computational abstraction that associates effectful computations with predicate transformers or Hoare-style contracts.

Discretization. Replacement of a continuous object by finite samples, cells, segments, time steps, or basis coefficients. A discretization parameter alone is not a proof of continuous-domain error.

Effect summary. A conservative description of the state, resources, stock regions, probe values, frames, or modalities read and written by an operation.

Effect system. A static system that records and constrains computational or machine effects in addition to ordinary value types.

Elaboration. The pass that resolves names, defaults, units, frames, tools, parameters, and syntactic sugar into an explicit, typed representation.

Enclosure. A set guaranteed to contain an exact but potentially unknown quantity or geometry. Intervals and conservative swept volumes are enclosures.

End-to-end theorem. A statement relating checked source intent and assumptions to all permitted final physical executions, composed across every compiler, controller, and runtime boundary.

Evidence. Data consumed by a checker to establish a claim: proof terms, interval bounds, abstract states, parse-back traces, separating axes, spatial covers, solver certificates, or exhaustive state tables.

Exact claim. A claim established without an approximation tolerance relative to the stated mathematical input model. It can still depend on assumptions connecting the model to reality.

Execution instance. One uniquely identified controller run of one stored job under one machine-state epoch. A filename does not uniquely identify an execution instance.

Feasible set. The set of candidates satisfying all hard constraints. Optimization should search within this set rather than trade safety against cost.

Final-byte validation. Re-parsing and interpreting the exact serialized bytes that will be uploaded, then checking equivalence or refinement against the certified controller IR.

Frame. A coordinate system with an identity and transform relationships. Coordinates without frames are incomplete semantic values.

Frame graph. A graph of named coordinate frames and transforms, possibly with interval-valued uncertainty and time-varying measurements.

Free-space link. A non-cutting connection between machining segments whose tool-and-holder sweep is certified to avoid current stock, fixtures, protected geometry, and forbidden machine regions.

G-code block. One controller input record containing words evaluated under modal and dialect-specific semantics. Textual order inside a block does not necessarily imply sequential execution.

Gouge. Material removal beyond the allowed target or allowance region. A gouge claim requires a target or protected-material model.

Hash binding. Inclusion of an artifact hash in a claim, protocol step, authorization, or signature so the statement cannot silently transfer to different bytes.

Hausdorff distance. A metric measuring the greatest nearest-point deviation between two sets. It is useful for path approximation but does not alone encode orientation, process, or topology preservation.

Height field. A surface representation with one height per planar location. It is efficient for three-axis material models but cannot represent arbitrary undercuts or multiple layers.

Hoare triple. A contract of the form $\{P\} c \{Q\}$ stating that command c , when started in a state satisfying P , establishes Q if it terminates.

Hybrid system. A system combining discrete mode transitions with continuous state evolution, such as controller states plus machine position, velocity, and braking dynamics.

Identity morphism. In a path category, the zero-motion path at a pose. It acts as the neutral element for path composition under the chosen equality notion.

Information order. A partial order describing precision of abstract values, often written $a \sqsubseteq b$ when b contains at least as much uncertainty or information according to the selected convention.

Inner approximation. A set guaranteed to lie inside the true set. Inner removal approximations support claims that material was definitely removed.

Intent IR. A representation of manufacturing features, working steps, allowances, tolerances, resources, and dependencies before a specific tool-center path is selected.

Interlock. A condition preventing or terminating an operation when required safety or process state is absent.

Interval arithmetic. Arithmetic on closed intervals with outward rounding so the exact real-valued result remains enclosed.

Invariant. A property true initially and preserved by every relevant transition or compiler step.

IR legality. The syntactic, typing, capability, and semantic conditions an artifact must satisfy at a given intermediate representation level.

Job bundle. The complete execution package: exact program bytes, machine and firmware profiles, tool/holder models, stock/target/fixture artifacts, frame data, assumptions, certificates, and metadata.

Kleisli composition. Composition of effectful functions through a monad or indexed effect abstraction rather than ordinary function composition.

Language sandbox. An execution environment intended to restrict a macro or scripting language. A same-realm wrapper that retains access to host globals is not a security boundary.

Lattice. A partially ordered set in which every pair has a least upper bound and greatest lower bound. Many abstract domains use lattices to merge control-flow information and compute fixed points.

Legality checker. A checker deciding whether an IR contains only operations, types, frames, units, and capabilities permitted at that level.

Lipschitz bound. A constant L satisfying $d(f(x), f(y)) \leq Ld(x, y)$. It describes how input uncertainty or approximation can be amplified by a transformation.

Liveness property. A temporal property stating that some desired event eventually occurs, such as an abort eventually reaching a stopped or faulted state.

Lowering. Refinement from a higher-level IR to a more concrete IR by selecting machine, controller, geometry, or serialization details.

Machine envelope. The admissible set of axis configurations or tool poses under travel, soft-limit, fixture, and machine-structure constraints.

Machine IR. A representation after target-machine capabilities, frames, kinematics, limits, and trajectories have been made explicit but before controller syntax is serialized.

Machine profile. A versioned artifact describing kinematics, axis limits, rates, acceleration, jerk, spindle and tool capabilities, controller dialect, and other target constraints.

Macro language. A language executed at compile time to construct an inert AST or plan. It should not retain arbitrary executable closures inside trusted compiler passes.

Material-removal semantics. A semantics in which cutting commands transform stock solids, typically by set difference with a cutting sweep.

Metric. A distance function satisfying non-negativity, identity, symmetry, and triangle inequality. Approximation claims must name the metric under which their bounds hold.

Modal state. Controller state that persists across blocks until replaced, such as motion mode, units, plane, feed, spindle speed, or work offset.

Modal compression. Omission of redundant modal words during serialization. It is correct only if interpreted traces remain equivalent under the specified initial state and dialect.

Model checking. Exhaustive or symbolic exploration of a finite or finitely abstracted transition system to establish temporal properties or find counterexamples.

Monad. A compositional abstraction for computations with effects. For CAM, state, failure, nondeterminism, logging, and external interaction are relevant effects.

Monotone analysis. An analysis whose transfer functions preserve the abstract-domain order, enabling fixed-point computation and sound iteration.

Nondeterminism. Multiple possible successor states or traces arising from underspecified intent, sensor uncertainty, communication outcomes, faults, or environmental variation.

Numerical kernel. The small collection of arithmetic, predicates, interval operations, and geometric primitives trusted by certificate checkers.

Objective. A quantity minimized or maximized among feasible candidates, such as cycle time, rapid distance, tool changes, energy, or predicted wear.

Operational semantics. Rules describing how a configuration changes step by step as commands execute.

Operations research. Mathematical modeling and optimization of decisions such as operation sequencing, tool assignment, linking, and feed scheduling under constraints.

Outer approximation. A set guaranteed to contain the true set. Outer sweeps and obstacles support collision-absence and no-gouge claims.

Parse-back validation. Parsing emitted controller bytes back into structured IR and comparing their interpreted semantics with the pre-serialization artifact.

Partial correctness. A guarantee that the postcondition holds if execution terminates. It does not prove termination or eventual progress.

Pass contract. A specification of a pass's accepted input, produced output, semantic relation, error bound, assumptions, diagnostics, witness, and checker.

Path. A parameterized geometric curve through a frame. It does not by itself define timing, controller interpolation, or material-removal process.

Path equivalence. The chosen relation under which two path values are considered semantically equal, often equality modulo monotone reparameterization and a stated geometric tolerance.

Path parameterization. A monotone function mapping time to geometric path progress. It determines velocity, acceleration, jerk, and cycle time along a fixed path.

Planner. A generally complex and heuristic producer that chooses a toolpath, schedule, or process implementation satisfying an intent specification.

Postcondition. A proposition promised after an operation terminates from a state satisfying its precondition.

Postprocessor. A target-specific compiler backend lowering machine or canonical IR into a controller dialect and exact serialized job bytes.

Precondition. A proposition that must hold before an operation may execute or a theorem may be applied.

Predicate transformer. A function mapping desired postconditions backward to sufficient preconditions. Weakest-precondition semantics is a principal example.

Process model. A model of cutting, probing, spindle, stock, tool engagement, or machine dynamics used to interpret and check operations.

Proof obligation. A proposition that must be established before a pass result, job bundle, or runtime action can be accepted.

Proof-producing analysis. An analyzer that emits evidence allowing a smaller checker to validate its result rather than requiring the analyzer itself to be trusted.

Protected material. Target or fixture material that a claim forbids the cutting sweep from entering beyond a stated tolerance.

Provenance. Structured information linking output objects and diagnostics to source spans, compiler passes, parameters, input artifacts, and tool versions.

Quantization. Mapping continuous or high-precision values to discrete integer bins. Quantization can be useful, but a lossy packed key must not be mistaken for unique topology identity.

Rapid safety. A claim that a non-cutting motion's tool-and-holder sweep avoids current stock and obstacles under controller rapid semantics and machine uncertainty.

Raw command. An escape hatch containing target syntax whose effects have not been derived from a trusted parser. Conservative analyses must reject it or set affected abstract state to unknown.

Reachability. The set of states obtainable from an initial state under the transition relation and assumptions.

Reference semantics. A simple, explicit semantic implementation used as the standard against which optimized or target-specific implementations are compared.

Refinement. A relation in which a concrete implementation removes choices or adds detail without introducing behavior forbidden by the abstract specification.

Refinement mapping. A mapping from concrete states and traces to abstract states and traces used to demonstrate that one system implements another.

Representation invariant. A property ensuring an in-memory value is internally coherent, such as a path endpoint matching its last segment. It is narrower than a machining-safety claim.

Residual stock. Material remaining after the modeled removal operations. Conservative residual stock usually requires outer stock and inner removal approximations.

Robust predicate. A predicate whose result is correct for the mathematical input model despite floating-point degeneracy, normally through filtered, adaptive, exact, or interval methods.

Runtime assurance. A small trusted mechanism that monitors an advanced component or physical system and intervenes before a safety property can be violated.

Safety property. A temporal property stating that a bad event never occurs. A finite violation has a counterexample prefix.

Semantic preservation. Equality, refinement, or bounded correspondence between the meanings of a pass input and output.

Semantic waist. A compact intermediate language separating many high-level producers from many low-level targets while retaining the physical distinctions needed for correctness.

Serialization. Conversion of structured controller IR to exact bytes, including number formatting, line endings, comments, checksums, and encoding.

Signature. A cryptographic authentication of bytes or statements. A signature establishes origin and integrity under a key-management assumption, not semantic correctness.

Simulation-only result. Evidence from one or more concrete executions or samples without a theorem covering all relevant states or the continuous domain.

Small-step semantics. An operational semantics in which each rule performs one local transition, making interleavings, faults, and protocol states explicit.

Soundness. The property that every fact reported as established is true of all represented concrete behaviors under the stated assumptions. A sound analysis may conservatively reject safe programs.

Source map. A mapping from generated IR nodes, commands, and diagnostics back to source syntax and macro-expansion provenance.

SSA state token. A single-assignment value consumed and produced by state-changing IR operations, making effect order and data dependencies explicit.

Staged programming. Execution of one program phase to construct code or data for a later phase. The phase boundary should remove ambient authority and retain only inert, validated artifacts.

Stock monotonicity. The ideal subtractive-process invariant $S_{i+1} \subseteq S_i$. It excludes material addition but does not by itself establish correct removal.

Swept volume. The union of all poses of a solid along a trajectory. Different claims require the cutting tool sweep or the complete tool-and-holder assembly sweep.

Target surface. The desired final boundary or protected part geometry against which gouge, allowance, and residual-material claims are stated.

Temporal logic. A logic with operators over traces, such as always $\Box$, eventually $\Diamond$, and next, used for controller safety and liveness properties.

Time-optimal path parameterization. Selection of progress $s(t)$ along a fixed geometric path to minimize time while satisfying velocity, acceleration, torque, jerk, tracking, and process constraints.

Total correctness. Partial correctness plus termination or liveness under stated assumptions.

Trace. A finite or infinite sequence of states and observable events representing one system execution.

Trace refinement. Inclusion of concrete traces, after abstraction, in the set allowed by an abstract program.

Trajectory. A time-indexed pose or configuration function. A path becomes a trajectory only after time parameterization.

Translation validation. Independent checking that one actual pass output correctly refines its actual input, instead of proving the transformer correct for all inputs.

Trusted computing base. The code, semantics, numeric primitives, key material, and assumptions whose correctness must be trusted for the assurance argument to hold.

Typestate. Static representation of protocol state in types so only operations valid in the current state can be expressed.

Uncertainty set. A set of possible values for a physical or numerical quantity, such as a transform interval, tool-radius interval, or following-error bound.

Validation. Checking an artifact against a relation, policy, or specification. Validation should state exactly what was checked and by what method.

Verification. Establishment of a formal or mathematically justified claim. In this book the term is reserved for claims with explicit propositions, assumptions, and evidence rather than visual inspection alone.

Weakest precondition. The least restrictive precondition sufficient to ensure a desired postcondition after execution of a command.

Witness. Producer-supplied data demonstrating a construction or optimization choice, such as a path decomposition, correspondence, abstract invariant, separating plane, or schedule assignment.

Work coordinate system. A frame relating part-program coordinates to machine coordinates. Its identity and uncertainty are runtime-critical inputs to travel and collision claims.

Appendix G

Guided Reading and Project Tracks

The four chapters can be read linearly, but implementation teams often need a narrower path.

G.1 Compiler and language track

Read Chapter 1’s refinement section, all of Chapter 2, and Chapter 3 through serialization and translation validation. Implement the inert macro-to-AST boundary, the IR ladder, state tokens, content-addressed artifacts, a reference semantics, and final-byte parse-back before adding advanced planners.

G.2 Verification and geometry track

Read Chapter 1’s mathematical and semantic sections, then Chapter 3 from contracts through trusted checkers. Implement robust topology predicates, interval transforms, separate inner/outer stock models, typed error budgets, and property-specific certificate claims.

G.3 Controller and runtime track

Read Chapter 1’s operational and trace semantics, Chapter 2’s effects and identity sections, and Chapter 4’s protocol/runtime section. Implement a closed command grammar, explicit state machine, state epochs, hash-bound authorization, abort recovery, and a small monitor.

G.4 Planning and optimization track

Read the intent and path material in Chapters 1 and 2, the planning/linking and robust-geometry material in Chapter 3, and the operations-research material in Chapter 4. Treat every optimizer as an untrusted candidate producer whose output must pass a separate feasibility checker.

G.5 Twelve-week implementation course

Week	Reading	Deliverable
1	Chapter 1: intent	Specify one pocket and three acceptable implementations.
2	Chapter 1: math and frames	Implement dimensional types and a checked frame graph.
3	Chapter 1: semantics	Write a reference semantics for five canonical commands.
4	Chapter 2: staging and IR	Build an isolated JavaScript-to-AST boundary and IR schemas.
5	Chapter 2: effects and identity	Add state tokens, provenance, and content-addressed artifacts.
6	Chapter 3: pass contracts	Implement one certifying pass and translation validator.
7	Chapter 3: lowering	Lower a pocket to controller IR and validate bytes by parse-back.
8	Chapter 3: analysis	Add weakest-precondition rules and a modal abstract interpreter.
9	Chapter 3: geometry	Add robust predicates, interval transforms, and error budgets.
10	Chapter 3/4: certificates and protocol	Build a certificate DAG and hash-bound runtime handshake.
11	Chapter 4: optimization	Add precedence-aware sequencing and a checked linker.
12	Chapter 4: integration	Run the complete dry-run and attended air-cut acceptance ladder.

References

The bracketed identifiers in the text refer to the entries below. DOI links name the version of record when available; institutional pages are used for standards and technical reports.

The bracketed identifiers in the text refer to the entries below. DOI links name the version of record when available; institutional pages are used for standards and technical reports.

[R1] Thomas R. Kramer, Frederick M. Proctor, and Elena R. Messina. *The NIST RS274/NGC Interpreter, Version 3*. NISTIR 6556, National Institute of Standards and Technology, 2000. <https://www.nist.gov/publications/nist-rs274ngc-interpreter-version-3>.

[R2] Frederick M. Proctor, Thomas R. Kramer, and John L. Michaloski. *Canonical Machining Commands*. NISTIR 5970, National Institute of Standards and Technology, 1997. <https://doi.org/10.6028/NIST.IR.5970>.

[R3] ISO 14649-1. *Industrial Automation Systems and Integration – Physical Device Control – Data Model for Computerized Numerical Controllers – Part 1: Overview and Fundamental Principles*. International Organization for Standardization.

[R4] C. A. R. Hoare. “An Axiomatic Basis for Computer Programming.” *Communications of the ACM* 12, no. 10 (1969): 576-580, 583. <https://doi.org/10.1145/363235.363259>.

[R5] Edsger W. Dijkstra. “Guarded Commands, Nondeterminacy and Formal Derivation of Programs.” *Communications of the ACM* 18, no. 8 (1975): 453-457. <https://doi.org/10.1145/360933.360975>.

[R6] Gordon D. Plotkin. “A Structural Approach to Operational Semantics.” Originally DAIMI FN-19, Aarhus University, 1981; reprinted in *Journal of Logic and Algebraic Programming* 60-61 (2004): 17-139. <https://doi.org/10.1016/j.jlap.2004.03.009>.

[R7] Patrick Cousot and Radhia Cousot. “Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints.” In *Proceedings of POPL 1977*, 238-252. <https://doi.org/10.1145/512950.512973>.

[R8] Leslie Lamport. “The Temporal Logic of Actions.” *ACM Transactions on Programming Languages and Systems* 16, no. 3 (1994): 872-923. <https://doi.org/10.1145/177492.177726>.

[R9] Martín Abadi and Leslie Lamport. “The Existence of Refinement Mappings.” *Theoretical Computer Science* 82, no. 2 (1991): 253-284. [https://doi.org/10.1016/0304-3975\(91\)90224-P](https://doi.org/10.1016/0304-3975(91)90224-P).

[R10] Eugenio Moggi. “Notions of Computation and Monads.” *Information and Computation* 93, no. 1 (1991): 55-92. [https://doi.org/10.1016/0890-5401\(91\)90052-4](https://doi.org/10.1016/0890-5401(91)90052-4).

[R11] Philip Wadler. “The Essence of Functional Programming.” In *Proceedings of POPL 1992*, 1-14. <https://doi.org/10.1145/143165.143169>.

- [R12] Robert E. Strom and Shaula Yemini. “Typestate: A Programming Language Concept for Enhancing Software Reliability.” *IEEE Transactions on Software Engineering* SE-12, no. 1 (1986): 157-171. <https://doi.org/10.1109/TSE.1986.6312929>.
- [R13] Robert Atkey. “Parameterised Notions of Computation.” *Journal of Functional Programming* 19, nos. 3-4 (2009): 335-376. <https://doi.org/10.1017/S095679680900728X>.
- [R14] John M. Lucassen and David K. Gifford. “Polymorphic Effect Systems.” In *Proceedings of POPL 1988*, 47-57. <https://doi.org/10.1145/73560.73564>.
- [R15] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. “Efficiently Computing Static Single Assignment Form and the Control Dependence Graph.” *ACM Transactions on Programming Languages and Systems* 13, no. 4 (1991): 451-490. <https://doi.org/10.1145/115372.115320>.
- [R16] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. “MLIR: Scaling Compiler Infrastructure for Domain Specific Computation.” In *Proceedings of CGO 2021*, 2-14. <https://doi.org/10.1109/CGO51591.2021.9370308>.
- [R17] Xavier Leroy. “Formal Verification of a Realistic Compiler.” *Communications of the ACM* 52, no. 7 (2009): 107-115. <https://doi.org/10.1145/1538788.1538814>.
- [R18] Amir Pnueli, Michael Siegel, and Eli Singerman. “Translation Validation.” In *Tools and Algorithms for the Construction and Analysis of Systems, TACAS 1998*, 151-166. <https://doi.org/10.1007/BFb0054170>.
- [R19] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. “Alive2: Bounded Translation Validation for LLVM.” In *Proceedings of PLDI 2021*. <https://doi.org/10.1145/3453483.3454030>.
- [R20] George C. Necula. “Proof-Carrying Code.” In *Proceedings of POPL 1997*, 106-119. <https://doi.org/10.1145/263699.263712>.
- [R21] Andrew W. Appel. “Foundational Proof-Carrying Code.” In *Proceedings of the 16th Annual IEEE Symposium on Logic in Computer Science*, 247-256, 2001. <https://doi.org/10.1109/LICS.2001.932501>.
- [R22] Jonathan Richard Shewchuk. “Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates.” *Discrete & Computational Geometry* 18 (1997): 305-363. <https://doi.org/10.1007/PL00009321>.
- [R23] IEEE. *IEEE Standard for Interval Arithmetic (Simplified)*. IEEE Std 1788.1-2017. <https://standards.ieee.org/ieee/1788.1/6074/>.
- [R24] Ramon E. Moore, R. Baker Kearfott, and Michael J. Cloud. *Introduction to Interval Analysis*. SIAM, 2009. <https://doi.org/10.1137/1.9780898717716>.
- [R25] Tomás Lozano-Pérez. “Spatial Planning: A Configuration Space Approach.” *IEEE Transactions on Computers* C-32, no. 2 (1983): 108-120. <https://doi.org/10.1109/TC.1983.1676196>.
- [R26] Silvia Sellán, Noam Aigerman, and Alec Jacobson. “Swept Volumes via Spacetime Numerical Continuation.” *ACM Transactions on Graphics* 40, no. 4 (2021). <https://doi.org/10.1145/3450626.3459780>.
- [R27] Masatomo Inui, Takashi Sakurai, and Nobuyuki Umezu. “Data Conversion Technology between Triple Dixel Model and Polygonal Model.” *Journal of the Japan Society for Precision Engineering* 76, no. 2 (2010): 226-231. <https://doi.org/10.2493/jjspe.76.226>.
- [R28] Weihang Zhang and Ming-Chuan Leu. “Surface Reconstruction Using Dixel Data from Three Sets of Orthogonal Rays.” *Journal of Computing and Information Science in Engineering* 9, no. 1 (2009): 011008. <https://doi.org/10.1115/1.3086034>.

- [R29] J. A. Sethian. “A Fast Marching Level Set Method for Monotonically Advancing Fronts.” *Proceedings of the National Academy of Sciences* 93, no. 4 (1996): 1591-1595. <https://doi.org/10.1073/pnas.93.4.1591>.
- [R30] Ron Kimmel and J. A. Sethian. “Computing Geodesic Paths on Manifolds.” *Proceedings of the National Academy of Sciences* 95, no. 15 (1998): 8431-8435. <https://doi.org/10.1073/pnas.95.15.8431>.
- [R31] Ilker Kucukoglu, Tulin Gunduz, Fatma Balkancioglu, Emine Chousein Topal, and Oznur Sayim. “Application of Precedence Constrained Travelling Salesman Problem Model for Tool Path Optimization in CNC Milling Machines.” *An International Journal of Optimization and Control: Theories & Applications* 9, no. 3 (2019): 59-68. <https://doi.org/10.11121/ijocta.01.2019.00662>.
- [R32] Qiang Zhang, Shurong Li, and Jianxin Guo. “Minimum Time Trajectory Optimization of CNC Machining with Tracking Error Constraints.” *Abstract and Applied Analysis* 2014, Article 835098. <https://doi.org/10.1155/2014/835098>.
- [R33] Hung Pham and Quang-Cuong Pham. “A New Approach to Time-Optimal Path Parameterization Based on Reachability Analysis.” *IEEE Transactions on Robotics* 34, no. 3 (2018): 645-659. <https://doi.org/10.1109/TRO.2018.2819195>.
- [R34] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. <https://lamport.azurewebsites.net/tla/book.html>.
- [R35] Rajeev Alur, Costas Courcoubetis, Thomas A. Henzinger, and Pei-Hsin Ho. “The Algorithmic Analysis of Hybrid Systems.” *Theoretical Computer Science* 138, no. 1 (1995): 3-34. [https://doi.org/10.1016/0304-3975\(94\)00202-T](https://doi.org/10.1016/0304-3975(94)00202-T).
- [R36] J. Tanner Slagel, Lauren M. White, Aaron Dutle, César A. Muñoz, and Nicolas Crespo. “A Formal Verification Framework for Runtime Assurance.” In *NASA Formal Methods 2024*, 322-328. https://doi.org/10.1007/978-3-031-60698-4_19.
- [R37] Walid Taha and Tim Sheard. “MetaML and Multi-Stage Programming with Explicit Annotations.” *Theoretical Computer Science* 248, nos. 1-2 (2000): 211-242. [https://doi.org/10.1016/S0304-3975\(00\)00053-0](https://doi.org/10.1016/S0304-3975(00)00053-0).
- [R38] John C. Reynolds. “Separation Logic: A Logic for Shared Mutable Data Structures.” In *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science*, 55-74, 2002. <https://doi.org/10.1109/LICS.2002.1029817>.
- [R39] Kenji Maillard, Danel Ahman, Robert Atkey, Guido Martínez, Cătălin Hrițcu, Exequiel Rivas, and Éric Tanter. “Dijkstra Monads for All.” *Proceedings of the ACM on Programming Languages* 3, ICFP, Article 104 (2019). <https://doi.org/10.1145/3341708>.
- [R40] Allison Barnard Feeney and Simon P. Frechette. “Testing STEP-NC Implementations.” In *Proceedings of World Automation Congress 2002*. National Institute of Standards and Technology. <https://www.nist.gov/publications/testing-step-nc-implementations>.
- [R41] George C. Necula and Peter Lee. “Efficient Representation and Validation of Proofs.” In *Proceedings of the 13th Annual IEEE Symposium on Logic in Computer Science*, 93-104, 1998. <https://doi.org/10.1109/LICS.1998.705646>.
- [R42] Thomas R. Kramer and Frederick M. Proctor. *Feature-Based Control of a Machining Center*. NISTIR 5926, National Institute of Standards and Technology, 1996. <https://www.nist.gov/publications/feature-based-control-machining-center>.
- [R43] John L. Michaloski, Thomas R. Kramer, Frederick M. Proctor, Xun Xu, Sid Venkatesh, and David Odendahl.

“STEPNC++ – An Effective Tool for Feature-Based CAM/CNC.” In *Advanced Design and Manufacturing Based on STEP*. Springer, 2009. <https://www.nist.gov/publications/stepnc-effective-tool-feature-based-camcnc>.

Source Snapshot and Reproducibility Notes

The implementation-specific case study is based on the supplied `dropcut-studio.zip` snapshot and the public repository branch `task/cnc-control-dropcut` near commit `e82bed1e5a00f38f4441e6ea13e1265edc775928`, inspected in August 2026. The case study is not a statement about later revisions. Where firmware behavior is discussed, the reviewed source evidence used the Makera stock Carvera firmware repository at commit `1683b6fb5c7ec1d341c476c6fdb2a22f7a26220e` and explicitly distinguished stock behavior from community-firmware changes.

The textbook's diagrams are generated from Graphviz or Matplotlib source stored with the Markdown edition. The PDF is rendered from the same Markdown source. Equations, algorithms, and TypeScript interfaces are pedagogical specifications; they must be implemented, tested, and connected to machine-specific assumptions before they can support a production certificate.